



Univerza v Mariboru



Fakulteta za elektrotehniko,
računalništvo in informatiko

Gordana Arsova

Analiza nelinearnih sistemov vodenja s strojno opremo v zanki

Hardware-in-the-Loop Analysis of Nonlinear Control Systems

Magistrsko delo

Maribor, Marec 2026



Univerza v Mariboru



Fakulteta za elektrotehniko,
računalništvo in informatiko

Gordana Arsova

Analiza nelinearnih sistemov vodenja s strojno opremo v zanki

Hardware-in-the-Loop Analysis of Nonlinear Control Systems

Magistrsko delo

Maribor, Marec 2026

Analiza nelinearnih sistemov vodenja s strojno opremo v zanki

Hardware-in-the-Loop Analysis of Nonlinear Control Systems

Magistrsko delo

Študentka:	Gordana Arsova, dipl.inž.el
Študijski program:	Študijski program 2. stopnje Elektrotehnika
Smer:	Avtomatika in robotika
Mentor:	izr. prof. dr. Andrej Sarjaš, univ. dipl. inž. el
Lektor:	Arianna Posega

CC BY-SA 4.0

ZAHVALA

Iskreno se zahvaljujem svojemu mentorju, prof. dr. Andreju Sarjašu, za njegovo strokovno vodenje, potrpežljivost in stalno podporo skozi celoten proces izdelave magistrskega dela. Njegovo znanje, predanost in pripravljenost pomagati so bili ključnega pomena za uspešen zaključek tega dela.

Zahvaljujem se tudi vsem profesorjem na FERi, ki so prispevali k mojemu izobraževanju ter mi podali dragoceno znanje in širši pogled na področje inženirstva. Vsako predavanje, vaje in laboratorijsko delo sem iskreno uživala in hvala vam, ker ste zgled tega, kakšen naj bo dober inženir. Vse profesorje zelo cenim kot pedagoge, inženirje in kot ljudi ter si želim slediti vašemu zgledu.

Moji družini – mojim staršem, sestri in bratu – se zahvaljujem za neomajno podporo, razumevanje in spodbudo skozi celoten študij. Brez vas danes ne bi bila tukaj. Oseba, kakršna sem danes, sem postala zaradi vas in upam, da ste ponosni name.

Na koncu bi se rada zahvalila še vsem svojim prijateljem in kolegom, tako v Makedoniji kot v Sloveniji, za podporo, prijateljstvo in vse skupne trenutke, ki so to pot naredili ne le možno, temveč tudi nepozabno.

Analiza nelinearnih sistemov vodenja s strojno opremo v zanki

Ključne besede: strojna oprema v zanki, backstepping regulator, regulacija drsnega režima, kaskadna regulacija

UDK: 681.511.4(043.2)

Povzetek

V magistrskem delu je predstavljen pristop strojne opreme v zanki (Hardware-in-the-Loop, HIL) za implementacijo in validacijo adaptivnega vodenja nelinearnih sistemov. Nelinearni sistemi so v sodobnih inženirskih aplikacijah zelo pogosti, njihovo vodenje pa predstavlja velik izziv zaradi kompleksne dinamike, neznanih parametrov in prisotnosti motenj. Namen naloge je raziskati učinkovitost različnih naprednih regulacijskih metod ter preveriti njihovo delovanje v realno-časovnem okolju s pomočjo HIL pristopa.

V okviru naloge so obravnavane različne regulacijske metode, ki se pogosto uporabljajo za vodenje nelinearnih sistemov. Med njimi so adaptivna backstepping regulacija, regulacija drsnega režima (Sliding Mode Control – SMC) ter klasična proporcionalno-derivativna (PD) regulacija. Poleg teh metod je predlagan tudi hibridni pristop, ki združuje prednosti posameznih regulatorjev z namenom izboljšanja stabilnosti, robustnosti in odzivnosti sistema. Poseben

poudarek je namenjen primerjavi teh metod z vidika natančnosti sledenja referenčnemu signalu, odpornosti na motnje ter občutljivosti na spremembe parametrov sistema.

Za potrebe simulacije je brezkrtačni enosmerni motor (BLDC) modeliran kot sistem drugega reda v okolju MATLAB/Simulink. Takšna poenostavitev omogoča lažjo implementacijo regulacijskih algoritmov in analizo njihovega delovanja, hkrati pa ohranja ključne dinamične lastnosti sistema. V model je vključen tudi neznan oziroma spremenljiv parameter, ki simulira negotovosti v realnem sistemu in omogoča preizkus adaptivnih lastnosti regulatorjev.

Regulacijski algoritmi so implementirani na industrijskem krmilniku Beckhoff PLC, kar omogoča izvajanje v realnem času. Komunikacija med simulacijskim okoljem in krmilnikom je vzpostavljena preko ADS (Automation Device Specification) protokola. Ta omogoča prenos podatkov med Simulink modelom in PLC-jem, vendar hkrati uvaja določeno zakasnitev, ki lahko vpliva na stabilnost in zmogljivost regulacijskega sistema. V nalogi je zato analiziran tudi vpliv komunikacijske latence na delovanje posameznih regulatorjev.

Za vrednotenje uspešnosti regulacijskih metod so uporabljeni različni kriteriji, kot so povprečna kvadratna napaka (RMSE), integral absolutne napake (IAE), integral kvadratne napake (ISE), maksimalno odstopanje ter čas ustalitve sistema. Poleg tega je analizirano tudi obnašanje sistema ob prisotnosti zunanjih motenj, kar omogoča oceno robustnosti posameznih pristopov. Rezultati kažejo, da adaptivne metode in nelinearni regulatorji, kot sta backstepping in SMC, dosegajo boljše rezultate v primerjavi s klasično PD regulacijo, predvsem v pogojih negotovosti in motenj.

Predlagani hibridni pristop izkazuje dodatne prednosti, saj združuje stabilnost in enostavnost PD regulacije z robustnostjo nelinearnih metod. S tem omogoča boljše prilagajanje spremembam v sistemu ter izboljšano dinamično odzivnost. Kljub temu pa kompleksnost implementacije in nastavitve parametrov predstavlja izziv, ki ga je potrebno upoštevati pri praktični uporabi.

Pomemben prispevek naloge je tudi demonstracija uporabnosti HIL pristopa za validacijo naprednih regulacijskih strategij. HIL omogoča testiranje regulatorjev v realno-časovnem okolju brez potrebe po neposredni uporabi fizičnega sistema, kar zmanjšuje stroške in tveganja. Poleg tega omogoča lažjo identifikacijo težav, povezanih z implementacijo, kot so zakasnitve, diskretizacija in omejitve strojne opreme.

Na koncu naloga potrjuje, da je HIL pristop učinkovito orodje za razvoj in testiranje regulacijskih algoritmov, zlasti pri kompleksnih nelinearnih sistemih. Rezultati kažejo, da kombinacija simulacijskega modela in realno-časovne implementacije omogoča zanesljivo oceno zmogljivosti regulatorjev ter predstavlja pomemben korak proti njihovi uporabi v realnih industrijskih aplikacijah.

Hardware-in-the-Loop Analysis of Nonlinear Control Systems

Key words: Hardware-in-the-loop, backstepping regulator, SMC regulation, cascade regulation

UDK: 681.511.4(043.2)

Abstract

This master's thesis presents a hardware-in-the-loop (HIL) framework for the implementation of adaptive control of nonlinear systems. Several control strategies are investigated, such as backstepping, sliding mode control (SMC), PD control, and a proposed hybrid approach. For simulation purposes, a brushless DC motor is modeled as a second-order system in Simulink, while the control algorithms are executed on a Beckhoff Programmable Logic Controller communicating via the ADS protocol. The study evaluates the performance and robustness of the implemented methods, demonstrating the effectiveness of the HIL approach for real-time validation of advanced control strategies.



Fakulteta za elektrotehniko, računalništvo in informatiko
(ime članice UM)

IZJAVA O AVTORSTVU ZAKLJUČNEGA DELA

Ime in priimek študenta/-ke: Gordana Arsova

Študijski program: ELEKTROTEHNIKA

Naslov zaključnega dela: Analiza nelinearnih regulacijskih sistemov s strojno opremo v zanki

Mentor/-ica: Andrej Sarjaš

Somentor/-ica: _____

Podpisani/-a študent/-ka Gordana Arsova

- izjavljam, da je zaključno delo rezultat mojega samostojnega dela, ki sem ga izdelal/-a ob pomoči mentorja/-ice oz. somentorja/-ice;
- izjavljam, da sem pridobil/-a vsa potrebna soglasja za uporabo podatkov in avtorskih del v zaključnem delu in jih v zaključnem delu jasno in ustrezno označil/-a;
- na Univerzo v Mariboru neodplačno, neizključno, prostorsko in časovno neomejeno prenašam pravico shranitve avtorskega dela v elektronski obliki, pravico reproduciranja ter pravico ponuditi zaključno delo javnosti na svetovnem spletu preko DKUM; sem seznanjen/-a, da bodo dela, deponirana/objavljena v DKUM, dostopna široki javnosti pod pogoji licence Creative Commons BY-NC-ND, kar vključuje tudi avtomatizirano indeksiranje preko spleta in obdelavo besedil za potrebe tekstovnega in podatkovnega rudarjenja in ekstrakcije znanja iz vsebin; uporabnikom se dovoli reproduciranje brez predelave avtorskega dela, distribuiranje, dajanje v najem in priobčitev javnosti samega izvirnega avtorskega dela, in sicer pod pogojem, da navedejo avtorja in da ne gre za komercialno uporabo;
- dovoljujem objavo svojih osebnih podatkov, ki so navedeni v zaključnem delu in tej izjavi, skupaj z objavo zaključnega dela.

Uveljavljam permissivnejšo obliko licence Creative Commons: CC BY-SA 4.0
(navedite obliko)

Kraj in datum:

Maribor, 23.03.2026

Podpis študenta/-ke:

G. Arsova

Table of Contents

1. Introduction.....	1
2. Origins of hardware-in-the-loop simulation.....	3
2.1 Application of hardware-in-the-loop	6
3. Nonlinear control algorithms	7
3.1 Adaptive backstepping regulator	7
3.2. Adaptive sliding mode control regulator	10
3.3 Hybrid backstepping regulator with an added sliding surface	14
3.4 PD regulation with nonlinear compensation	15
4. Theoretical background for BLDC motors	17
4.1 Fundamentals of motor theory	18
4.2 Classification of electric motors.....	20
4.3 Brushless DC Motors	22
4.4 Brushless DC motor and model concept.....	31
3.5 Modeling of the BLDC motor in MATLAB	35
4.6 Cascade control structure in BLDC Motor Drives	36
5. System analysis of the BLDC motor model	38
5.1 Open-loop equilibrium analysis.....	38
6. Derivation of control algorithms for BLDC motor model.....	41
6.1 Adaptive backstepping regulator	41
6.2 Adaptive sliding mode control (SMC) regulator	45
6.3 Hybrid approach of backstepping regulator with added sliding surface	46
6.4 PD Regulation with nonlinear compensation	48
7. Hardware-in-the-loop Structure	50
7.1 Simulink subsystems.....	51
7.2 Automation device specification (ADS) protocol.....	53
8. Implementation of control algorithms on Beckhoff PLC	60

9. Results and analysis	62
9.1 High update-rate simulation results.....	63
9.2 Low update-rate simulation results	68
9.3 Low update rate simulation with an added disturbance.....	71
9.4 Analysis	74
10. Conclusion	78

Table of Figures

Figure 2.1 Common mechatronics system	4
Figure 2.2 Signal Hardware-in-the-loop system	5
Figure 2.3 Power Hardware-in-the-loop system	5
Figure 2.4 Mechanics Hardware-in-the-loop system	6
Figure 3.1 Sliding phases of SMC Regulator	12
Figure 3.2 Feedforward and feedback controllers in a system	15
Figure 4.1 Coil in a magnetic field.....	18
Figure 4.2 Components of electrical motor.....	19
Figure 4.3 Synchronous and Asynchronous Motors.....	21
Figure 4.4 Brushless DC Motor	22
Figure 4.5 BLDC Motor Diagrams - single phase and three phase	23
Figure 4.6 Rotor Magnets Cross-Sections.....	23
Figure 4.7 Motor Rotation	24
Figure 4.8 Electric driver circuits.....	25
Figure 4.9 Three phase BLDC motor commutation sequence.....	27
Figure 4.10 Clark and Park transformations	28
Figure 4.11 Implemented cascade FOC control.....	28
Figure 4.12 DC motor equivalent circuit.....	31
Figure 4.13 BLDC Motor equivalent circuit.....	32
Figure 4.14 Simulink model of BLDC motor.....	35
Figure 4.15 Cascade control structure for electrical motor	36
Figure 5.1 Open-loop dynamics	40
Figure 5.2 Open-loop response	40
Figure 6.1 Flow diagram - PD control signal with nonlinear compensation.....	48
Figure 7.1 HIL Structure implemented in Simulink.....	50
Figure 7.2 Subsystems of HIL Structure	51
Figure 7.3 ADS Protocol structure	53
Table 7.1. ADS Client Server model	54
Figure 7.4 AMS Header structure	54
Figure 7.6 ADS Read Response command	55
Figure 7.5 ADS Read Request command	55
Figure 7.7 ADS Write Request command	56
Figure 7.8 ADS Read Response command	56
Figure 7.9 Diagram of ADS Communication between Simulink and Twin CAT	57
Figure 8.1 Lab implementation.....	60

Figure 9.1 Time response of the adaptive backstepping regulator. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	63
Figure 9.2 Evolution of the unknown parameter and its estimate under adaptive backstepping control.....	64
Figure 9.3 Time response of the adaptive backstepping regulator with an added adaptation of k_1 . Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	64
Figure 9.4 Evolution of the unknown parameter and its estimate under adaptive backstepping control with an added adaptation of k_1 parameter	65
Figure 9.5 Time response of the adaptive SMC regulator with an added adaptation of k_1 . Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	65
Figure 9.6 Evolution of the unknown parameter and its estimate under adaptive SMC control	66
Figure 9.7 Time response of the adaptive backstepping regulator with an added sliding surface. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	66
Figure 9.8 Evolution of the unknown parameter and its estimate under adaptive backstepping control with a sliding surface.....	67
Figure 9.9 Time response of nonlinear compensation with PD regulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	67
Figure 9.10 Time response of the adaptive backstepping regulator during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	68
Figure 9.11 Time response of the adaptive backstepping regulator with an added adaptation of k_1 during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	69
Figure 9.12 Time response of the adaptive SMC regulator during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	69
Figure 9.13 Time response of the adaptive backstepping regulator with an added sliding surface during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	70
Figure 9.14 Time response of nonlinear compensation with PD control during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	70
Figure 9.15 Time response of the adaptive backstepping regulator (k_1 added) during low update rate simulation with added disturbance. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	71
Figure 9.16 Evolution of the unknown parameter and its estimate under adaptive backstepping control with additional adaptation of k_1 with added disturbance.....	71
Figure 9.17 Time response of the adaptive SMC regulator during low update rate simulation with added disturbance. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	72
Figure 9.18 Evolution of the unknown parameter and its estimate under adaptive SMC control with added disturbance	72

Figure 9.19 Time response of the adaptive hybrid backstepping regulator with a sliding surface during low update rate simulation with added disturbance. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	73
Figure 9.20 Evolution of the unknown parameter and its estimate under adaptive hybrid backstepping regulator with a sliding surface with added disturbance	73
Figure 9.21 Time response of the nonlinear compensation with PD regulation during low update rate simulation with added disturbance. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2	73
Table 9.1 Nominal Metrics	75
Table 9.2 Nominal Metrics pre-disturbance [5s-39s]	76
Table 9.3 Post disturbance metrics	77

USED SYMBOLS

Electromagnetic Motor Model

<i>Symbol</i>	<i>Description</i>
F	Electromagnetic force acting on the conductor
B	Magnetic flux density
I	Electric current
L	Length of the conductor
θ	Angle between conductor and magnetic field
T	Electromagnetic torque
r	Distance from the shaft (radius)
N	Number of turns in the coil
K_t	Torque constant of the motor
E	Back electromotive force (back EMF)
v	Linear velocity of the conductor
ω	Angular velocity (rad/s)
K_e	Back EMF constant (V·s/rad)

Nonlinear Control Algorithms

<i>Symbol</i>	<i>Description</i>
x	State vector
$x_1, x_2, \dots, x_n$	System states
x_d	Desired (reference) state
u	Control input

<i>Symbol</i>	<i>Description</i>
$f(x), f_i(\cdot)$	Nonlinear system dynamics
$g(x), g_i(\cdot)$	Input gain function
z_i	Error (virtual state) variables in backstepping
α_i	Virtual control laws
V	Lyapunov function
V_a	Augmented Lyapunov function
θ	Unknown parameter vector
$\hat{\theta}$	Estimated parameter vector
$\tilde{\theta}$	Parameter estimation error
Γ	Adaptation gain matrix
$\Phi(x)$	Known regression vector
σ	Sliding variable (sliding surface)
λ	Sliding surface gain
μ	Integral gain in sliding surface
e	Tracking error
$\dot{e}, \ddot{e}$	Error derivatives
$I(t)$	Integral of error
K_{prop}	Proportional gain in reaching law
K_{switch}	Switching gain
ε	Boundary layer thickness
$\text{sat}(\cdot)$	Saturation function
$\alpha(x, t), \beta(x, t)$	System representation functions

<i>Symbol</i>	<i>Description</i>
$\Psi(x, t)$	Lumped system dynamics
δ	Unknown disturbance/uncertainty
$\hat{\delta}$	Estimated disturbance
$\tilde{\delta}$	Disturbance estimation error
γ	Adaptation gain (SMC)
u_{comp}	Compensation control input
u_{fb}	Feedback control input
K_p	Proportional gain (PD controller)
K_d	Derivative gain (PD controller)
T_s	Sampling time
k	Discrete time index

USED ACRONYMS

Acronym Description

HIL	Hardware-in-the-Loop
BLDC	Brushless Direct Current (motor)
FOC	Field-Oriented Control
PLC	Programmable Logic Controller
ADS	Automation Device Specification (Beckhoff communication protocol)
SMC	Sliding Mode Control
PD	Proportional-Derivative (control)
PWM	Pulse Width Modulation
EMF	Electromotive Force
dq	Direct–Quadrature reference frame
$\alpha\beta$	Alpha–Beta (stationary reference frame)
MATLAB	Matrix-Laboratory
PMSM	Permanent Magnet Synchronous Motor
HiMAT	Highly Maneuvered Aircraft Technology
CASH	Computation and Simulation of HiMAT
SHIL	Signal Hardware-in-the-Loop
CHIL	Controller Hardware-in-the-Loop
PHIL	Power Hardware-in-the-Loop
MHIL	Mechanical Hardware-in-the-Loop
ECU	Electronic Control Unit

Acronym Description

HVAC Heating, Ventilation and Air Conditioning

FOC Field of Control

CAD Computer-Aided Design

TCP/IP Transmission Control Protocol/Internet Protocol

AMS/TCP Automation Message Specification running over TCP/IP

PC Personal Computer

POU Programmable Organization Unit

1. Introduction

Throughout history, technical development has always been driven by the pursuit of innovation, with progress being the primary measure of success. This progress has not always been linear and as such it was necessary to implement extensive testing in the face of the unknown.

This master's thesis highlights the importance of testing, validation and implementation, and integration of control logic in industrial systems. The digital era of Industry 3.0 introduced simulation as a way of accelerating the automation of complex industrial infrastructures - while in Industry 4.0 simulation evolved into operational technology, laying the foundation for hardware-in-the-loop testing. Hardware-in-the-loop is the focal point of this research. This method bridges the gap between the traditional testing approach and practical, hardware validated implementation. According to the current industry requirements, the need for a fast and reliable method arises, aiming to solve advanced problems to support the transition towards network reliant systems. Specifically, in this thesis, the objective is to apply adaptive trajectory-tracking control for brushless DC motors operating over industrial networks.

The goal is to develop and experimentally test several regulators capable of following a dynamic reference trajectory, even when there are significant and unpredictable communications delays. This reflects a practical reality in the industrial environment where distributed motor drives, remote PLC control, and network integration introduce variable latency into the control loops. To address this challenge, a delay tolerant adaptive control architecture is designed and implemented on a real Beckhoff PLC, whereas the system, which is the object of the control, is implemented in MATLAB/Simulink. In the experimental setup, both the Simulink model and the structured code written in Beckhoff's Twin CAT environment are linked in a hardware-in-the-loop framework. The controller runs externally and communicates through Ethernet using the ADS protocol. The fundamental assumption in this research is that trajectory tracking with high precision can be achieved even when signals such as commands and measurements travel across delayed communication, as well as testing the performance of several established regulators such as backstepping, sliding, PD and comparing the advantages and disadvantages of using adaptive parameters in said control logic.

The main motivation for the thesis is to keep up with emerging industry trends. Modern manufacturing systems are becoming more autonomous, interconnected and adaptive, integrating cyber-physical systems, digital twins, and network centered architectures. As Industry 4.0 evolves towards Industry 5.0, significance shifts not only to efficiency but also towards rapid prototyping and real time validation that can meet the requirements of future manufacturing

facilities. In this context, the objective of this thesis aligns with ongoing industrial trends and the anticipated expansion of the hardware-in-the-loop simulation market.

2. Origins of hardware-in-the-loop simulation

Hardware-in-the-Loop simulation is a well-established method, used during system development and early system prototyping. HIL is a simulation technique, in which parts of a real system are integrated into the simulation loop. The idea of integrating real hardware parts into the simulation loop stems from the initial difficulty of modeling actuators. In that sense, rather than using control logic implemented on mathematical models of the system, real hardware was included in the simulation loop to improve the validity of the simulation. Another important distinction of this technique is that it is a real time simulation, where hardware components such as actuators, controllers or sensors are placed in the simulation environment so that the data exchange, sampling and timing constraints match those of the system.

The first attempt to formalize this technique was done in 2005, as mentioned in the 44th IEEE Conference on Decision and Control, and the European Control Conference [2], where M.Bacic defines the core elements of a HIL as: real-time simulation, which implements the mathematical model of the system or part of said system, real hardware component which is under test and lastly a bidirectional data exchange. Additionally, Bacic also defines transparency and robustness as the key concepts regarding HIL. Transparency of the simulations is defined as a measure of how well and how closely the behavior of the real hardware matches its behavior in the actual physical system. It reflects the quality of coupling between the hardware and the simulated environment. In order to determine how transparent the HIL system is, we need to answer how well the simulator recreates the physical interaction around the hardware. This physical interaction is defined as boundary or boundary conditions that the simulation must recreate so that the hardware “does not differentiate” whether it is in a simulation or is “experiencing” the physical world. Ideal transparency would implicate an ideal prediction or modeling of the system in the simulation, which is highly unlikely in the engineering world. That is why Bacic also defines the term robustness of prediction, which determines the validity and reliability of the results in the face of uncertainty.

The earliest beginnings of HIL can be found among NASA’s development work on the Highly Maneuvered Aircraft Technology Vehicles (HiMAT) experimental program in the 1970s and 1980s. In order to reduce development risk, accelerate testing and validate control concepts to the advancement of fighter jets, NASA employed multiple simulations which correspond to different frameworks of modern HIL systems. Four distinct simulations were developed, each increasing in complexity and hardware involvement. The “Basic simulation” was a software only simulation which was used to develop the flight control laws. It is important to note that in modern terminology, the Basic simulation, modeled entirely in software, would be considered a software-in-the-loop system. The second simulation was the “Verification Simulator”. It was run

on host computers and executed actual flight control logic in order to verify the accuracy and reliability of the control laws. Moving towards a truer HIL, NASA's third simulation was called "Computation and Simulation of HiMAT" or simply "CASH". In this simulation, real time computing was introduced as well as hardware, pilot controls, and input/output interfaces were incorporated. Lastly, the "Iron Bird" Simulation integrated physical aircraft components such as actuators, hydraulic elements, servos, and other components into the simulation loop. To summarize, the first simulation was a software-only simulation, then in the second, real code execution was added. In the third simulation, a controller hardware was added, which lastly progressed to incorporating actual physical components of the aircraft and even different subsystems to ultimately test the behavior of the system against the developed control [8].

What is already evident from the very beginnings of the HIL framework is that depending on which hardware component is incorporated in the simulation loop, different types of HIL systems can be defined. These types of HIL systems were methodically represented in the article "Review of hardware-in-the-loop - a hundred-year progress in the pseudo-real testing" [4].

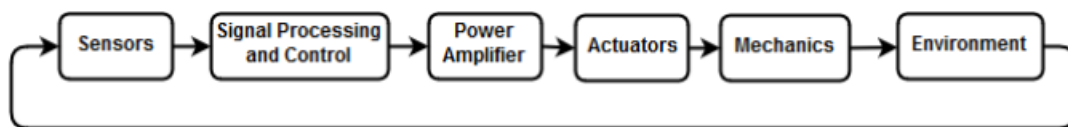


Figure 2.1 Common mechatronics system

Figure 2.1 represents a common mechatronics system form, which is used to differentiate different HIL types based on the hardware component and the simulated components [4].

It shows how the sensors measure environmental signals. The signals are then processed by the signal processor, which generates low power signals being fed into the power amplifier. This output signal then feeds into the actuators whose act affects the mechanical components.

Based on which parts of the system are real hardware and what kind of interaction they have with the simulation, different types of HIL systems have been defined [4]:

- Signal HIL
- Power HIL
- Mechanical HIL

Signal hardware-in-the-loop is most prominent in the automotive industry, especially more so with the rapid growth of electric vehicle development, which relies heavily on extensive controller validation in areas such as battery management, motor control and power electronics.

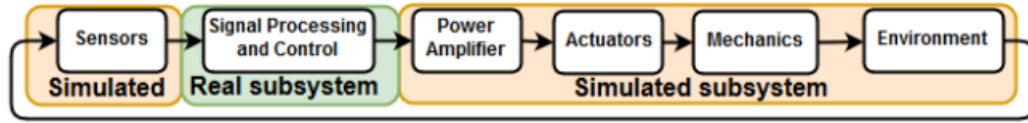


Figure 2. Error! No text of specified style in document. Signal Hardware-in-the-loop system

Figure 2.2. shows the signal HIL model. In this type of model, the entire system model is simulated in a real time simulation environment, while the hardware component is the actual controller [25]. That is why another acceptable name for this type of HIL architecture is controller-in-the-loop or controller hardware-in-the-loop. In terms of the HiMAT vehicle validation simulations, the one that could be classified as signal or controller-in-the-loop type of a system is the CASH or Computation and Simulation of HiMAT.

Although the terms signal hardware-in-the-loop (SHIL) and controller hardware-in-the-loop are often used interchangeably, they emphasize different aspects of the same methodology. It is quite evident that the term Controller HIL accentuates the presence of the real controller in interaction with the simulated environment, whereas signal HIL emphasizes the type of interaction between the simulated plant and the controller. Mainly, signal HIL as a term is used in power electronics to differentiate between signal-level testing and power-level configurations. Signal level testing refers to exchanges of low-power signals or measurement and control signals with the simulated model, where no energy is being transferred.

When energy transfer is involved in simulation, the system is called power HIL. As explained in [16], the internal emulation of the power system is one part, while the rest of the system is realized as an externally connected hardware device. Power HIL is therefore defined as a HIL configuration, in which a power device such as a converter, motor drive, a PV inverter, etc. is tested. The simulation provides the simulated environment for the testing process, so that the power device can transfer and receive actual power signals. This allows the verification of the behavior of the system under realistic electrical conditions.

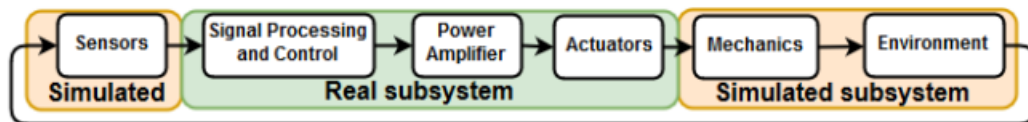


Figure Error! No text of specified style in document..2 Power hardware-in-the-loop system

The third type of HIL configuration is the mechanical HIL, which adds real actuators and investigates its behavior. In parallel, the fourth HiMAT vehicle simulation, the “Iron Bird” simulation combines elements of both power HIL (PHIL) and mechanical HIL (MHIL). More

specifically, it is neither purely PHIL nor purely MHIL, rather it is full-scale integrated HIL that includes both power signals exchange as well as real actuators.

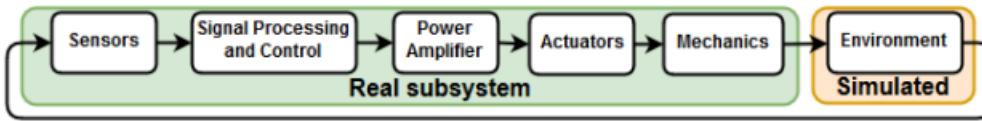


Figure Error! No text of specified style in document..3 Mechanics hardware-in-the-loop system

2.1 Application of hardware-in-the-loop

The different types of HIL systems have their own advantages and fields of application. The main advantage of controller HIL (CHIL) systems is the overcoming of timing constraints of system prototyping and development. Moreover, with the controller HIL framework, the process has significantly lower costs during the development phase. The largest field of application of CHIL is in the automotive industry. For example, the ECU is wired to a simulator through analog input and digital output channels while sensor signals are generated by the model and fed to the ECU. The ECU then outputs actuator commands which are fed back into the model.

A common HIL setup in the automotive field connects a real ECU to a real-time driver, driveline and vehicle model (dSPACE power-HIL) to validate control algorithms.

The development of medical equipment has also opened another field of application for the HIL framework. This field requires comprehensive testing procedures with strict requirements for validation of medical components. One example is the deep brain recording system which was validated in a HIL test bench built on a single board DSP [16].

As demonstrated by the application of HIL in both the automotive and medical systems, hardware-in-the-loop simulation has evolved into an engineering methodology used in many different fields and industries. However, despite the domain specific implementations, all applications rely on the same concept of connecting real hardware to a simulated environment.

3. Nonlinear control algorithms

For the purpose of this master's thesis, several nonlinear control algorithms were explored and later implemented with the aim of controlling a simulated BLDC motor model. This chapter is dedicated to the control theory and methodology of designing control laws.

3.1 Adaptive backstepping regulator

For some systems, certain nonlinearities appear multiplied with unknown parameters and give rise to the problem of controlling nonlinear systems with parametric uncertainty. This is the reason why the original recursive backstepping technique developed by Petar V. Kokotovic (circa 1990), resulted in an adaptive backstepping procedure developed by Ioannis Kanellakopoulos, as credited by Kokotovic himself [11]. The adaptive backstepping technique builds on the original idea of the backstepping regulator, mainly the aim to design stabilizing controls for a special class of nonlinear dynamical systems. The backstepping controller design is the step-by-step partial stabilization of individual states in a system where one assumes that the system is described by a chain of integrators.

This technique also includes determining the stability of the system by using the Lyapunov criteria. The idea revolves around constructing a Lyapunov function recursively so that at each step a candidate function is defined for the subsystems. Coupling the Lyapunov function with a virtual control or a virtual state which is selected in such a way that the selected Lyapunov function becomes negative definite, guaranteeing the stability of the subsystem. In the adaptive procedure, the Lyapunov function is further augmented with parameter-estimation error terms, and adaptation laws are determined so that stability is still assured.

The methodology used in this thesis for the derivation of the equations needed for the control law for the backstepping regulation is largely based on the Professor Kokotovic's lecture at the University of California, Santa Barbara [11].

- Step 0 – The system is put in strict-feedback structure

The backstepping procedure is designed for systems that can be transformed into a cascade chain as such:

$$\dot{x}_1 = f_1(x_1) + g_1(x_1)x_2 \quad (3.1)$$

$$\dot{x}_2 = f_2(x_1, x_2) + g_2(x_1, x_2)x_3 \quad (3.2)$$

$$\dot{x}_n = f_n(x_1, \dots, x_n) + g_n(x_1, \dots, x_n)u, \quad (3.3)$$

u only enters the last equation and each state acts as a virtual input for the previous equation.

- Step 1– Stabilization of the first subsystem

In this step, it is imagined as if x_2 is used to stabilize at 0 the first equation with a feedback law α_1 so that the first Lyapunov function is negative.

We define the virtual state z_1 as:

$$z_1 = x_1 - x_{1d} \quad (3.4)$$

The Lyapunov function is chosen as:

$$V_1 = \frac{1}{2} z_1^2 \rightarrow \dot{V}_1 = z_1 \dot{z}_1 \quad (3.5)$$

Since x_1 depends on x_2 , x_2 can be chosen as:

$$\alpha_1(x_1, x_{1d}, \dot{x}_{1d}) \quad (3.6)$$

Lastly, in this step z_2 is also defined as:

$$z_2 = x_2 - \alpha_1 \quad (3.7)$$

This is done because after x_2 is computed as a function of z_2 and α_1 , and placed into the Lyapunov function, the derivative of the function now contains a negative term plus a coupling term proportional to z_2 . This coupling term is what the next step recursion step aims to cancel.

- Step 2 – Stabilization of the second subsystem

As in the previous step, we imagine as if we were to use x_3 to stabilize at 0 the system (x_1, x_2) with a feedback law $\alpha_2(z_1, z_2)$. To design α_2 , first a Lyapunov function is constructed:

$$V_2 = V_1 + \frac{1}{2} z_2^2 \rightarrow \dot{V}_2 = \dot{V}_1 + z_2 \dot{z}_2 \quad (3.8)$$

$$\dot{z}_2 = \dot{x}_2 - \dot{\alpha}_2 \quad (3.9)$$

At this step, the derivative of x_2 depends on x_3 . Again, a stabilizing target form is chosen for error dynamics. Once the target dynamic is computed into the second Lyapunov function, we aim to get the negative terms, such as in Step 1.

- Step i – Repeating steps to reach the control signal u

For each step, the same principal repeats depending on the order of the system. First, a state variable z is defined, then a Lyapunov function is chosen. For the Lyapunov function to be negative definite, target dynamics are chosen to cancel out the positive terms and add damping.

This recursion is why this methodology is called backstepping: the process starts by designing x_1 and then proceeding back through the chain until the control signal u is reached.

Building on these general steps from the backstepping methodology, the adaptive backstepping methodology prepares for unknown constant parameters by augmenting the Lyapunov function.

In Professor Kokotovic's lecture, the unknown parameter was chosen as a constant parameter vector θ . The adaptive idea of Kanellakopoulos, is to augment the Lyapunov function at the step where the unknown parameter appears.

$\tilde{\theta}$ is the estimation of the unknown parameter, so that the estimation error can be written as:

$$\tilde{\theta} = \theta - \hat{\theta} \quad (3.10)$$

In the Lyapunov function where the parameter first appears, the Lyapunov derivative is expected to contain an extra cross term:

$$\dot{V}_n = (\text{negative terms}) + z_j \Phi(x) \tilde{\theta} \quad (3.11)$$

Where $\Phi(x)$ is a known variable depending on states and reference signals.

The adaptive idea is then shown as an adaptive Lyapunov function:

$$V_a = V_n + \frac{1}{2} \tilde{\theta}^T \Gamma^{-1} \tilde{\theta} \quad (3.12)$$

Where, $\Gamma > 0$ is an adaptation gain matrix.

Once the augmented function is differentiated, the result is:

$$\dot{V}_a = (\text{negative terms}) + \tilde{\theta}^T \left(\Phi^T(x) z_j - \Gamma^{-1} \dot{\hat{\theta}} \right) \quad (3.13)$$

Lastly, this is where the adaptation law is chosen to cancel the cross terms so that the Lyapunov function guarantees the stability of the system:

$$\dot{\hat{\theta}} = \Gamma \Phi^T(x) z_j \quad (3.14)$$

With this, the Lyapunov function becomes negative definite, which was the aim of the added step in the adaptive backstepping methodology.

3.2. Adaptive Sliding Mode Control Regulator

Sliding mode control (SMC) is one of the most successful approaches in handling bounded uncertainties and disturbances. Historically, the development of the novel idea of SMC began in the Soviet Union in the late 1950s. The idea is based on using a custom-defined function, named the sliding variable. As soon as the properly designed sliding variable becomes equal to zero, it defines the sliding surface. This means that the development of the sliding mode control regulator rests on the design of the sliding variable. The proper design of the sliding variable yields a suitable closed-loop performance while the system trajectories belong to the sliding surface. Taking this into account, the aim is to design a control law to drive the system, i.e. the system's trajectories to the chosen sliding surface.

There are multiple sources such as Utkin's 1981 article published in both Russian and subsequently in English [24], as well as the book "Sliding Mode Control and Observation" by the authors Shtessel, Edwards, Fridman and Levant [22], where a two-step procedure was defined. The first step is designing the sliding variable which defines the ideal behavior of the system. The second step is to choose a control law, which will make the sliding surface attractive to the system's state in the presence of external and internal disturbances/uncertainties.

- Step 1 – Determining the sliding variable and surface

For a second order system, defined as:

$$\dot{x}_1 = f_1(x, t) \quad (3.15)$$

$$\dot{x}_2 = f_2(x, t) + b(x, t) \quad (3.16)$$

Firstly, for a tracking objective $x(t)$ following $x_d(t)$, the error is defined:

$$e = x_{d1} - x_1 \quad (3.17)$$

$$\dot{e} = \dot{x}_{d1} - \dot{x}_1 = \dot{x}_{d2} - f_1(x, t) \quad (3.18)$$

$$\ddot{e} = \ddot{x}_{d1} - \ddot{x}_1 \quad (3.19)$$

$$\ddot{e} = \alpha(x, t) + \beta(x, t)u \quad (3.20)$$

When making the choice for a sliding surface, the usual choice is:

$$\sigma = \dot{e} + \lambda e, \lambda > 0 \quad (3.21)$$

The implication here is that when the sliding variable is equal to zero, the error exponentially decays.

$$\dot{\sigma} = \ddot{e} + \lambda \dot{e} \quad (3.22)$$

$$\dot{\sigma} = (\alpha(x, t) + \lambda \dot{e}) + \beta(x, t)u \quad (3.23)$$

Which can be rewritten as:

$$\dot{\sigma} = \Psi(x, t) + \beta(x, t)u \quad (3.24)$$

Since one of the conditions is that the chosen sliding surface is attractive so that the system's trajectories steer in towards it, a practical reaching law is:

$$\dot{\sigma} = -K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) \quad (3.25)$$

With K_{prop} , K_{switch} , $\sigma > 0$.

$$sat\left(\frac{\sigma}{\varepsilon}\right) = \begin{cases} sgn(\sigma), & |\sigma| > \varepsilon \\ \frac{\sigma}{\varepsilon}, & |\sigma| \leq \varepsilon \end{cases} \quad (3.26)$$

- Step 2 – Determining the control law

To determine the control law u , the model expression should be equal to the reaching law from the previous step:

$$\Psi(x, t) + \beta(x, t)u = -K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) \quad (3.27)$$

$$u = \beta^{-1}(x, t) \left(-\Psi(x, t) - K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) \right) \quad (3.28)$$

Once the control law is derived, the next step is to check the stability according to Lyapunov. The Lyapunov function in the SMC methodology is defined as:

$$V = \frac{1}{2}\sigma^2 \quad (3.29)$$

$$\dot{V} = \sigma\dot{\sigma} = -K_{prop}\sigma^2 - K_{switch}\sigma sat\left(\frac{\sigma}{\varepsilon}\right) \leq 0 \quad (3.30)$$

With this, the stability of the system is confirmed according to Lyapunov's criteria.

Integral action can be introduced at the sliding variable design step to modify the on-surface error dynamics [7]. Therefore, the previous methodology can be broadened by adding an integral action.

$$I(t) = \int_0^t e(\tau)d\tau \quad (3.31)$$

The first step would be to rewrite the sliding variable to include the integral action:

$$\sigma = \dot{e} + \lambda e + \mu I \quad (3.32)$$

To determine the behavior of the system on the surface, the sliding variable is set to equal zero.

$$\dot{e} + \lambda e + \mu I = 0 \quad (3.33)$$

$$e'' + \lambda \dot{e} + \mu e = 0 \quad (3.34)$$

This implies that the integral action converts the on-surface behavior into a stable second-order linear error model.

$$\dot{\sigma} = e'' + \lambda \dot{e} + \mu e \quad (3.35)$$

The next step is the same as Step 2 as previously explained, where the control law is determined or derived from equaling the sliding variable with the reaching law.

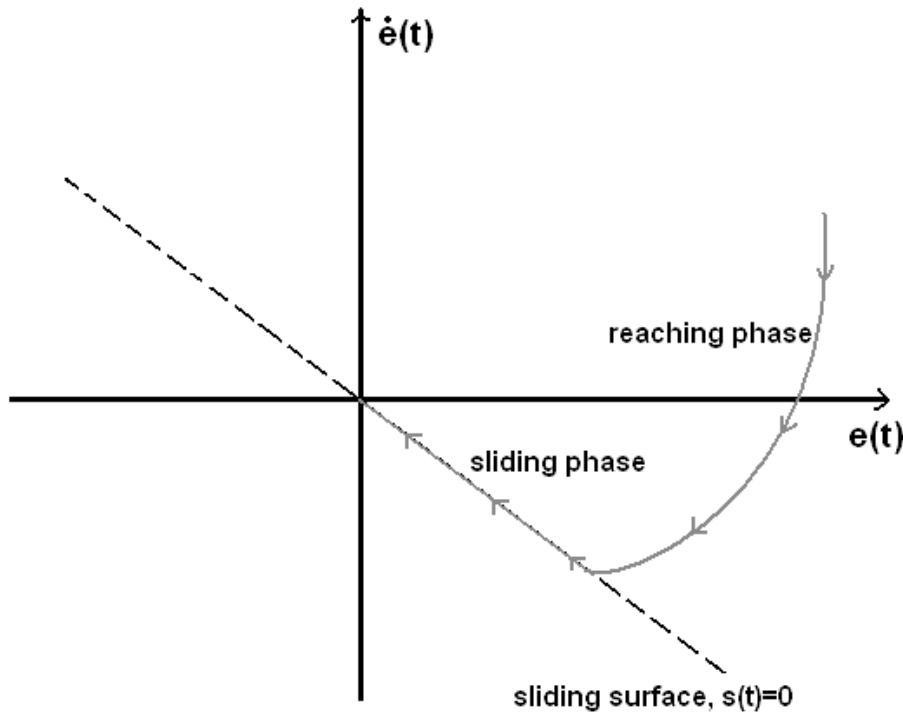


Figure 3.1 Sliding phases of the SMC regulator

The SMC methodology allows for the system's behavior to be interpreted in the following two phases: the reaching phase and the sliding phase. In the reaching phase, the control signal aims to drive the sliding function towards zero. This phase is governed by the reaching law:

$$\dot{\sigma} = -K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) \quad (3.36)$$

Once the trajectories enter the vicinity of the sliding surface, the system enters the sliding phase. During this phase, the error dynamics are derived from the definition of the surface, being:

$$\sigma = \dot{e} + \lambda e + \mu \int e \, dt \quad (3.37)$$

The two phases of the SMC are shown in Figure 3.1. It illustrates both the reaching phase as well as the evolution into the sliding phase.

Having to face the problem with uncertainty, such as an unknown parameter, adaptive control is needed to broaden the SMC methodology, same as with the backstepping regulator. The pioneers of adding the adaptation law to the derivation of the SMC procedure are Utkin and Poznyak [3], however their approach is based on solving the problem with adaptation of the switching gain. They proposed two adaptations, the sigma adaptation and the dynamic adaptation.

In this master's thesis, specifically when designing the Sliding Regulator for the system with uncertainty in terms of the unknown parameter, the uncertainty is handled by implementing the Lyapunov cross-term cancellation method. The idea is that the procedure is updated with the uncertainty as such:

$$\dot{\sigma} = \Psi_0(x, t) + \beta u + \varphi(x, t)\delta \quad (3.38)$$

And the control law is rewritten as:

$$u = \beta^{-1}(x, t) \left(-\Psi_0 - \varphi\hat{\delta} - K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) \right) \quad (3.39)$$

$$\dot{\sigma} = -K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) + \varphi(x, t)\tilde{\delta} \quad (3.40)$$

The augmented Lyapunov function is:

$$V = \frac{1}{2}\sigma^2 + \frac{1}{2\gamma}\tilde{\delta}^2, \quad \gamma > 0 \quad (3.41)$$

$$\dot{V} = \sigma\dot{\sigma} - \frac{1}{\gamma}\tilde{\delta}\dot{\tilde{\delta}} \quad (3.42)$$

Lastly, the adaptation law should be chosen as:

$$\dot{\tilde{\delta}} = \gamma\sigma\varphi(x, t) \quad (3.43)$$

This methodology is further used for the design and derivation of the SMC control law.

3.3 Hybrid backstepping regulator with an added sliding surface

The previously mentioned regulators, backstepping and SMC, each have their own advantages and disadvantages. The main difference is in the way the control law is derived and how it applies to the motor.

The backstepping regulator relies on the accuracy of the model of the system which is the aim of the applied control. One of the limitations of the backstepping regulator is that disturbances affect the system's model, which in turn would affect the control signal u , which tries to overcome the already modeled nonlinearities. This implies that the backstepping regulator is only as good as the model of the system. The Sliding regulator uses a different approach for deriving the control law, one that doesn't rely as much on the modeling of the system because it is designed to handle the bounded disturbances and uncertainties. For that reason SMC can tolerate modeling errors better because the switching reaching term provides a corrective action to overcome the uncertainties.

This chapter proposes a combined approach, which would allow to improve the robustness lacking in the backstepping approach while still preserving the smooth model-based tracking. The proposed approach relies on the backstepping methodology combined with a sliding surface term. In this way, the nonlinear cancellation and the desired error dynamics are coupled with the sliding compensation methodology.

One of the earliest statements made about the combination of both the backstepping and sliding mode regulator was done by Rios-Bolivar in his doctoral thesis at the University of Sheffield under the mentorship of Professor Zinober, both pioneers of the combined methodology [19]. Furthermore, the idea of the sliding mode backstepping approach was broadened in [12] and [21]. Based on the approaches in the abovementioned articles as well as the nominal backstepping procedure, the methodology for the backstepping sliding regulation has been defined in three steps. Steps zero and one are the same as in the backstepping methodology, and in step two, the composite error via the virtual state z_2 is defined. In the combined approach, the reaching law of the sliding regulator is added to the derivative of z_2 . The idea is that performance and the handling of unknown parameters will be improved by utilizing the backstepping methodology along with adding the reaching law.

3.4 PD regulation with nonlinear compensation

The backstepping regulator, SMC, and the combined approach of backstepping with sliding mode control are all feedback regulators, which means that the control signal u was computed from the measured states, not from the reference alone.

In this chapter, the theoretical background of model-based nonlinear compensation and feedback design is studied and later, in the following chapters, implemented into Twin CAT's Function Block.

Feedforward compensation is a common design choice in control theory because it can separate the design problem into two parts: feedback controller and feedforward controller. It can also be used very effectively to improve the set-point response of the system. An effective use of feedforward requires a system structure that has two degrees of freedom (DOF), when the tracking objective is separated from disturbance rejection by combining a feedforward and a feedback controller [10].

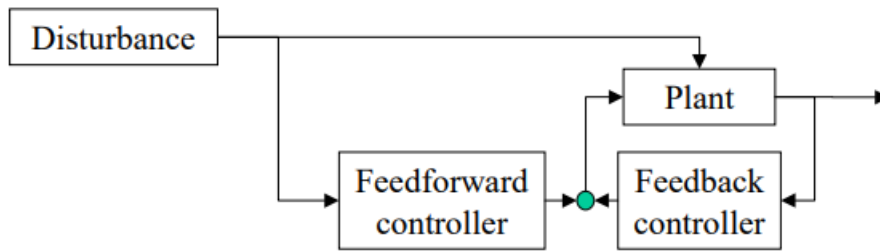


Figure 3.2 Feedforward and feedback controllers in a system

Building on the idea of two separate controllers, whose outputs are summed in order to get the final contributing control input to the plant, the same principle is adopted in [20]. The novel idea of the paper is to apply a PID-computed-torque controller for a three-link robot. Similarly to the feedforward idea, the design includes a control to cancel and compensate for nonlinear dynamics as well as control for stabilizing the error dynamics. In [9], authors presented a dual PD control implemented on a nonlinear system, the ball and plate system model-based compensation was done before the PD regulation was executed.

The implemented controller used in this master's thesis consists of a parallel combination of model-based nonlinear compensation as well as parameter estimation and a PD error-feedback regulator. The compensation controller relies on the measured states of the system, making it a feedback controller as well. The total command is their sum. Both regulators rely on measured signals, which is why the structure remains closed-loop feedback.

For a general system in the form of:

$$\dot{x}_1 = x_2 \quad (3.44)$$

$$\dot{x}_2 = f(x) + g(x)u \quad (3.45)$$

Where:

x represents the system states,

$f(x)$ represents nonlinear dynamics,

$g(x)$ represents the input gain,

u represents the control input.

The nonlinear control to be applied is proposed as: $u = u_{comp} + u_{fb}$, where the first term represents a model-based control input, while the second term represents a feedback stabilization term.

After compensation, the closed-loop dynamics are approximately: $\dot{x}_2 \approx u_{fb}$, allowing for a proportional-derivative feedback law to be introduced:

$$u(t) = K_p e(t) + K_d \dot{e}(t) \quad (3.46)$$

K_p – proportional gain

K_d – derivative gain

$e(t)$ - error

Lastly, since the regulation is to be implemented in Twin CAT environment, a discrete form of the PD regulator is needed. This is because the control logic is run on a PLC/real-time task and the controller output is updated at a fixed task rate.

$$u[k] = K_p e[k] + K_d \dot{e}[k] \quad (3.47)$$

or,

$$u[k] = K_p e[k] + K_d \frac{e[k] - e[k-1]}{T_s} \quad (3.48)$$

4. Theoretical background for BLDC motors

The hardware-in-the-loop methodology for the purpose of validating the control algorithms applied to physical systems during the research and development process is dependent on industry requirements. Therefore, the industrial and manufacturing processes are the ones that dictate the domain of utilization of this methodology.

When looking at the floor of a factory, the motion of industrial elements and components fills up the place, from conveyor belts to robots that move in their encapsule, it is the electrical motor systems that drive all this motion. Their widespread use is why electric motor systems are a relevant application domain for HIL.

Market reports say that BLDC motors are poised to grow in utilization for industrial purposes during the period covered in the report's period from 2024 to 2028 [18]. The reason for this predicted growth is the analysis of the automotive sector, particularly the expansion of electric and hybrid vehicles.

These motors are utilized in applications such as:

- consumer durables: air conditioners, washing machines and refrigerators because they offer reliability compared to traditional brushed motors.
- automotive sector: electric and hybrid vehicles.
- industrial automation: BLDC motors are integral to factory automation, industrial robots, tooling systems and other industrial systems

Moreover, BLDC motors are also used in heating, ventilation and air conditioning (HVAC) systems, medical devices, aerospace components, and renewable energy systems [18].

The recent developments in the market also suggest that brushless DC motors continue to be the main elements in the manufacturing field. For example the Allied Motion Technologies' rare-earth brushless DC motors designed for high-performance medical respirators, AMTEK Inc. expanding its portfolio by developing BLDC for aerospace applications, as well as Anaheim Automation Inc. targeting the consumer electronics field with their BLDC motors [18].

In an already successful market domain predicted to only keep on expanding, the control and regulation of BLDC motors is becoming a very important qualification skillset for engineers in any industrial domain field. That is what this master's thesis is hoping to accomplish with combining a simulation of a key element in industrial automation as well as incorporating a real hardware component in a hardware-in-the-loop framework.

4.1 Fundamentals of motor theory

Fundamental motor theory defines electric motors as components that convert electrical energy into mechanical motion using the interaction between magnetic fields, electric currents, and mechanical forces. This principle of energy transformation is universal for DC motors, AC motors, as well as BLDC, and PMSM. The only difference lies in how they implement the conversion of energy.

The shared idea is that a current carrying conductors induces two forces of opposite direction when the current passes through the magnetic field [26].

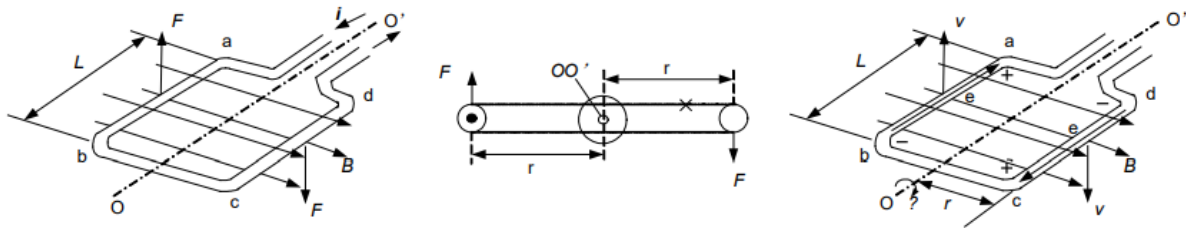


Figure 4.1 Coil in a magnetic field

When a conductor of length L carries current I inside the magnetic field, where B represents the magnetic field density, it experiences a force F .

$$F = BIL \sin(\theta) \quad (4.1)$$

A single force is not enough to rotate a motor, rather a pair of forces are needed to act at a radius. When opposite sides of a coil experience forces in opposite directions, a rotational effect or torque is produced.

$$T = 2rBILN = KtI \quad (4.2)$$

where:

r – distance from the shaft,

N – number of turns,

Kt – torque constant which is motor specific.

When the motor rotates, the conductors are now moving through the magnetic field, which in turn induces voltage opposing the applied voltage. This is called back electromotive force (back EMF). Despite the name, back EMF is a voltage generated by the electromagnetic effects and has units of volts and behaves as a voltage source. Intuitively, the faster the rotor spins, the larger the back EMF:

$$E(\text{one conductor}) = BLv \sin(\theta) \quad (4.3)$$

In a well-designed motor, conductors are arranged, so motion is perpendicular to the magnetic field. Therefore, $\sin(\theta) \approx 1$ and,

the back electromotive force for one coil is:

$$E = 2BLvN \quad (4.4)$$

$$E = 2BLr\omega N \quad (4.5)$$

$$E = Ke\omega \quad (4.6)$$

where:

ω - angular velocity (rad/s),

r - internal radius of the motor,

$Ke=2rBLN$ - electromotive force constant (V's/rad)

Each electric motor is composed of a rotor and a stator. As their name suggests, the stator is the stationary component that contains the electrical winding responsible for generating the magnetic field which is required for the torque production. The rotor is the rotating part of the electric motor which rotates and interacts with the magnetic field generated by the stator.

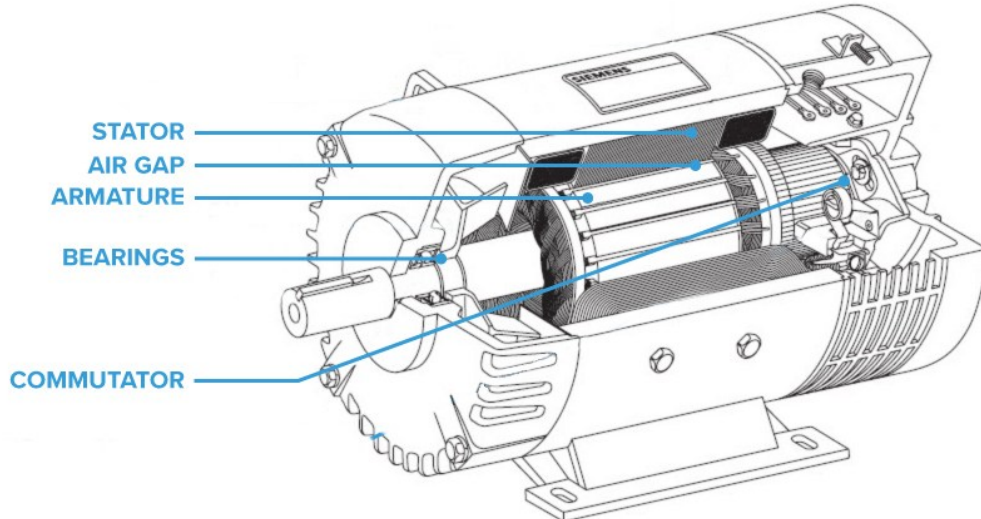


Figure 4.2 Components of an electrical motor

The rotor is supported by bearings, which transfer the axial and radial loads from the shaft to the motor housing and allow low friction rotation [14]. Bearings are the mechanical components that allow the rotor to rotate with minimal friction and transfer the loads to the motor housing. The two types of loads are the radial loads, perpendicular to the shaft and axial loads, which go along the shaft.

A commutator is another fundamental component of electric motors, and it is a current-switching mechanism, whose job is to reverse the direction of the current in the rotor at the correct moment so that the electromagnetic torque always acts in the same rotational direction.

Another important element in the principle of working of the electric motor is the air gap, which is the physical separation of the rotor and the stator. It is the element that prevents mechanical contact between the two components and allows the magnetic flux to cross.

Even though the basic components of electric motors are the same for each motor, there are multiple classification criteria according to which different types of electric motors can be defined.

4.2 Classification of electric motors

- Classification by power-supply

This classification refers to the type of power supplied to the electrical motor. More specifically, it makes a distinction between the nature of electrical energy delivered to the motor terminals. From this classification, the main types of electric motors are the DC-supplied motor and the AC-supplied motor. The paper notes that the “rated-power classification for both DC and AC motors is considered the most suitable approach for studies in control methods and applications” [23].

DC-supply motors – the magnetic field is produced by direct current. This group includes brushed DC motors, brushless DC (BLDC) motors, stepper motors, and servo motors.

AC-supply motors – the magnetic field is produced by alternating current. This group comprises single-phase and three-phase AC motors such as induction and synchronous types.

- Classification by torque production mechanism

This classification criteria answers the question of why the rotor starts to turn rather than vibrate or starts to heat up. In all electric motors, as explained in the beginning of Chapter 4, when voltage is applied to the motor and current flows through the conductors, a magnetic field is created that exerts forces which when applied at a radius create torque.

The main types, according to this classification are synchronous and asynchronous motors. Even though this classification originated in AC theory, it is a much broader field that encompasses both DC and AC motors. Synchronous motors run at a constant speed regardless of the load acting on them. The interaction between the rotating and a constant magnetic field leads the motor to run at a constant speed. The rotating magnetic field is produced by the stator of the synchronous motor, while the rotor creates a constant magnetic field. The field coil stator is excited by a three-phase AC supply and produces a rotating magnetic field with a synchronous

speed. At the same time, the rotor is excited by a DC power supply and acts as a permanent magnet or is itself made of permanent magnets.

Asynchronous motors, or more commonly known as induction motors, are still the most common type being used today, with around 90% of motors used in industrial applications being induction motors. Asynchronous means that the motor operates at less than its synchronous speed. The synchronous speed is the speed of the stator rotating magnetic field. Synchronous speed is dependent on the frequency and number of poles.

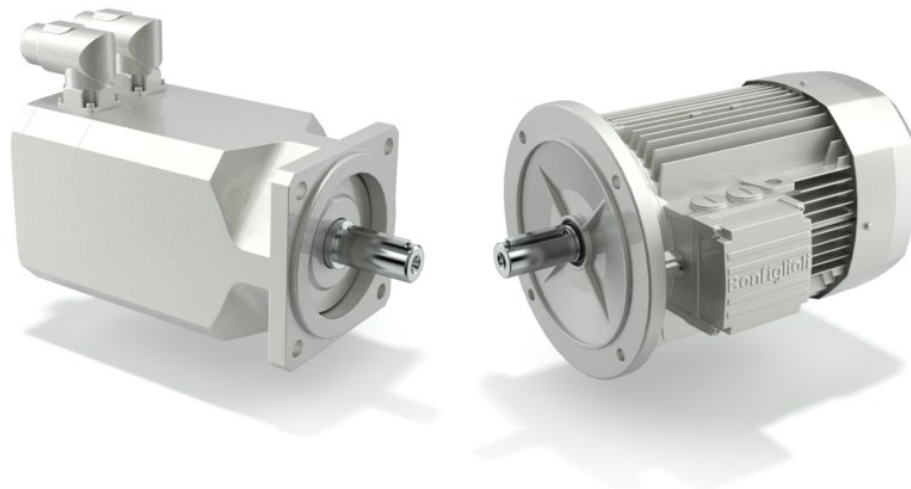


Figure 4.3 Synchronous and Asynchronous Motors

- Classification by field generation method

This classification criteria captures how magnetic energy is stored and controlled in the motor. This classification method divides motors into motors with electromagnetic field generation and motors with permanent magnet field generation. This method determines the different classes of motors based on the type of magnets used in the motor components. Firstly, we have electromagnets that generate magnetism through the flow of electric current. When an electric current passes through a coil of wire, it creates a magnetic field. This effect is amplified in motors by wrapping the wire around a ferromagnetic core which strengthens the magnetic effect. Then we have permanent magnets which derive their magnetic field from the internal alignment of their atoms. The magnetic field is constant, meaning that no external energy source is required to function. Therefore, permanent magnet motors have a constant magnetic field, while electromagnetic motors use coils of wire to generate the magnetic field, but only when a current flows through the wire.

4.3 Brushless DC motors

Conventional DC motors have traditionally been used in applications where power sources are available, for example aircrafts and automobiles. However, one drawback of continuous use of conventional motors is the excessive sparking and brush wear. The reason for this is that the small DC motors were too small to use compensating windings and interpoles, which brought about the sparking on the commutator brushes. In addition, the high rotational speed of these motors caused increased brush wear, requiring maintenance of the motors. These disadvantages are the reason brushless DC motors were developed and designed to overcome the drawbacks of conventional motors. These motors are called brushless DC motors because they run from a DC power source but do not have neither commutators, nor brushes [5].

The basic components of a brushless DC motor are:

- A permanent magnet rotor
- A stator with single, three or more phase winding
- A rotor position sensor
- An electronic circuit to control the phases of the rotor winding

According to the previous classification of electric motors, BLDC motors can be classified as a DC-supplied, permanent-magnet synchronous electric motor in which electromagnetic torque is produced by the interaction between a stator-generated rotating magnetic field and a rotor magnetic field provided by permanent magnet.

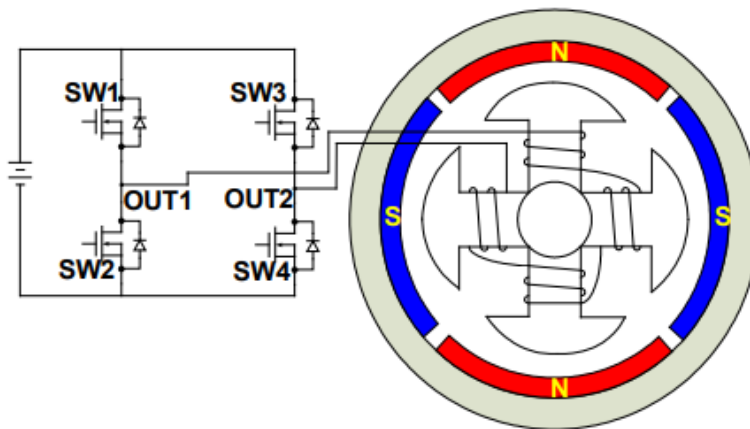


Figure 4.4 Brushless DC motor

4.3.1 Principle of working of a BLDC

There are three classifications of the BLDC motor: single-phase, two-phase, and three-phase. Single-phase and three-phase motors are the most commonly used. A single-phase motor has one stator winding, while in a three-phase motor, the stator contains three sets of windings (labeled with A, B, C), which are spatially displaced by 120 electrical degrees.

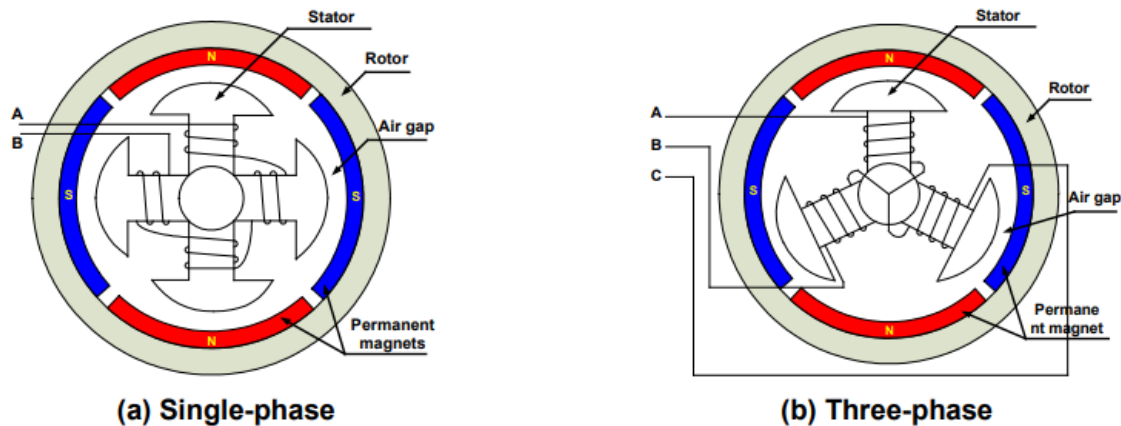


Figure 4.5 BLDC motor diagrams - single phase and three phase

When electric current flows through a stator phase, it generates an electromagnetic pole whose polarity depends on the direction of the current. By sequentially energizing the stator phases, a rotating magnetic field is created around the stator bore. The rotating field is not continuous by nature, rather it is controlled by the switching of phase currents.

The rotor consists of a shaft and a hub fitted with permanent magnets arranged in alternating north and south poles, typically forming between two and eight pole pairs [26].

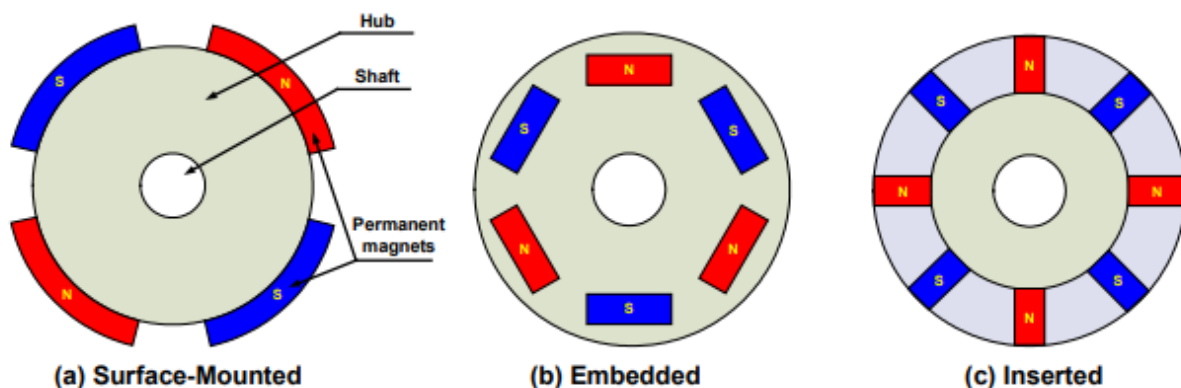


Figure 4.6 Rotor magnets cross-sections

As shown in Figure 4.6, these magnets may be surface-mounted, embedded, or inserted, depending on the motor's design.

While the stator's magnetic field is dependent on the energizing of the phases, the rotor's magnetic field is inherent because it solely depends on the magnetic properties of the material.

When the stator produces a magnetic field, the rotor magnets experience attractive and repulsive forces. The rotor tends to align itself with the stator's magnetic field in a position of minimum magnetic reluctance. However, before full alignment occurs, the controller switches current flow to the next phase, causing the stator's magnetic field to shift. This in turn causes the rotor to follow its inclination of alignment, rotating to align with the already shifted stator's magnetic field.

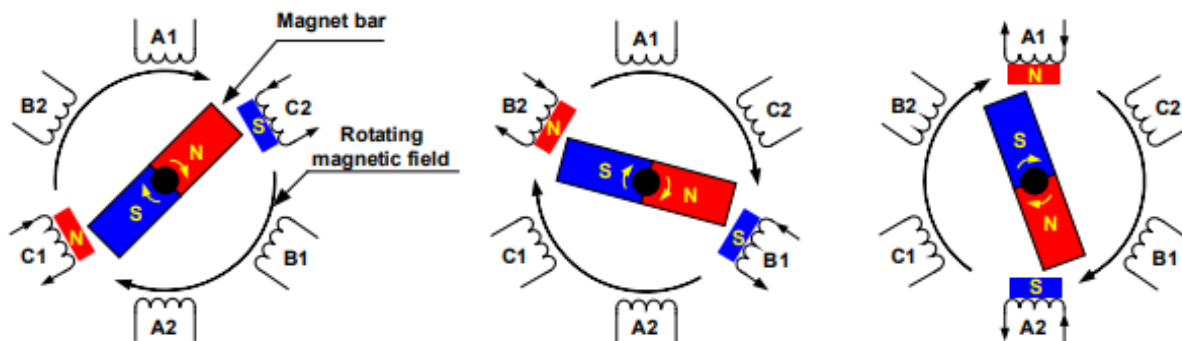


Figure 4.7 Motor rotation

The torque is generated by the interaction between the magnetic fields of the stator windings and the permanent magnet rotor. The alignment which the rotor aims to achieve results in a tangential force that acts to reduce the angular misalignment between these two fields. The torque is afterwards maintained by the electronic rotating or shifting of the stator field so that misalignment is preserved.

The magnitude of the produced torque depends on several parameters, including:

- the amplitude of the stator phase currents,
- the number of turns in the stator windings,
- the magnetic strength and geometry of the rotor magnets,
- the air-gap length,
- and the effective radius of force application.

Because the rotor magnetic field is provided by permanent magnets, no rotor excitation current is required, which improves efficiency and reduces losses.

In BLDC motors, the subsequent energizing of the stator phases is achieved through electronic commutation using a power electronic controller or commonly referred as an electronic speed

controller (ESC). The ESC switches the current flow in each of the phases by a predetermined sequence in order to produce a rotating magnetic field in the stator.

The feedback for the rotor position is usually obtained by the Hall-effect sensors and is used by the ESC to determine when to switch the current. Another BLDC motor design, especially in applications where the rotor is in closed housing and requires minimal electrical entries, such as a compressor, is the sensorless BLDC motor. In this type of motor, the sensorless drive monitors the back EMF instead of the position detected by the Hall sensor.

4.3.2 Electronics commutation principle

The electronic speed controller (ESC) is responsible for performing electronic commutation. Brushless DC motors rely on electric switches to realize the current commutation and thus cause the rotor to continuously rotate. Single-phase BLDC motors are usually connected in an H-bridge, while three-phase motors use a three-phase bridge structure as shown in Figure 4.8 [26].

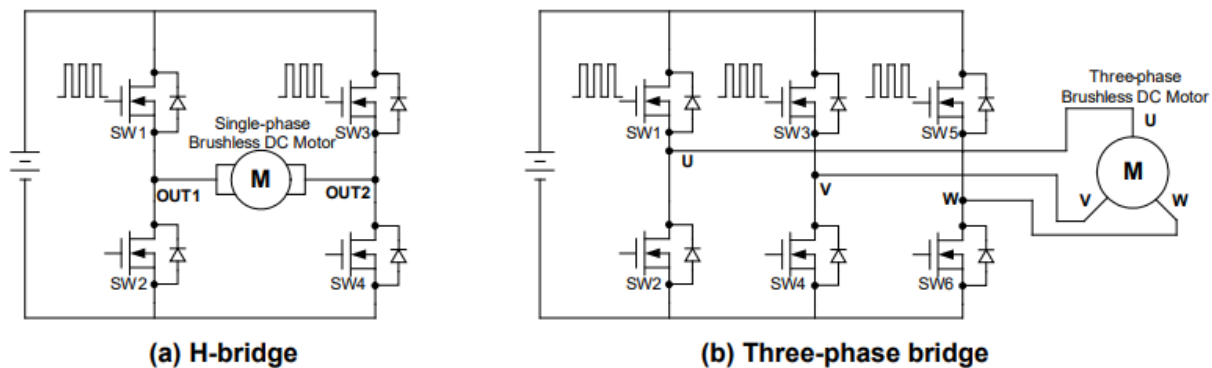


Figure 4.8 Electric driver circuits

The ESC receives a pulse-width modulation (PWM) signal, which represents the desired motor speed or torque, while the actual commutation sequence is determined by rotor position feedback. Depending on whether a single-phase BLDC or a three-phase BLDC motor is used, the number of Hall sensors used for rotor position feedback changes. In a single-phase motor, a single Hall sensor is used, while in a three-phase motor the feedback is provided by three Hall sensors positioned 120° electrical apart.

In the example shown in Figure 12, a three-phase motor is shown, where the three Hall sensors are depicted as “a”, “b” and “c”. Based on the combination of Hall sensor signals, the ESC selects the appropriate commutation state and activates a corresponding set of MOSFETs in the three-phase inverter, shown as “SW1” to “SW6”. This example assumes the sensor goes HIGH when the rotor’s North Pole passes by and goes LOW when the rotor’s South Pole passes by.

Based on the Hall sensor signals; the ESC switches the MOSFETs in the three-phase inverter. Each phase has a high-side and low-side MOSFET, and at any given step, one phase is connected to the positive voltage, another to ground, and the third is left floating. This allows the current to flow between two phases, generating a magnetic field in the stator. The rotor follows this field, producing torque and causing rotation.

As the rotor moves, the Hall sensors detect the changing magnetic poles, and the ESC switches the MOSFETs to energize the next phase in the sequence. This sequence repeats continuously. One electrical cycle consists of six steps, each advancing the stator field by 60° electrical. The rotor never fully reaches perfect alignment with the stator field because the sequencing of the current switching does not allow it and therefore keeps the rotor aiming for alignment with the stator field.

However, the number of electrical cycles does not always correspond to a complete mechanical revolution of 360° . Instead, it is determined by the number of rotor pole pairs. In this example, the rotor had two rotor pairs, which brings the conclusion of one mechanical revolution equaling two electrical cycles.

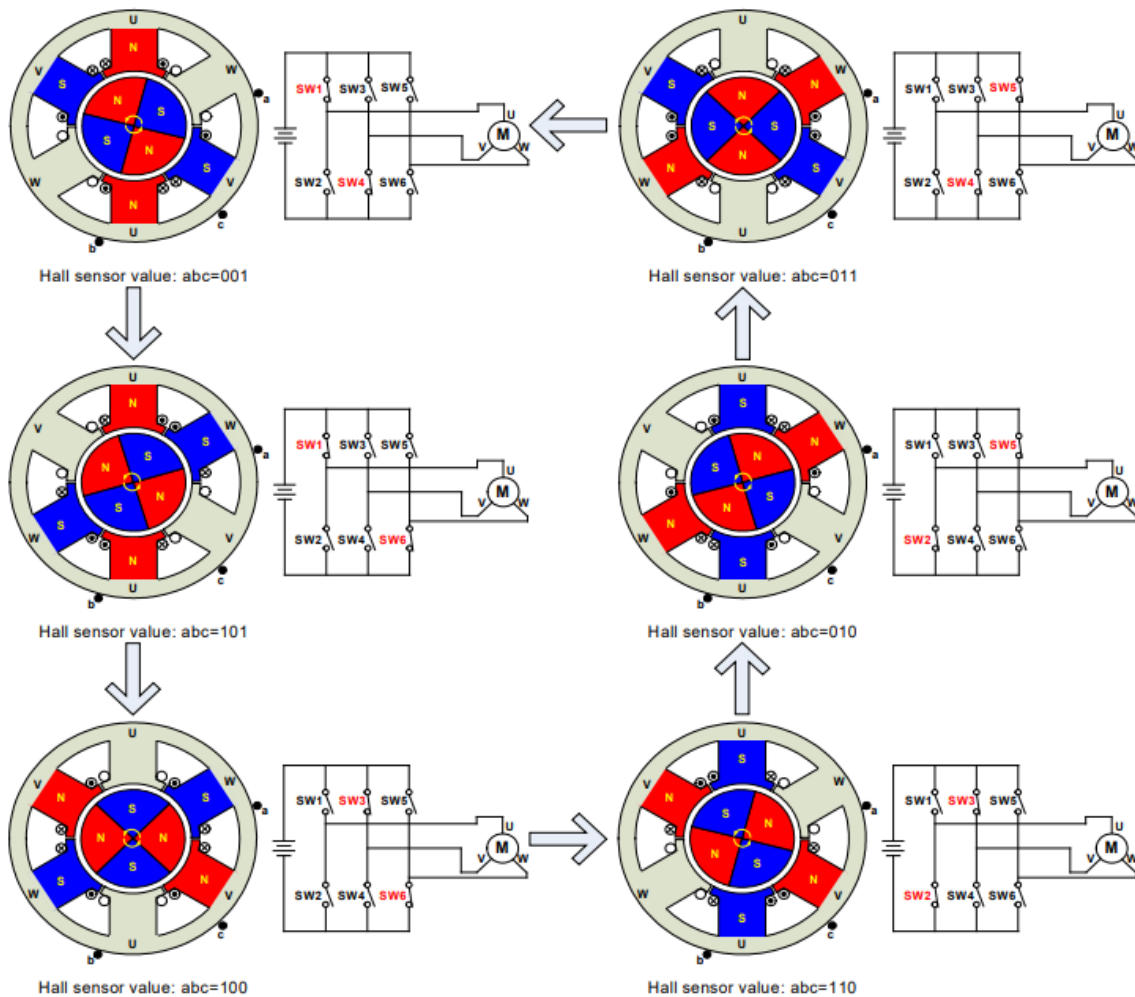


Figure 4.9 Three phase BLDC motor commutation sequence

As mentioned before, in brushless DC motor design, a sensorless motor is also possible. This means that a Hall sensor for rotor position feedback is not included in the BLDC design, and therefore the switching of the current flow cannot be sequenced. The way the sequencing is done is by detecting the back EMF.

During this operation, the ESC energizes two of the three stator phases, while the third phase is left unpowered. As the permanent magnets on the rotor rotate around the stator field, the back electromotive force (back-EMF) is induced in the winding. By monitoring this voltage and detecting its zero-crossing, the ESC determines the rotor position and performs the next commutation step. Since back-EMF is only present when the motor is rotating, the ESC initially starts the motor in open-loop mode and switches to sensorless operation once sufficient speed is reached.

4.3.3 Control methods

BLDC motors use electronic switching to realize the current commutation. Rotor position is important, so that the switching can occur at the right time. In the previous section, two design choices were explored, the sensor and sensorless BLDC motors. In each case, through the use of Hall-effect sensors or measuring the back-EMF, the position of the rotor can be detected or determined subsequently.

In practical motor drive systems, there are several control models used such as: trapezoidal, sinusoidal and Field-Oriented Control (FOC). Trapezoidal control is also called a six-step commutation control, and was already explored in the previous section as it explains best the principle of working of the BLDC motors. The sinusoidal control differs from the trapezoidal by continuously varying voltage across all phases to follow sine wave. In the trapezoidal control only two phases conduct at a time. FOC, or also called vector control, is another type of control model. The idea behind this method is the transformation of the three-phase stator quantities into a two-axis coordinate system through the use of Clark and Park transformations.

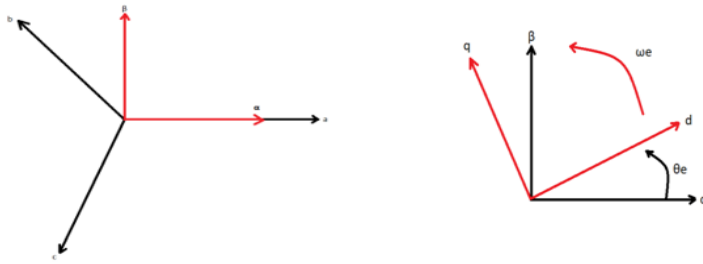


Figure 4.10 Clark and Park transformations

Specifically, the phase variables (In Figure 4.10 represented as a, b, c, although commonly used as u, v, w) are first mapped into the stationary $\alpha\beta$ frame and subsequently into the rotating dq frame. In this frame, the direct-axis is aligned with the rotor flux and primarily influences the magnetic field, while the quadrature-axis is responsible for torque production.

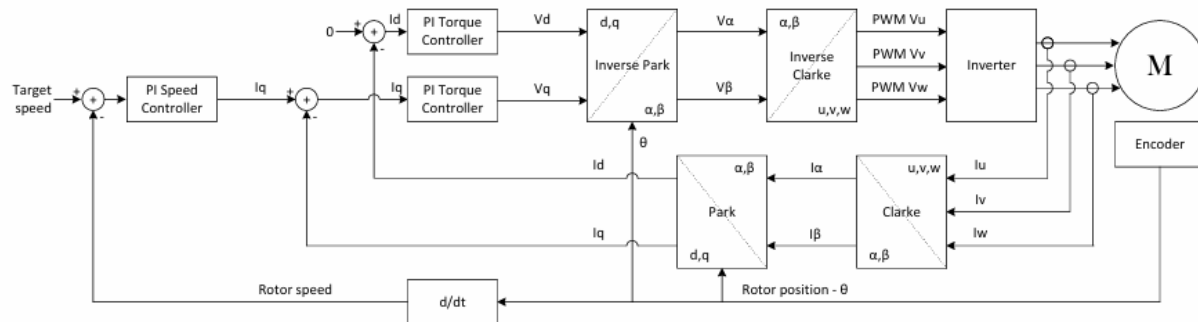


Figure 4.11 Implemented cascade FOC control

Figure 4.11 shows a cascade FOC control. The control structure involves transformations in both directions. The measured three-phase currents are transformed into the two-axis frame by using the Clark and Park transformations. In the rotating dq frame, the current control is performed. In this reference frame, the direct-axis current i_d is regulated to zero, while the quadrature-axis i_q is controlled to generate the required electromagnetic torque.

The reference value for i_q is provided by the outer speed control loop, whereas the current controllers regulate both i_d and i_q to their references. The resulting control voltages V_d and V_q are then transformed back through the inverse Park and Clark transformations to obtain the three-phase voltage signals, which are applied to the motor via the inverter.

In this context, the focus of this thesis is placed on the outer loop of the motor-drive system, where the objective is to regulate the mechanical output, specifically position tracking which will be further explored in the next chapters.

4.3.3 Key industrial challenges in BLDC control

There are several challenges imposed on the control of BLDC motors, especially in industry settings. It is important to know what the challenges are, so that the control logic, which is to be developed, overcomes them.

The key industrial challenges are:

- Accurate rotor-position information

For accurate and reliable BLDC motor control, position feedback from the rotor is important for the timing of the commutation. For sensor operation of the BLDC motors (Hall-effect sensors), the control challenge is to achieve smooth and accurate commutation using the limited and imperfect position data which is at disposal.

The other scenario is one where no sensor is used – sensorless BLDC motor, where the controller essentially must guess the rotor position through measuring the back EMF. However, this works well at medium and high speeds, while at low speed or standstill, the signal is very weak and thus, estimation becomes difficult. This proposes the challenge of estimating rotor position accurately when the signal is weak, noisy, or missing [13].

- Torque ripple during commutation

Since the switching of the current is done in six discrete steps, each switching event changes the magnetic force. This in turn causes pulsating torque, mechanical vibration, and acoustic noise. The challenging part here is that the torque ripple causes wear in mechanical systems, reduces efficiency, and cannot be used in precision or low-noise application. That is why the controller must adjust the commutation timing as well as control the PWM signal being sent to the ESC [13].

- Parameter sensitivity

Motor parameters are not constant, and even though control algorithms assume certain parameter values, when those parameters drift, current control becomes inaccurate, and the torque estimation becomes harder. This presents a problem especially in sensorless BLDC motors, and it is why the controller must remain stable during parameter changes and be able to compensate when these changes appear [13].

- Dynamic trajectory tracking

The need for dynamic control of the trajectory can be attributed to the fact that industrial systems, such as robots and CNC machines, require continuous time-dependent motion. It means that the control logic or algorithm must consider the acceleration and deceleration manner of the motor motion in order to perform precise tasks.

4.4 Brushless DC motor and model concept

4.4.1 Mathematical model of a typical DC motor

A typical DC motor equivalent circuit is shown in Figure 4.12 below.

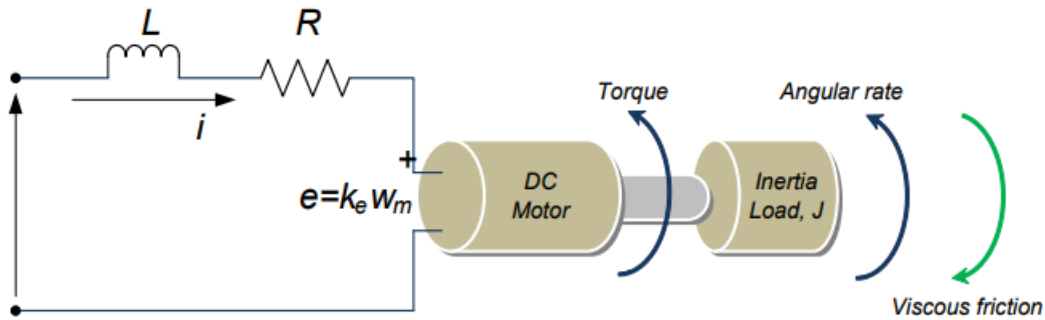


Figure 4.12 DC motor equivalent circuit

The basic components which are represented are:

R – armature resistance,

L – armature inductance,

e – back EMF.

Using the Kirchhoff's Voltage Law, the following equation can be obtained:

At steady state:

$$V = Ri + e \quad (4.7)$$

Therefore, the equation for a non-steady state would be:

$$e = -Ri - L \frac{di}{dt} + V \quad (4.8)$$

Where:

i - armature current,

V – DC source voltage.

What these equations describe about the system in mathematical symbols, can be translated into intuitive logic based on the principle of working of the DC motors. Thus, when DC voltage is applied to the motor terminals, it creates a potential difference across the armature windings.

The current rises following RL, and torque starts spinning the shaft. As speed rises, the back EMF reduces the current which naturally limits the speed.

Once the electromagnetic torque is produced, i.e. the current interacts with the magnetic field inside the motor via the Lorentz force on conductors, the rotor accelerates and since it is a rotating mass, it obeys Newton's second law of motion:

$$J \frac{d\omega}{dt} = \Sigma \tau i, \quad (4.9)$$

meaning that the product of the inertia load – J, and the rate of angular velocity - ω would equal the sum of all the torques [12].

Typically, the mathematical model of a brushless DC motor is not entirely different from the conventional DC motor. The main difference lies in the presence of multiple stator phases, which significantly influence overall motor behavior. These phases directly affect the equivalent resistance and inductance of the BLDC system. For example, a simple symmetrical three-phase BLDC motor with an internal wye (Y) connection could give a brief illustration of the phase concept.

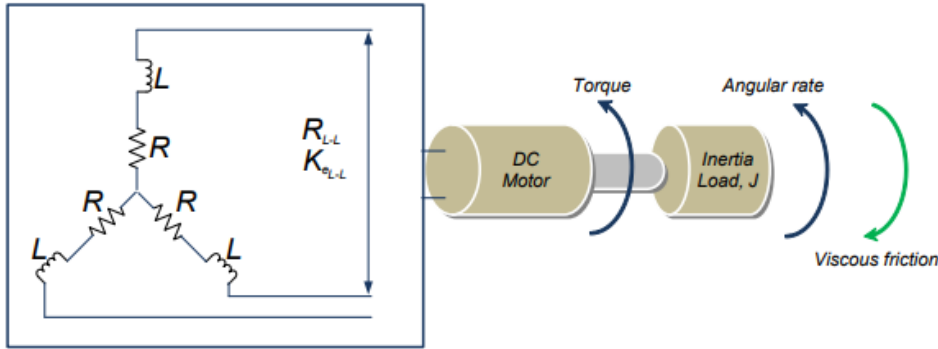


Figure 4.13 BLDC motor equivalent circuit

Newton's second law of motion can be written down as:

$$J \frac{d\omega}{dt} = \tau_{em} - \tau_{fric} - \tau_{load} - \tau_{magnetic}$$

This equation shows that part of the generated torque is used to accelerate the rotor inertia, while the rest is dissipated due to viscous friction, external mechanical loads, and position-dependent magnetic effects.

Real BLDC motors are affected by several non-ideal phenomena, including:

- Cogging torque, caused by stator slotting

- Magnetic saturation, leading to nonlinear flux–current and flux–position relationships
- Magnetic hysteresis, introducing asymmetry and memory effects

To capture these effects in a simplified but meaningful way, the position-dependent magnetic torque is modeled as a nonlinear stiffness term of Duffing type:

$$\tau_{magnetic}(\theta) = k\theta + k_3\theta^3 \quad (4.10)$$

This expression can be derived from a magnetic potential energy function,

$$V(\theta) = \frac{1}{2}k\theta^2 + \frac{1}{4}k_3\theta^4 \quad (4.11)$$

where the magnetic torque is given by:

$$\tau_{magnetic} = \frac{d\theta}{dV} \quad (4.12)$$

4.4.2 State-space formulation

Combining the electrical torque generation and mechanical dynamics yields the nonlinear equation of motion:

$$J\dot{\omega} = u - B\omega - k\theta - k_3\theta^3 \quad (4.13)$$

where u represents the control input related to the electromagnetic torque, and β models viscous damping.

Dividing both sides by J (and absorbing constants into parameters), we write:

$$\dot{\omega} = -\beta\omega - \delta\theta - k\theta^3 + u \quad (4.14)$$

Defining the states as:

$$x_1 = \theta, \quad x_2 = -\frac{1}{2}\omega \quad (4.15)$$

The system can be written in state-space form as follows:

$$\dot{x}_1 = \dot{\theta} = \omega \quad (4.16)$$

$$\omega = -2x_2 \quad (4.17)$$

This way we get the first state equation:

$$\dot{x}_1 = -2x_2 \quad (4.18)$$

For the second state equation, we start by differentiating x_2 .

$$\dot{x}_2 = -\frac{1}{2}\dot{\omega} \quad (4.19)$$

Lastly, we substitute:

$$\omega = -2x_2 \text{ and } \theta = x_1 \quad (4.20)$$

Simplified, we get this equation:

$$\dot{x}_2 = \frac{\beta}{2}(-2x_2) + \frac{\delta}{2}x_1 + \frac{k}{2}x_1^3 - \frac{1}{2}u \quad (4.21)$$

$$\dot{x}_2 = -\beta(x_2) + \frac{\delta}{2}x_1 + \frac{k}{2}x_1^3 - \frac{1}{2}u \quad (4.22)$$

The scaling of time and inputs is introduced to normalize the dynamics of the motor and develop a canonical nonlinear state-space representation. Such a representation is appropriate because it enables the primary physical dynamics of the motors to be preserved.

Firstly, a new variable is introduced to mathematically achieve the time scaling: $\tau = \alpha t$

This implies that: $\frac{d}{dt} = \alpha \frac{d}{d\tau}$

$$\dot{x}_2 = -\frac{\beta}{\alpha}x_2 + \frac{\delta}{2\alpha}x_1 + \frac{k}{2\alpha}x_1^3 - \frac{1}{2\alpha}u \quad (4.23)$$

When we apply $\frac{2}{\alpha} = 2 \rightarrow \alpha = 1, \frac{\beta}{\alpha} = 3, \frac{\delta}{2\alpha} = \delta, \frac{k}{2\alpha} = \frac{1}{3},$

we get the final normalized system:

$$\dot{x}_1 = -2x_2 \quad (4.24)$$

$$\dot{x}_2 = \delta x_1 - \frac{1}{3}x_1^3 - 3x_2 + u \quad (4.25)$$

With the state-space formulation normalized, the equations above allow for the modeling of the BLDC system in MATLAB/Simulink.

3.5 Modeling of the BLDC motor in MATLAB

The chosen state variables describe the mechanical motion of the BLDC motor rotor, namely its position and speed. The second state is introduced as a scaled form of the rotor speed in order to express the motor dynamics in a convenient mathematical form for analysis and control design, while the control input represents the electromagnetic torque generated by the stator currents.

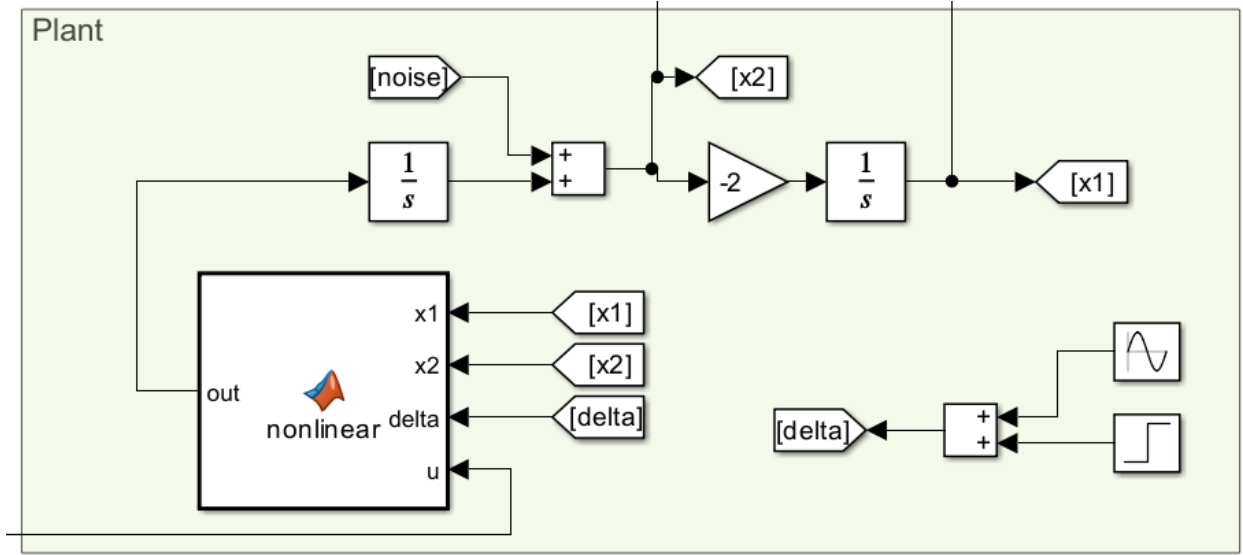


Figure 4.14 Simulink model of the BLDC motor

The system was modeled in Simulink using two integrators for the state variables, a gain block for linear damping, summation blocks for signal combination, and external inputs for control and disturbance. A MATLAB Function Block was also included for the nonlinear dynamics where the function with inputs x_1 , x_2 , δ and u was defined as:

function out = nonlinear (x_1 , x_2 , δ , u)

out = $\delta * x_1 - 1/3 * x_1^3 - 3 * x_2 + u$;

The output of the function is the second state equation $\dot{x}_2$, and once integrated represents the state variable x_2 . Then, from the first state equation, we compute once we multiply x_2 by gain of -2 .

The parameter δ is modeled as a time-varying coefficient, representing the linear magnetic stiffness of the motor, which varies during the operation. It is computed as the sum of a step signal and a sine wave function generating a sinusoidal signal. The step function models a sudden parameter change, while the sinusoidal component models periodic effect induced from the process of commutation.

4.6 Cascade control structure in BLDC motor drives

In the Simulink model and the state equations, the signal annotated as u can be seen. This signal represents the control output from the regulators and is accepted as an input signal into the Simulink modeled system.

In practical BLDC motor drive systems, control is commonly organized in a cascade structure. This means that different control loops operate in series at different time scales.

In this regard, control loops are usually referred to as inner and outer control loops. The inner control loop is the current loop and, consequently, the torque loop since the torque is proportional to the current. It has fast dynamics compared to the outer control loop and is always placed inside the ESC. It uses PWM signals and electronic commutation to switch phases. This loop ensures the exact and fast implementation of the torque commands from the motor. However, the outer control loop has slower dynamics and is responsible for the control of the speed and position of the motor. For the speed and position control to occur, there is the need for the use of the feedback loop that provides the control loop with the speed and position of the motor. This is always used where the BLDC motors use the control through the cascade control methodology, whereby the position loop is the outermost control loop and is placed outside the speed control loop and compares the desired position with the actual position to determine the required change in the torque over time to track the desired path.

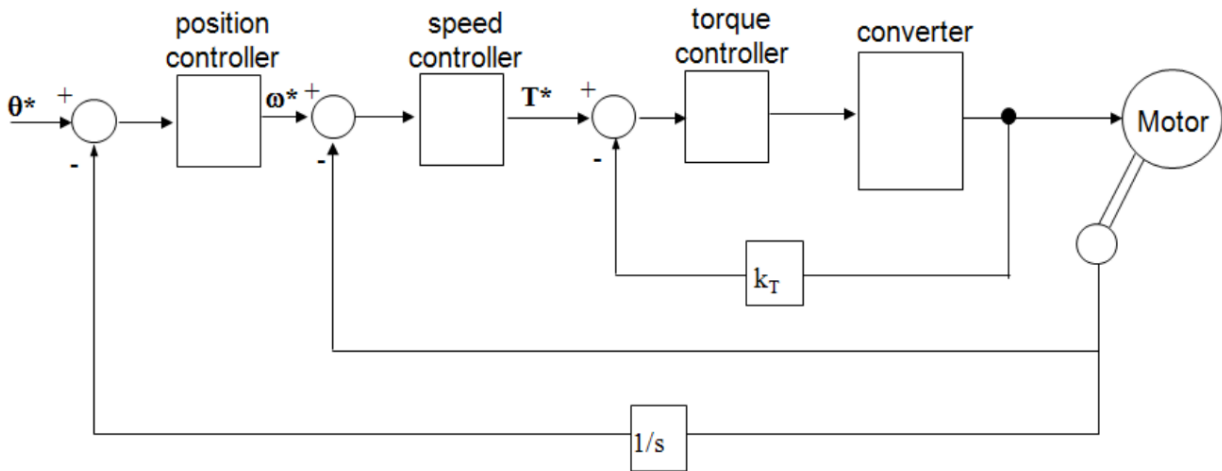


Figure 4.15 Cascade control structure for electrical motor

In this master's thesis, the BLDC motor model used is the mechanical state-space model, where the state variable x_1 represents the rotor position θ , and the time derivative of x_1 represents the rotor angular velocity. The reference trajectory for the state variable x_1 is defined, which implies that the control action for the system is for position tracking and not for speed regulation. Hence, the control action for the proposed work lies at the motion control level.

The control input u appears in the mechanical equations explicitly in Newton's second law of rotational motion:

$$J\dot{\omega} = \Sigma\tau \tag{4.26}$$

This means that our control signal u represents a scaled electromagnetic torque command.

The electrical dynamics of the motor, inverter switching times, PWM signal processing, and commutation logic are not represented in this thesis. Rather, it is assumed that:

- The internal current (torque) control loop is fast and well-designed.
- The ESC can track the torque command u accurately.
- PWM and commutation effects are handled by the ESC itself.

With this assumption, the design of the mechanical control system is decoupled from the inverter and is in accordance with real-world implementations.

However, it is important to note that the signal u is neither a PWM signal nor a voltage command. Rather, it forms a high-level torque request, which is a result of the motion controller. In a real-world system, the signal would then be processed by the electronic speed controller, which converts it to a current reference and uses pulse-width modulation and commutation to apply it to the motor. Thus, the controller developed in this master's thesis works in collaboration with the ESC and does not replace it.

5. System analysis of the BLDC motor model

Before control logic is applied to the BLDC motor model, a system analysis is needed. The first step in system analysis is to determine what type of system it is.

A second-order dynamic system is one whose response can be described by a second-order ordinary differential equation (ODE). A second-order ODE is one in which the highest-order derivative is a second derivative.

Given the state-space model, the state equations can be expressed using a second-order differential equation in a single variable.

With the differentiation of the first state equation, a dependency is shown:

$$\ddot{x}_1 = -2\dot{x}_2 \quad (5.1)$$

Using the second state equation, we can rewrite the previous equation as:

$$\ddot{x}_1 = -2\delta x_1 + \frac{2}{3}x_1^3 + 6\dot{x}_1 - 2u \quad (5.2)$$

This confirms that the system is second order and belongs to the class of nonlinear oscillatory systems (proven by the use of the cubic term in the second state equation).

5.1 Open-Loop equilibrium analysis

An equilibrium point in the state space is a point at which the rates-of-change for all of the state variables are zero [17].

For the beforementioned system, equilibrium points are obtained by solving:

$$\dot{x}_1 = 0, \quad \dot{x}_2 = 0 \quad (5.3)$$

Solving the equilibrium equations reveals that the system exhibits two distinct dynamical regimes depending on the magnitude of the control input u . The section below shows the results from the equilibrium analysis done in MATLAB.

```
Number of real equilibria: 3
Equilibrium 1:  $x_1 = 0.0000$ ,  $x_2 = 0.0$ 
Eigenvalues:  $-0.3820 + 0.0000i$ ,  $-2.6180 + 0.0000i$ 
Stability: Stable (Stable Node)
Equilibrium 2:  $x_1 = 1.2247$ ,  $x_2 = 0.0$ 
Eigenvalues:  $0.5616 + 0.0000i$ ,  $-3.5616 + 0.0000i$ 
Stability: Unstable (Saddle Point)
Equilibrium 3:  $x_1 = -1.2247$ ,  $x_2 = 0.0$ 
Eigenvalues:  $0.5616 + 0.0000i$ ,  $-3.5616 + 0.0000i$ 
Stability: Unstable (Saddle Point)
```

```

--- u_real = 500 (u_norm = 0.1250) ---
Regime: 3-equilibrium regime
Number of real equilibria: 3
Equilibrium 1:  $x_1 = 1.3345$ ,  $x_2 = 0.0$ 
Eigenvalues:  $0.6936 + 0.0000i$ ,  $-3.6936 + 0.0000i$ 
Stability: Unstable (Saddle Point)
Equilibrium 2:  $x_1 = -1.0726$ ,  $x_2 = 0.0$ 
Eigenvalues:  $0.3843 + 0.0000i$ ,  $-3.3843 + 0.0000i$ 
Stability: Unstable (Saddle Point)
Equilibrium 3:  $x_1 = -0.2620$ ,  $x_2 = 0.0$ 
Eigenvalues:  $-0.3222 + 0.0000i$ ,  $-2.6778 + 0.0000i$ 
Stability: Stable (Stable Node)

--- u_real = 1000 (u_norm = 0.2500) ---
Regime: 1-equilibrium regime
Number of real equilibria: 1
Equilibrium 1:  $x_1 = 1.4237$ ,  $x_2 = 0.0$ 
Eigenvalues:  $0.8030 + 0.0000i$ ,  $-3.8030 + 0.0000i$ 
Stability: Unstable (Saddle Point)

--- u_real = 1500 (u_norm = 0.3750) ---
Regime: 1-equilibrium regime
Number of real equilibria: 1
Equilibrium 1:  $x_1 = 1.5000$ ,  $x_2 = 0.0$ 
Eigenvalues:  $0.8979 + 0.0000i$ ,  $-3.8979 + 0.0000i$ 
Stability: Unstable (Saddle Point)

```

The numerical equilibrium analysis illustrates the evolution of the open-loop system's behavior for the nonlinear BLDC motor model with increasing control input u . The results shown above indicate the existence of different regions, each having a different number of equilibrium points. For low levels of control input, the system has three real equilibrium points. In this region, there is one locally asymptotically stable equilibrium point and two unstable saddle points. At the point where the control input u is zero, the stable equilibrium point is at the origin. This represents the physical system where the rotor is at a neutral position and has zero velocity. The existence of the negative real eigenvalues shown above implies that the system will return to the stable equilibrium point after a small disturbance. This implies that the system can approach a stable operating point if the system's initial condition lies close to the stable equilibrium point for low levels of the control input. However, as the input increases, the system approaches another regime. In this regime, a single real equilibrium exists, and it is unstable, i.e. an unstable saddle point. It implies that for higher torque commands, the open loop dynamics are unstable and will diverge as shown in Figure 5.2.

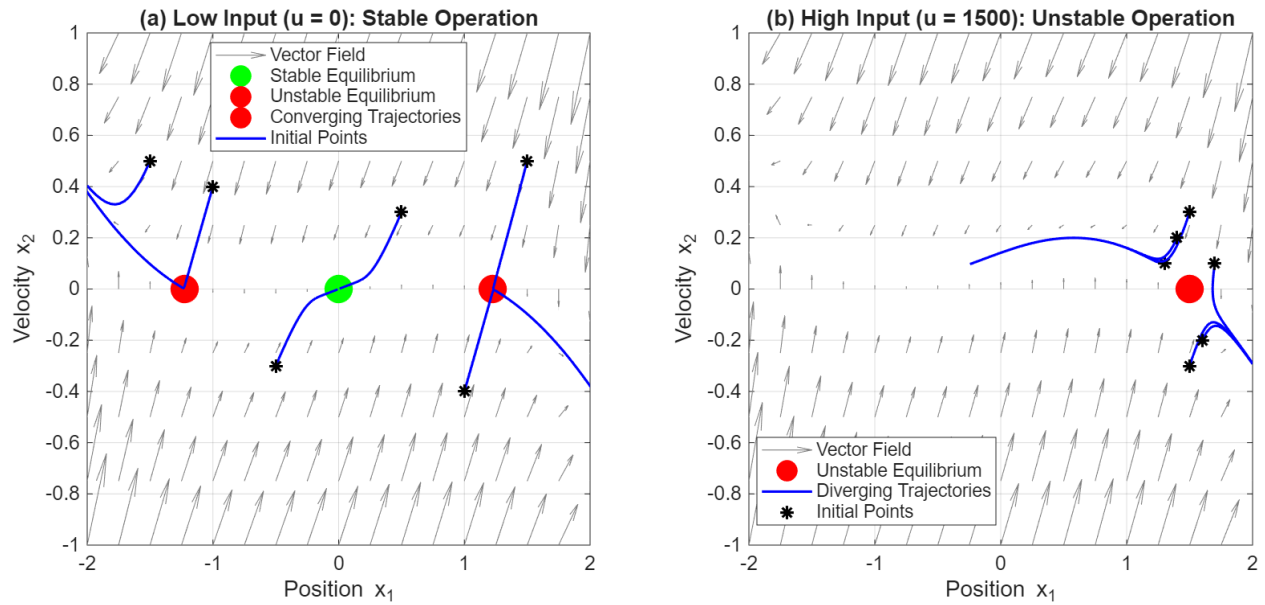


Figure 4.1 Open-loop dynamics

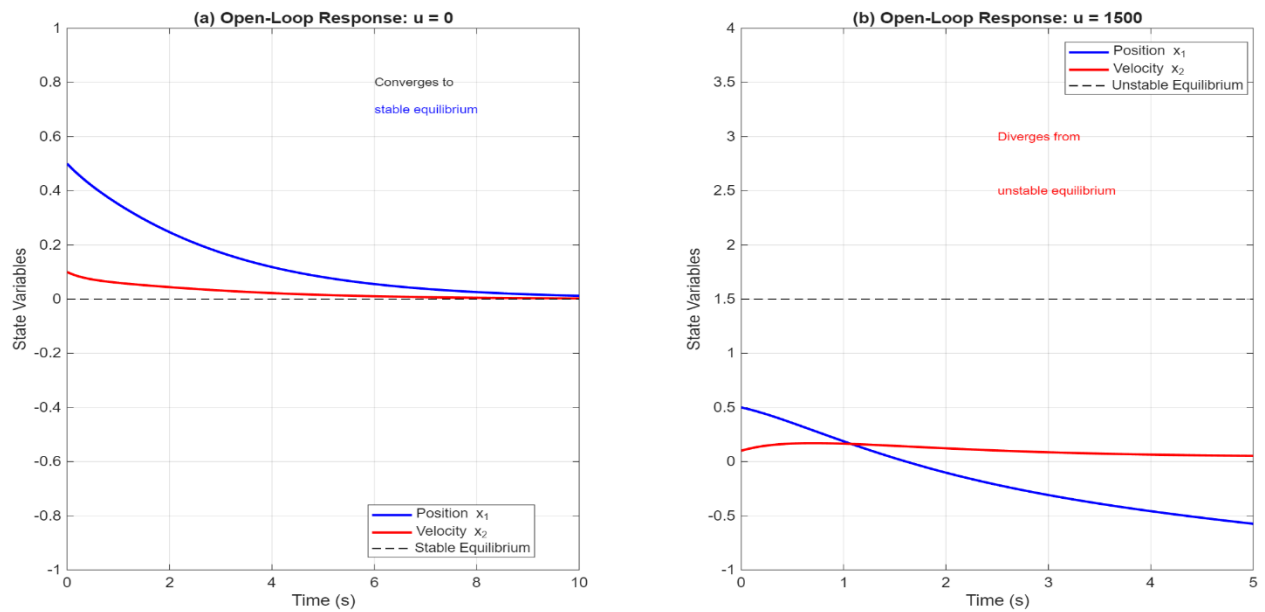


Figure 5.2 Open-loop response

The results, as well as the two emerging regions, show that the open-loop BLDC motor model is naturally stable in only a small portion of the operating range, while for more than 90% of the input range, the system does not converge to a stable equilibrium without active control. Consequently, this means that a controller must generate stability and regulate the system. Another important point to note is that the bifurcation point depends on the parameter δ which varies due to temperature and load changes as well as magnetic saturation. That is why in this master's thesis, adaptive control is applied in order to ensure stability across varying operating conditions.

6. Derivation of control algorithms for the BLDC motor model

The state-space equations for the BLDC motor model derived in Chapter 4 are as follows:

$$\dot{x}_1 = -2x_2 \quad (4.24)$$

$$\dot{x}_2 = \delta x_1 - \frac{1}{3}x_1^3 - 3x_2 + u \quad (4.25)$$

Based on these equations and using the methodology for adaptive backstepping, adaptive SMC, the hybrid approach of combining backstepping regulation with SMC, and lastly the PD control with nonlinear compensation, the control laws are derived in this chapter.

6.1 Adaptive backstepping regulator

Firstly, the systems state equations are needed, which basically confirms Step zero of this technique, that the system is put in strict feedback structure.

The state equations reveal a second-order nonlinear system with an unknown parameter. The control signal is u and the unknown parameter is δ .

The objective is for x_1 to follow a dynamic reference signal, noted as $x_{d1}(t)$.

The reference derivative is $x_{d2}(t) = \dot{x}_{d1}(t)$.

Step 1 is to define the tracking error as:

$$e_1 = x_{d1} - x_1 \quad (6.1)$$

Differentiating and using $\dot{x}_1 = -2x_2$ yields:

$$\dot{e}_1 = \dot{x}_{d1} - \dot{x}_1 \rightarrow \dot{e}_1 = x_{d2} + 2x_2 \quad (6.2)$$

According to Step 1 of the methodology, a Lyapunov function is chosen as:

$$V_1 = \frac{1}{2}e_1^2 \quad (6.3)$$

Its derivative is:

$$\dot{V}_1 = e_1 \dot{e}_1 = e_1(x_{d2} + 2x_2) \quad (6.4)$$

At this point, the derivative of the first function cannot be made negative definite using only e_1 , because it contains the term x_2 , which is not directly controlled. To stabilize e_1 , the behavior of

x_2 must be shaped. This idea is achieved through the composite error variable represented by the virtual state z_2 .

$$z_2 = k_1 e_1 + 2x_2 \quad (6.5)$$

The justification behind the choice made for the virtual state to be defined as is, is that it behaves as a damped error coordinate. It combines the k_1 gain, which affects how strongly the error contributes, with the motion term x_2 . Now x_2 can be expressed as:

$$2x_2 = z_2 - k_1 e_1 \quad (6.7)$$

It is then substituted into the following equation:

$$\dot{e}_1 = x_{d2} + 2x_2 = x_{d2} + z_2 - k_1 e_1 \quad (6.8)$$

Lastly, we get the Lyapunov function as:

$$\dot{V}_1 = e_1(x_{d2} + z_2 - k_1 e_1) = -k_1 e_1^2 + e_1 z_2 + e_1 x_{d2} \quad (6.9)$$

The key observation in this step is that a stabilizing term already appears which is negative, as was expected, but the derivation also contains a coupling term preventing the Lyapunov function from being negative definite.

Step 2 is the next step of stabilizing the second subsystem by defining the second Lyapunov function as:

$$V_2 = V_1 + \frac{1}{2} z_2^2 = \frac{1}{2} e_1^2 + \frac{1}{2} z_2^2 \quad (6.10)$$

The derivative is:

$$\dot{V}_2 = \dot{V}_1 + z_2 \dot{z}_2 = -k_1 e_1^2 + e_1 z_2 + e_1 x_{d2} + z_2 \dot{z}_2 \quad (6.11)$$

Now, to solve for the last term, the derivative of z_2 is needed:

$$\dot{z}_2 = k_1 \dot{e}_1 + 2\dot{x}_2 \quad (6.12)$$

$$\dot{z}_2 = k_1(x_{d2} + 2x_2) + 2\left(\delta x_1 - \frac{1}{3}x_1^3 - 3x_2 + u\right) \quad (6.13)$$

$$\dot{z}_2 = k_1 x_{d2} + (2k_1 - 6)x_2 + 2\delta x_1 - \frac{2}{3}x_1^3 + 2u \quad (6.14)$$

The problematic terms in the derivative of the second Lyapunov function can be cancelled out by choosing a desired or target dynamic for $\dot{z}_2$.

Therefore, $\dot{z}_2$ desired can be chosen as:

$$\dot{z}_2 \text{ desired} = k_1 x_{d2} - z_2 - e_1 \quad (6.15)$$

To check if this is mathematically correct, $\dot{z}_2$ *desired* can be substituted in the Lyapunov function for regulation case $x_{d2} = 0$.

$$\dot{V}_2 = -k_1 e_1^2 + e_1 z_2 + e_1 x_{d2} + z_2 (k_1 x_{d2} - z_2 - e_1) \quad (6.16)$$

This ensures that:

$$\dot{V}_2 = -k_1 e_1^2 - z_2^2 < 0 \quad (6.17)$$

The parameter k_1 shapes the closed-loop behavior. It creates a stabilizing term, where k_1 is multiplied by the square of error e_1 . The implication of the gain k_1 here is that by increasing k_1 , the convergence rate of the primary error is also increased.

The last step is to ensure that $\dot{z}_2 = \dot{z}_2$ *desired* by computing for u with which the control signal is finally computed.

$$k_1 x_{d2} + (2k_1 - 6)x_2 + 2\delta x_1 - \frac{2}{3}x_1^3 + 2u = k_1 x_{d2} - z_2 - e_1 \quad (6.18)$$

$$(2k_1 - 6)x_2 + 2\delta x_1 - \frac{2}{3}x_1^3 + 2u = -z_2 - e_1 \quad (6.19)$$

The signal u can be explicitly expressed as:

$$2u = -(2k_1 - 6)x_2 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 \quad (6.20)$$

$$2u = -2k_1 x_2 + 6x_2 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 \quad (6.21)$$

Once $2x_2$ is substituted, the following equation is computed:

$$2u = -k_1(z_2 - k_1 e_1) + 6x_2 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 \quad (6.22)$$

$$2u = 6x_2 - k_1 z_2 + k_1^2 e_1 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 \quad (6.23)$$

Finally,

$$u = \frac{1}{2} \left(6x_2 - k_1 z_2 + k_1^2 e_1 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 \right) \quad (6.24)$$

With the control output computed, the methodology of the backstepping regulator was followed from Step zero to Step three. The next part is computing the adaptation law for the unknown parameter by following the proposed methodology by Kanellakopoulus.

Firstly, to handle the uncertainty, an estimate is introduced and the estimation error can be written as:

$$\delta^\sim = \delta - \delta^\wedge \quad (6.25)$$

The adaptive equivalent version of the control signal uses the estimate in place of the unknown parameter:

$$u = \frac{1}{2} \left(6x_2 - k_1 z_2 + k_1^2 e_1 - 2\delta^\wedge x_1 + \frac{2}{3} x_1^3 - z_2 - e_1 \right) \quad (6.26)$$

The augmented Lyapunov function can be written as:

$$V_a = V_1 + \frac{1}{2} z_2^2 + \frac{1}{2} \delta^{\sim 2} = \frac{1}{2} e_2^2 + \frac{1}{2} z_2^2 + \frac{1}{2} \delta^{\sim 2} \quad (6.27)$$

$$\dot{V}_a = e_1 \dot{e}_1 + z_2 \dot{z}_2 + \delta^{\sim} \dot{\delta}^{\sim} \quad (6.28)$$

Since the unknown parameter is constant,

$$\dot{\delta}^{\sim} = -\dot{\delta}^\wedge \quad (6.29)$$

$$\dot{V}_a = (-k_1 e_1^2 + e_1 z_2 + e_1 x_{d2}) + (z_2(k_1 x_{d2} - z_2 - e_1) + 2z_2 \delta^{\sim} x_1) + \delta^{\sim} (-\dot{\delta}^\wedge) \quad (6.30)$$

$$\dot{V}_a = -k_1 e_1^2 - z_2^2 + x_{d2}(e_1 + k_1 z_2) + \delta^{\sim} (2z_2 x_1 - \dot{\delta}^\wedge) \quad (6.31)$$

With the condition being that the augmented Lyapunov function must be smaller than 0, the adaptation law can be chosen as:

$$\dot{\delta}^\wedge = 2z_2 x_1 \quad (6.32)$$

The Lyapunov proof is shown for the constant-reference case ($x_{d2}=0$) to clearly explain the backstepping design. Since the implemented reference is sinusoidal ($x_{d2} \neq 0$), tracking performance is demonstrated through simulation plots of x_1 versus x_{d1} and the corresponding tracking error.

After deriving the adaptive law for the unknown parameter, it is noted that the gain k_1 also has a dominant influence and is affecting the closed-loop behavior because it appears in both the primary error term and the virtual state.

In the adaptive backstepping procedure implemented until now, the parameter k_1 was a constant. However, it is possible to consider a time-varying gain $k_1(t)$ which would influence transient response during the operation. That means the virtual state variable can be written as:

$$z_2(t) = k_1 e_1(t) + 2x_2(t) \quad (6.33)$$

With the derivation being:

$$\dot{z}_2 = k_1(t) \dot{e}_1 + 2\dot{x}_2 + \dot{k}_1(t) e_1 \quad (6.34)$$

The additional term containing the first derivative of the k_1 parameter is the differentiating term from the backstepping procedure that was followed.

This term further appears in the Lyapunov function as a product $z_2 \dot{k}_1 e_1$, which is not canceled by the control law that was derived. By keeping the same control law, the adjustment of k_1 does not become part of the Lyapunov derived control signal. Instead, it can be interpreted as an additional heuristic modification:

$$k_1(t^+) = \text{sat}_{[k_{1,min}, k_{1,max}]}(k_1(t) + \gamma k_1 |e_1|), |e_1| > e_H \quad (6.35)$$

What this means is that when $|e_1|$ is large, k_1 is increased to make the response more aggressive and drive the error down faster. When $|e_1|$ is small, k_1 is reduced slightly to avoid unnecessary control effort and to keep the response smoother.

The motivation for testing the adjustment of the parameter k_1 is that a fixed k_1 is always a compromise between fast tracking and smooth behavior. This tuning rule was explored to see whether that compromise can be improved for the operating conditions considered here.

6.2 Adaptive sliding mode control (SMC) regulator

Starting from the nonlinear plant, the tracking error is determined:

$$e_1 = x_{d1} - x_1 \quad (6.36)$$

$$\dot{e}_1 = \dot{x}_{d1} - \dot{x}_1 = \dot{x}_{d2} + 2x_2 \quad (6.37)$$

In the first step, the sliding surface is defined:

$$\sigma = k_{slide} e_1 + \dot{e}_1 + \mu \int e_1 dt \quad (6.38)$$

- $k_{slide} e_1$: proportional action on position error
- $\mu \int e_1 dt$: integral action to reduce steady-state offset

$$\dot{\sigma} = k_{slide} \dot{e}_1 + \ddot{e}_1 + \mu e_1 \quad (6.39)$$

$$\ddot{e}_1 = \dot{x}_{d2} + 2\dot{x}_2 \quad (6.40)$$

$$\ddot{e}_1 = \dot{x}_{d2} + 2 \left(\delta x_1 - \frac{1}{3} x_1^3 - 3x_2 + u \right) = \dot{x}_{d2} + 2\delta x_1 - \frac{2}{3} x_1^3 - 6x_2 + 2u \quad (6.41)$$

The derived terms for the derivatives of the error are then placed into the sliding variable:

$$\dot{\sigma} = k_{slide}(\dot{x}_{d2} + 2x_2) + \left(\dot{x}_{d2} + 2\delta x_1 - \frac{2}{3} x_1^3 - 6x_2 + 2u \right) + \mu e_1 \quad (6.42)$$

$$\dot{\sigma} = \dot{x}_{d2} + 2\delta x_1 - \frac{2}{3} x_1^3 - 6x_2 + 2u + k_{slide}(\dot{x}_{d2} + 2x_2) + \mu e_1 \quad (6.43)$$

Then in the second step, the derived sliding variable is set to be equal to the reaching law:

$$\dot{x}_{d2} + 2\delta x_1 - \frac{2}{3} x_1^3 - 6x_2 + 2u + k_{slide}(\dot{x}_{d2} + 2x_2) + \mu e_1 = -K_{prop}\sigma - K_{switch} \text{sat} \left(\frac{\sigma}{\epsilon} \right) \quad (6.44)$$

Lastly, solved for u , the control law is derived:

$$u = \frac{1}{2} \left(-K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) - \dot{x}_{d2} - 2\delta x_1 + \frac{2}{3}x_1^3 + 6x_2 - k_{slide}(x_{d2} + 2x_2) - \mu e_1 \right) \quad (6.45)$$

For the adaptation law, the estimation error is defined as:

$$\delta^\sim = \delta - \delta^\wedge \quad (6.46)$$

This means that the sliding variable has an additional term:

$$\dot{\sigma} = -K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) + 2\delta^\sim x_1 \quad (6.47)$$

The Lyapunov function is chosen as:

$$V = \frac{1}{2}\sigma^2 + \frac{1}{2\gamma_{adapt}}\delta^\sim{}^2, \quad \gamma_{adapt} > 0 \quad (6.48)$$

$$\dot{V} = \sigma\dot{\sigma} - \frac{1}{\gamma_{adapt}}\delta^\sim\dot{\delta}^\sim \quad (6.49)$$

$$\dot{V} = \sigma \left(-K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) + 2\delta^\sim x_1 \right) - \frac{1}{\gamma_{adapt}}\delta^\sim\dot{\delta}^\wedge \quad (6.50)$$

With this derivation, the adaptive law is chosen:

$$\dot{\delta}^\wedge = 2\gamma_{adapt}\sigma x_1 \quad (6.51)$$

This makes the Lyapunov function:

$$\dot{V} = K_{prop}\sigma^2 - K_{switch}\sigma sat\left(\frac{\sigma}{\varepsilon}\right) \leq 0 \quad (6.52)$$

The control law and the adaptation of the unknown parameter are coded in the Twin CAT's block according to the equations above.

6.3 Hybrid approach of backstepping regulator with added sliding surface

From the nominal backstepping procedure, Step 0 was defined as rewriting the plant in strict feedback form and in Step 1 the tracking error was defined. These steps are left unchanged.

$$\dot{e}_1 = \dot{x}_{d1} - \dot{x}_1 \rightarrow \dot{e}_1 = \dot{x}_{d2} + 2x_2 \quad (6.53)$$

$$V_1 = \frac{1}{2}e_1^2 \quad (6.54)$$

In Step 2, the backstepping composite error z_2 is defined as:

$$z_2 = k_1 e_1 + 2x_2 \quad (6.55)$$

$$\dot{z}_2 = k_1 x_{d2} + (2k_1 - 6)x_2 + 2\delta x_1 - \frac{2}{3}x_1^3 + 2u \quad (6.56)$$

In the combined approach, the reaching law of the sliding regulation is added to the desired derivative of z_2 . While in SMC the practical reaching law was defined as:

$$\dot{\sigma} = -K_{prop}\sigma - K_{switch}sat\left(\frac{\sigma}{\varepsilon}\right) \quad (6.57)$$

In the hybrid design, the same idea is applied by modifying the backstepping target dynamics:

$$z_{2desired}^{hyb} = k_1 x_{d2} - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) \quad (6.58)$$

$$k_1 x_{d2} - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) = k_1 x_{d2} + (2k_1 - 6)x_2 + 2\delta x_1 - \frac{2}{3}x_1^3 + 2u \quad (6.59)$$

$$-z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) = (2k_1 - 6)x_2 + 2\delta x_1 - \frac{2}{3}x_1^3 + 2u \quad (6.60)$$

$$2u = -(2k_1 - 6)x_2 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) \quad (6.61)$$

$$2u = -2k_1 x_2 + 6x_2 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) \quad (6.62)$$

$$2u = -k_1(z_2 - k_1 e_1) + 6x_2 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) \quad (6.63)$$

$$2u = 6x_2 - k_1 z_2 - k_1^2 e_1 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) \quad (6.64)$$

$$u = \frac{1}{2}\left(6x_2 - k_1 z_2 - k_1^2 e_1 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right)\right) \quad (6.65)$$

As can be observed, the backstepping and sliding approach adds a sliding term to the control signal u . This can be essentially represented as:

$$u_{BS+SCM} = u_{BS} - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) \quad (6.66)$$

The adaptive law stays the same so that the following is true:

$$u = \frac{1}{2}\left(6x_2 - k_1 z_2 - k_1^2 e_1 - 2\delta x_1 + \frac{2}{3}x_1^3 - z_2 - e_1 - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right)\right) \quad (6.67)$$

Lastly, the equation for the control law with the updated estimated parameter can also be written as:

$$u_{aBS+SCM} = u_{aBS} - K_{slide}sat\left(\frac{z_2}{\varepsilon}\right) \quad (6.68)$$

Where u_{aBS} is the adaptive equivalent version of the control signal that uses the estimate in place of the unknown parameter as seen in Chapter 6.1.

6.4 PD Regulation with nonlinear compensation

This method uses tracking error e_1 and the composite error z_2 as defined in the backstepping methodology.

$$e_1 = x_{d1} - x_1; \quad z_2 = k_1 e_1 + 2x_2 \quad (6.69)$$

The adaptive law also stays the same:

$$\dot{\delta}^\wedge = 2z_2x_1 \quad (6.70)$$

The nonlinear compensation term is derived from the system model:

$$x_2 = f(x, \delta) + 2u, \quad f(x, \delta) = \delta x_1 - \frac{1}{3}x_1^3 - 3x_2 \quad (6.71)$$

The compensation idea is to choose:

$$u_{comp} = -2\delta^\wedge x_1 + \frac{2}{3}x_1^3 - 6x_2 \quad (6.72)$$

This way, the compensating regulation handles the structured nonlinear physics of the system, while the chosen PD controller adds to the final control signal and only has to correct the remaining mismatch and reject the disturbances.

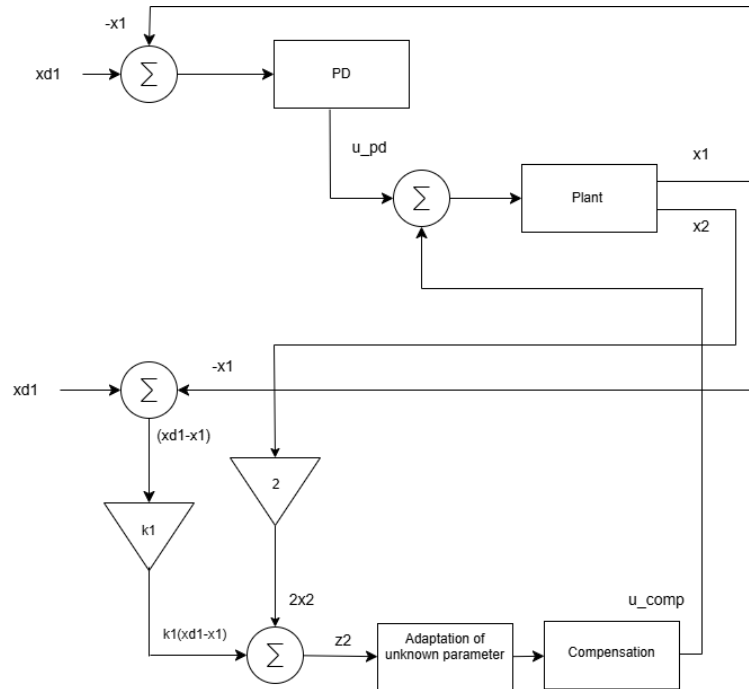


Figure 5.1 Flow diagram - PD control signal with nonlinear compensation

The proportional term applies immediate correction to the error. However, a standalone term can cause oscillation or overshoot. The derivative term acts as a damping and reduces the overshoot but is nevertheless sensitive to noise.

In Figure 6.1, the control logic is visualized for better representation, where both the compensating control as well as the feedback PD control are shown. The control outputs of both controllers are lastly summed together and given as a final control input to the plant in Simulink.

$$u = u_{PID} + u_{comp} \quad (6.73)$$

Although applied to a nonlinear plant, the inclusion of the compensating term simplifies the closed loop behavior to a second order system. However, final control law (6.73) remains nonlinear due to the compensation term, while the PD term shapes the control output for which tuning is needed.

7. Hardware-in-the-loop structure

In Chapter 7, the HIL architecture is presented and is used to validate the control strategies developed in the previous chapters.

The HIL structure consists of a Beckhoff programmable logic controller (PLC), and a mathematical model of a brushless DC motor (BLDC) implemented in MATLAB/ Simulink. According to the affirmed types of HIL structures, the use of a real PLC makes this structure controller-in-the-loop (CHIL).

In this setup, measurements are obtained from the simulated BLDC motor model and are transmitted to the PLC, where the regulator logic is executed in real time. The control signal is then sent back to the Simulink model, closing the loop through a bidirectional channel. The communication between the PLC and MATLAB/Simulink is done through Beckhoff's own ADS Protocol.

The primary objective of this experimental framework is to test and validate the control laws from the backstepping regulator, SMC, the hybrid approach of backstepping and SMC, and lastly the compensation of the model coupled with PID Regulator. Another objective of the master's thesis is to determine which regulator would be best suited for real-world implementation and regulation of the BLDC motor when there are network delays.

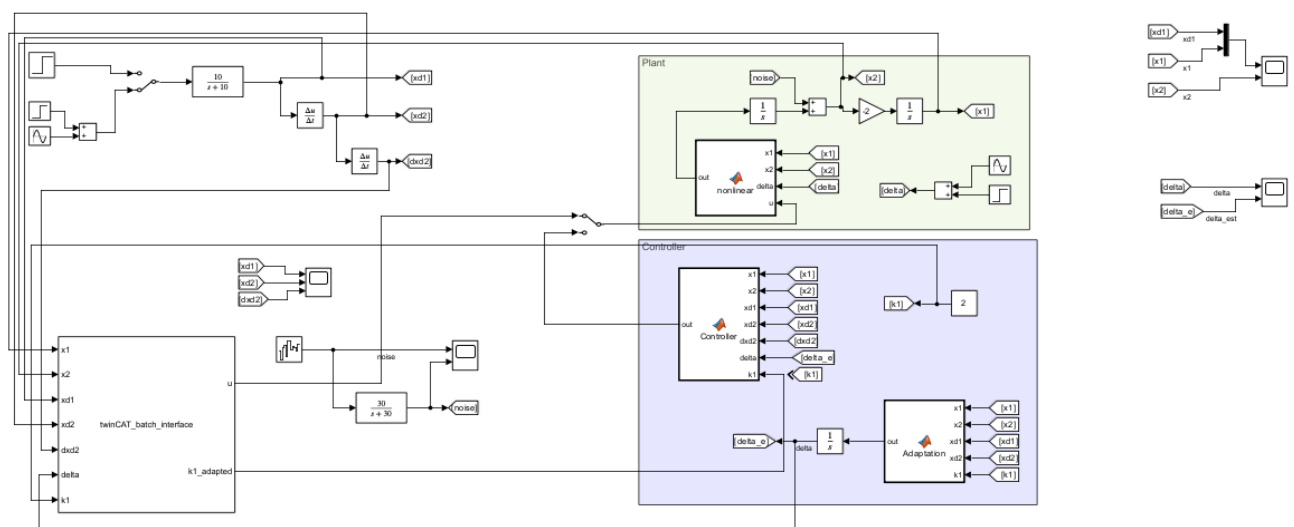


Figure 6.1 HIL Structure implemented in Simulink

Figure 7.1 shows the Simulink model of the BLDC motor, already discussed in Chapter 4.5 derived from the state-space equations. Furthermore, it shows the connection with Twin CAT and the direction of transmission of the signal.

7.1 Simulink subsystems

The Simulink model consists of several subsystems which are needed to properly model noise, the time-varying parameter, as well as setting the reference trajectory which the regulators aim to follow accurately.

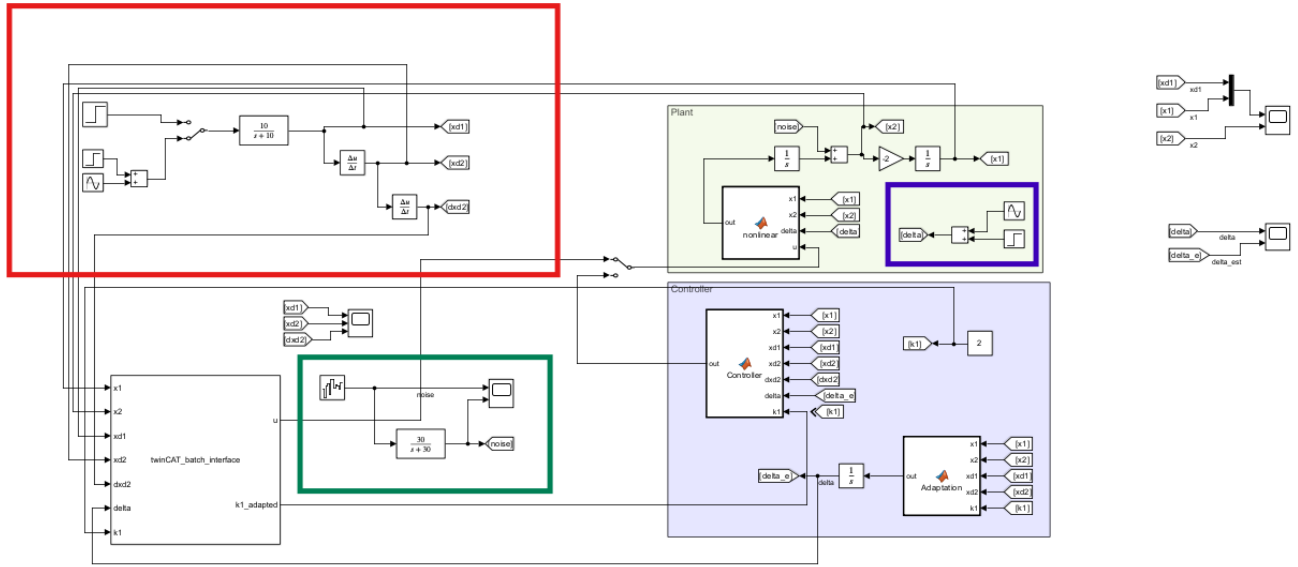


Figure 7.2 Subsystems of HIL Structure

- Dynamic reference subsystem (subsystem highlighted in red)
This block generated the desired rotor position trajectory x_{d1} and its derivatives, which are required by the control laws derived for the regulators. Particularly, the backstepping regulator and SMC regulator explicitly depend on the first and second derivatives of the reference signal.
The reference signal is constructed as a combination of elementary signals, more specifically step and sinusoidal signals, and is passed through a first-order low-pass filter in the form of:

$$G(s) = \frac{\omega_c}{s + \omega_c} \quad (7.1)$$

Where $\omega_c = 10$ rad/s. The purpose of the filter is to limit the bandwidth of the reference trajectory and ensure that the signal is continuously differentiable. Without applying

filtering, differentiation of sharp reference signals would lead to unrealistically large derivative values which would in turn affect the control signal.

The filtered reference $x_{d1}(t)$ is then differentiated by applying the derivative blocks from MATLAB's library, in order to obtain the reference velocity and acceleration signals:

$$x_{d2}(t) = \dot{x}_{d1}(t), \quad \ddot{x}_{d1}(t) \quad (7.2)$$

- Time-varying parameter subsystem for $\delta(t)$ (subsystem highlighted in blue)

The subsystem highlighted in blue is used to generate the time-varying parameter which appears in the normalized BLDC motor dynamics derived in Chapter 3. This parameter represents the effective linear magnetic stiffness of the motor and is assumed to be uncertain and subject to variation during the operation.

In the normalized state-space model, the parameter δ appears as the coefficient of linear-stiffness term in the rotor dynamics. Thus, rather than assuming the parameter is constant and known, it is modeled as a time-varying signal. The reason for this is to emulate an uncertain effective stiffness that changes during operation.

The parameter is generated in Simulink as:

$$\delta(t) = H(t - 1) + 0.6 \sin(0.9t) \quad (7.3)$$

$$H(t - t_s) = \{0 \text{ if } t < t_s, 1 \text{ if } t \geq t_s\} \quad (7.4)$$

$H(t - t_s)$ is the Heaviside step function [15].

The step block in Simulink represents sudden parameter deviation, which can occur due to operating changes. The sinusoidal signal generated via the Sine Wave block in Simulink introduces cyclic variation to represent ripple effects and other electromagnetic disturbances.

The modeling choice is also on par with the adaptive problematic, i.e. to regulate systems whose model changes over time, resulting in common choice of modeling time-varying parameters as in [6].

- Noise injection (subsystem highlighted in green)

The experimental model also includes an additive noise or disturbance signal to make sure that the model is also replicating real industrial settings such as non-deal measurement signals. Noise is generated using a band-limited white noise source and then passed through a first-order low pass filter to avoid high-frequency components.

7.2 Automation device specification (ADS) protocol

The previously developed regulators are executed on a Beckhoff programmable logic controller using the Twin CAT automation platform. Twin CAT (The Windows Control and Automation Technology) is the real-time environment for Beckhoff's PLCs and integrates PLC runtime and motion control within a unified framework.

In Twin CAT, control programs are executed in cyclic real-time tasks with predefined cycle time. During each cycle, the PLC reads the input signals, executes the control logic, and lastly updates the output variables. The deterministic execution model makes PLCs well suited for implementation in industrial scenarios, where predictable timing and reliability are required.

Communication between the PLC and the simulation environment is achieved by using the Automatic Device Specification (ADS) protocol.

ADS is a Beckhoff-developed communication protocol that enables direct access to Twin CAT runtime variables across different software environments and network interfaces. This protocol is what allows the PLC to read variables (measurements from Simulink) and write the variables (control signal to Simulink's plant).

7.2.1 ADS device configuration

According to [1], Twin CAT treats software components such as the PLC runtime as virtual automation devices that expose their internal variables through a standardized interface called ADS. In this setup, the PLC runtime acts as an ADS server, the external program MATLAB acts as an ADS client, and the communication is handled by the Twin CAT message router which routes ADS messages over TCP/IP.

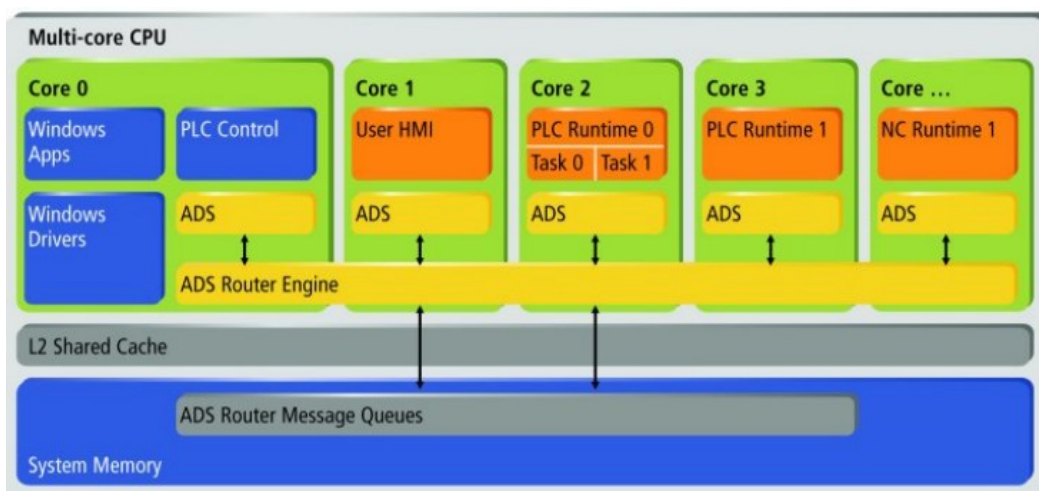


Figure 7.3 ADS protocol structure

Every ADS communication endpoint is defined by:

- AMS NetId
- AMS Port number

In this master's thesis, communication targets the Twin CAT 3 PLC runtime system 1, which is assigned to the predefined port 851.

Table 7.1. ADS Client Server model

ADS-Client	ADS-Server
Request ->	-> Indication
Confirmation <-	<- Response

The implemented ADS communication follows the typical client-server model in which an ADS request sent by the client is processed by the ADS server and returned as a response/confirmation.

The ADS services can be asynchronous, notification-based, or synchronous. In synchronous communication, which is the one implemented, the client thread is suspended for the duration of the ADS communication and returns only once the server's result is available.

ADS communication over Ethernet is encapsulated in an AMS/TCP packet which consists of an AMS/TCP header, an AMS header, and an ADS data field.

The AMS/TCP header contains the length of the data packet, while the AMS Header includes addressing information and control fields such as the command identifier and error handling.

The AMS Header contains the target and source of the AMS NetId and AMS Port as well as protocol control fields as shown in Figure 7.4.

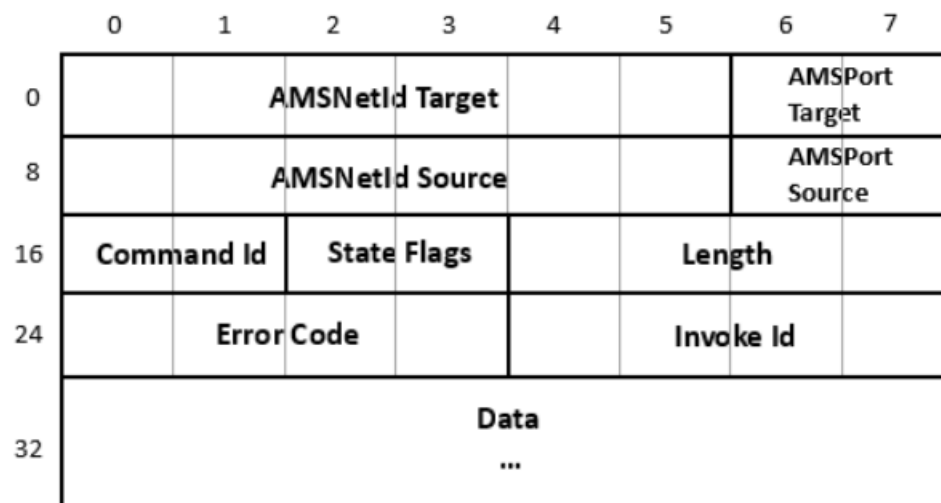


Figure 7.4 AMS header structure

Furthermore, the ADS protocol has predefined command formats for writing and reading variables, more specifically ADS Write and ADS Read.

- ADS Read



Figure 7.6 ADS Read Response command



Figure 7.5 ADS Read Request command

The ADS Read command is used to retrieve the control output computed by the PLC and given to the Simulink model as an input signal.

The ADS Read request contains:

- Index Group
- Index Offset
- Length: number of bytes requested

The ADS Read response returns:

- Result code: indicating success or error
- Length returned
- Data: the requested variable value(s)

In the MATLAB code, ADS Read is performed using symbolic variable handles, for example:

- GVL_HIL.u
- GVL_HIL.k1

Each call to ReadAny() generates a separate ADS Read request and waits for the corresponding response. The returned data is then applied to the Simulink model as the control input for the next simulation step.

In the current implementation, two ADS Read operations are executed per simulation step: one for the control input u and one for the adapted gain k_1 (only for the Backstepping regulator that adapts the value for parameter k_1).

- ADS Write

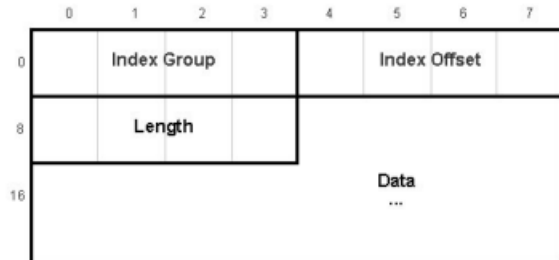


Figure 7.7 ADS Write Request command

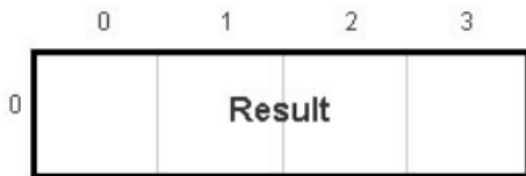


Figure 7.8 ADS Read Response command

The ADS Write command is used to transmit plant state variables and reference signals from Simulink to the PLC. These variables are written into globally accessible PLC variables defined in a Global Variable List (GVL), where they are used by the controller logic during the PLC execution cycle.

An ADS Write request consists of the following fields:

- Index Group: identifies the target memory region or service
- Index Offset: specifies the address within that region
- Length: number of bytes to be written
- Data: contains the values to be written

The corresponding ADS Write response returns a result code, indicating whether the write operation was successful or if an error occurred.

In the MATLAB implementation, symbolic variable access is used instead of explicit index group and offset addressing. For each PLC variable, a variable handle is created using its symbolic name, such as:

- GVL_HIL.x1
- GVL_HIL.x2
- GVL_HIL.xd1

- GVL_HIL.xd2
- GVL_HIL.dxd2
- GVL_HIL.delta
- GVL_HIL.k1

Internally, the Twin CAT ADS library resolves each symbolic variable handle to the corresponding index group and index offset required by the ADS protocol. Each call to WriteAny() results in an individual ADS Write transaction that transfers one double-precision floating-point value (8 bytes) to the PLC. As a result, during each simulation step, multiple ADS Write operations are performed, one for each reference variable, ensuring that the PLC always operates on the most recent data provided by the simulated plant.

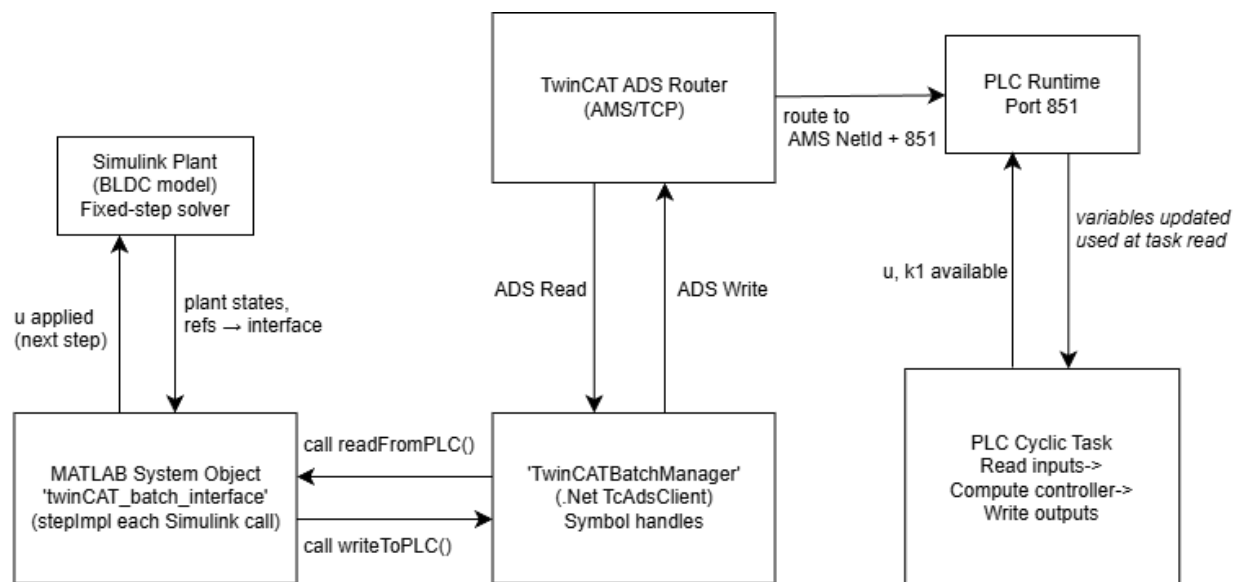


Figure 7.9 Diagram of ADS communication between Simulink and Twin CAT

The logical structure of the ADS-based communication loop is presented in Figure 7.9. The diagram represents a complete control iteration corresponding to a single Simulink solver step. At the beginning of each step, the simulated BLDC plant outputs the current state variables and reference signals. These values are then passed to a MATLAB system object named `twinCAT_batch_interface` which acts as the interface between Simulink and the PLC. Within the `stepImpl` method of the system object, the communication is handled explicitly. The system object then invokes the `TwinCATBatchManager` which uses the Beckhoff .NET ADS library (TwinCAT.Ads). The `TcAdsClient` is used to establish communication with the TwinCAT runtime and provides the method to read and write the variables namely ADS Read and ADS Write. The

TcAdsClient also internally handles the symbol resolution, index group, index offset addressing and message routing.

The ADS router forwards the request to the PLC runtime which is identified by its predefined port 851. The PLC executes the control algorithm in its cyclic task using the most recent values available in the GVL. After the controller logic has been executed, the computed control signal and adaptive variables are stored in the PLC memory.

Because ADS Read and ADS Write operations are implemented as a synchronous call in this setup, the simulation advances to the next step after all ADS transactions have completed, and the PLC response has been received. As a result, the effective controller update rate is directly tied to the Simulink fixed-step size, and the overall simulation speed is influenced by the ADS communication latency and PLC execution time.

9.2.2 ADS communication latency characterization

In the proposed CHIL configuration, data exchange between MATLAB/Simulink and the PLC is done by implementing synchronous ADS Read and ADS Write operations. This results in non-negligible communication delays. To characterize the ADS communication latency, a timing test was performed in MATLAB in which repeated single-variable ADS Read and ADS Write operations were executed using the TcAdsClient interface. The total time was measured in the MATLAB script by using the tic/toc function. The results indicate an average execution time of approximately 3.57 ms per ADS Read and 2.03 ms per ADS Write for symbolic variable access. In the implemented CHIL loop, each step involves multiple ADS transactions, seven write operations for the measurements that the Simulink model sends, and two read operations for the signals that the PLC sends. This brings the cumulative ADS communication overhead per simulation step approximately 21 ms, excluding the PLC cycle time and MATLAB overhead. So, in addition to the measured ADS communication, the PLC controller is executed within a cyclic task with a configured cycle time of 10 ms. Depending on the temporal alignment between the ADS communication and the PLC task execution, additional waiting time may occur before the updated variables are available. This means that the data latency for one closed-loop update can reach around 30 or more milliseconds.

These calculations were done for the purpose of achieving a real-time simulation, which is often defined as a simulation where the time taken to compute each step does not exceed the actual duration of the same time step in real life. In other words, to achieve real-time simulation, the simulation clock must run at the same speed as a wall clock. This requirement can be written as:

$$T_{wall\ per\ step} \leq h$$

Where $T_{wall\ per\ step}$ is the actual wall-clock time and h is the fixed simulation step size.

To achieve the real-time simulation, in this case, a fixed step size of at least $h=0.04s$ must be allocated.

8. Implementation of control algorithms on Beckhoff PLC

For the purposes of this master's thesis, as part of the CHIL structure, Beckhoff's PLC was implemented in the control loop. More specifically, Beckhoff's PLC model CX5320 was used. The setup is shown below; the PLC runs the Twin CAT 3 runtime environment, while the Simulink model of the BLDC motor is run on a personal computer (PC) running Windows 11. The PC hosts both the simulation environment and the ADS client used for communication with the PLC.

Communication between the PLC and the simulation computer is established through a standard Ethernet connection. Through this connection, the variables are sent back and forth between the PLC and the PC.

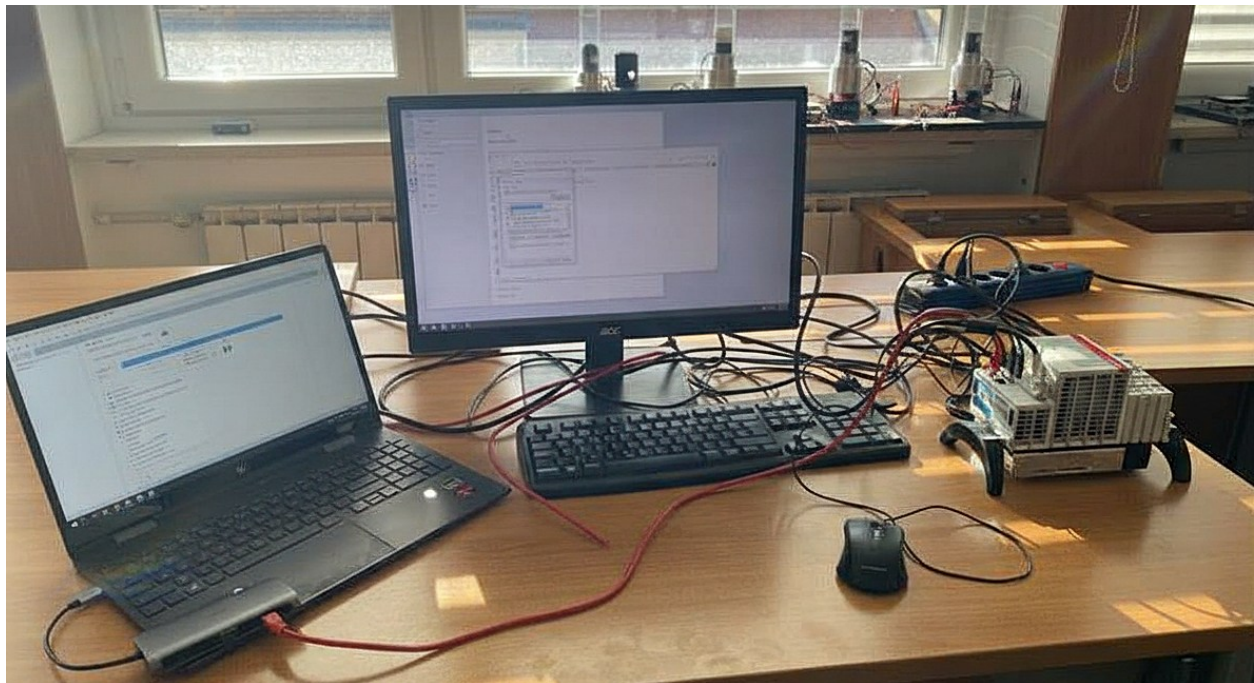


Figure 7.1 Lab implementation

The regulator logic derived in Chapters 5-8, is implemented in Twin CAT's Program Organization Units (POUs) which are then called in the MAIN program. The programming is done in structured text, and IEC 61131 is used as the programming standard. Ensuring compliance with the standard, the datatypes are defined as BOOL, LREAL, INTEGER, and their value range is clearly defined. The variable types such as global, local, and inputs are assigned variable classes such as VAR, VAR_GLOBAL, VAR_INPUT, etc.

For this master's thesis, a Global Variable List, containing the variables that are exchanged between the simulation environment and the PLC controller, was defined in the Twin CAT project. The GVL serves as a shared interface between MATLAB/Simulink and the PLC runtime.

The variables are declared using the VAR_GLOBAL construct, ensuring that they are accessible throughout the PLC project. This approach allows MATLAB to access the PLC variables symbolically using their names such as GVL_HIL.x1 which are hardcoded in the MATLAB script handling the ADS communication. The variables are all defined as LREAL data type which corresponds to double-precision floating point representation and matches the MATLAB double type.

Each regulator control logic is implemented as an individual Program Organization Unit (POU). Specifically, separate POUs were created for the adaptive backstepping regulator with adaptation of the unknown parameter δ , the adaptive backstepping regulator with simultaneous adaptation of δ and k_1 , the adaptive sliding mode controller, the hybrid sliding mode-backstepping approach and lastly the nonlinear compensation with the PD controller.

All regulator POUs are called as functions in the MAIN program, where they are selected using conditions. A Boolean variable is created and associated with each regulator and is set to TRUE when the corresponding regulator is to be selected, while all the other Boolean variables are set to FALSE. The Boolean variables are constructed in an IF-ELSE structure allowing only one regulator at a time to be called.

9. Results and analysis

This chapter presents the experimental results obtained using the CHIL structure. The objective is to evaluate and compare the performance of the implemented control algorithms under identical conditions to assess the practical implications.

As a first step, the communication and controller implementation was verified by comparing a backstepping regulator implemented directly in Simulink, making the first test done a software-in-the-loop verification. After the SIL test was done, the same control logic of the adaptive backstepping regulator was implemented into the Twin CAT environment to test if the same results were computed.

The obtained results were identical, confirming the ADS communication and PLC implementation do not alter the controller behavior.

Three categories of experiments are considered.

The first were non-real-time simulations with a high controller update rate, used to evaluate the nominal performance of each regulator.

In the second category, simulations with a reduced update rate, approaching real-time execution, were performed. These tests were used to assess the robustness of the regulators under practical timing and communication constraints.

The third simulation results introduced a disturbance in the x_1 state to evaluate the robustness of the implemented regulators. This test allowed to determine how well the controller restores tracking after an abrupt disturbance.

For each regulator, the results were presented in the form of time responses obtained from the Simulink scopes. The first plot shows the system position state x_1 together with the desired reference x_{d1} , while the second plot shows the corresponding velocity state x_2 .

The regulators for each category of results were tested under the same conditions, using the same reference signals and model parameters to make the comparison and draw conclusions.

The performance of the regulators was evaluated qualitatively based on the following criteria:

- accuracy of reference tracking,
- presence of steady-state error or phase lag,
- transient behavior and settling characteristics,
- boundedness of internal state responses.

An ideal controller is characterized by accurate tracking of the desired trajectory, stable and bounded internal states, fast convergence without excessive overshoot, and absence of oscillatory behavior. These criteria are used consistently throughout this chapter to compare the performance of different control strategies.

9.1 High update-rate simulation results

The first category of results presented in this chapter corresponds to simulations performed using a fixed step fourth order Runge-Kutta solver (ODE4) with a step size of 0.001s. A fixed step of 0.001s means that the controller is updated every millisecond, or an update rate of 1kHz. This configuration represents a high controller update rate. At this update rate, discretization effects and communication delays have a negligible influence on the control-loop behavior. It closely reflects the ideal behavior predicted by the controller designs, and the simulations are used for comparing the tracing capabilities and transient responses of the different regulators independent of practical timing constraints.

Presented bellow are the graphs taken from the Scope in Simulink, illustrating the time response of each regulator as well as the evolution of the unknown parameter and its estimate as computed by each regulator.

Adaptive backstepping regulator with adaptation of the δ parameter

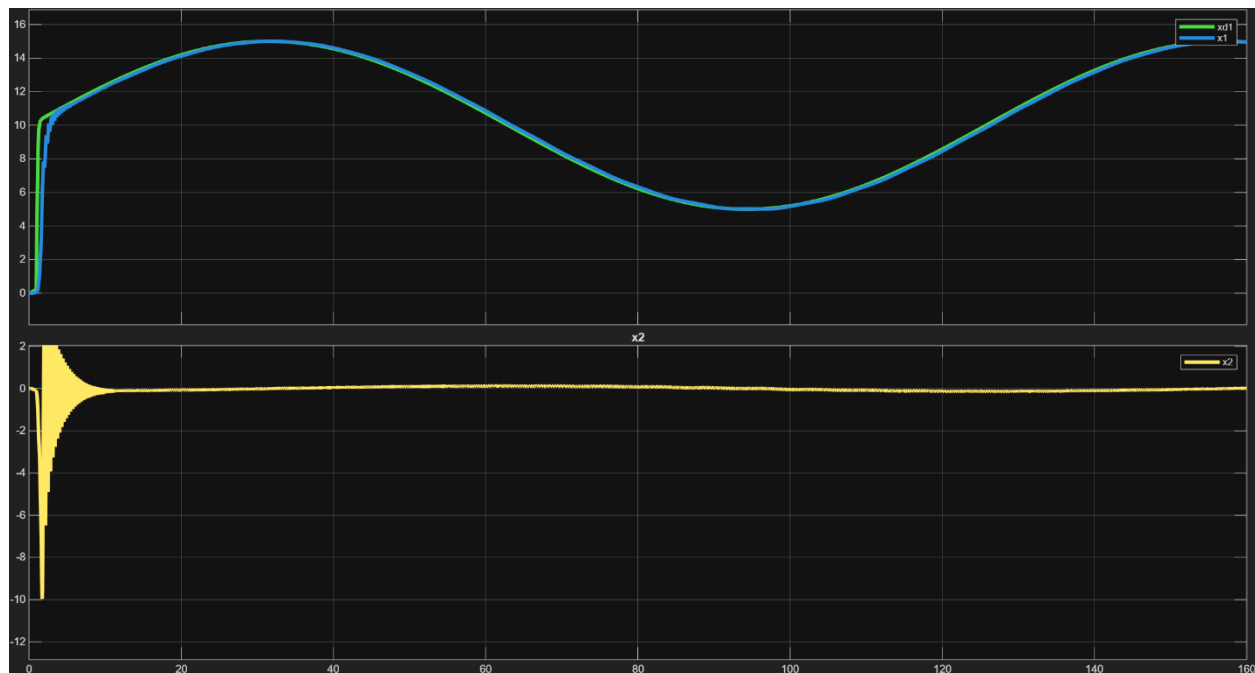


Figure 8.1 Time response of the adaptive backstepping regulator. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

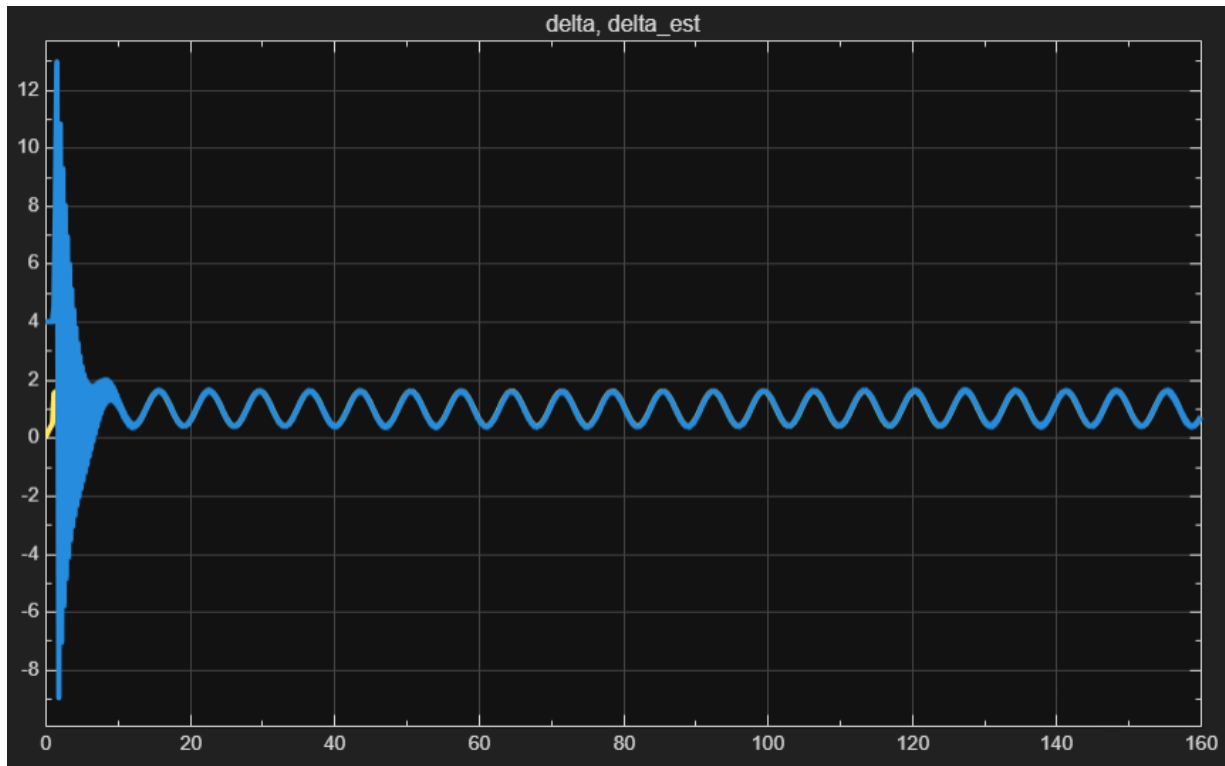


Figure 9.2 Evolution of the unknown parameter and its estimate under adaptive backstepping control

Adaptive backstepping regulator with adaptation of the δ and k_1 parameters

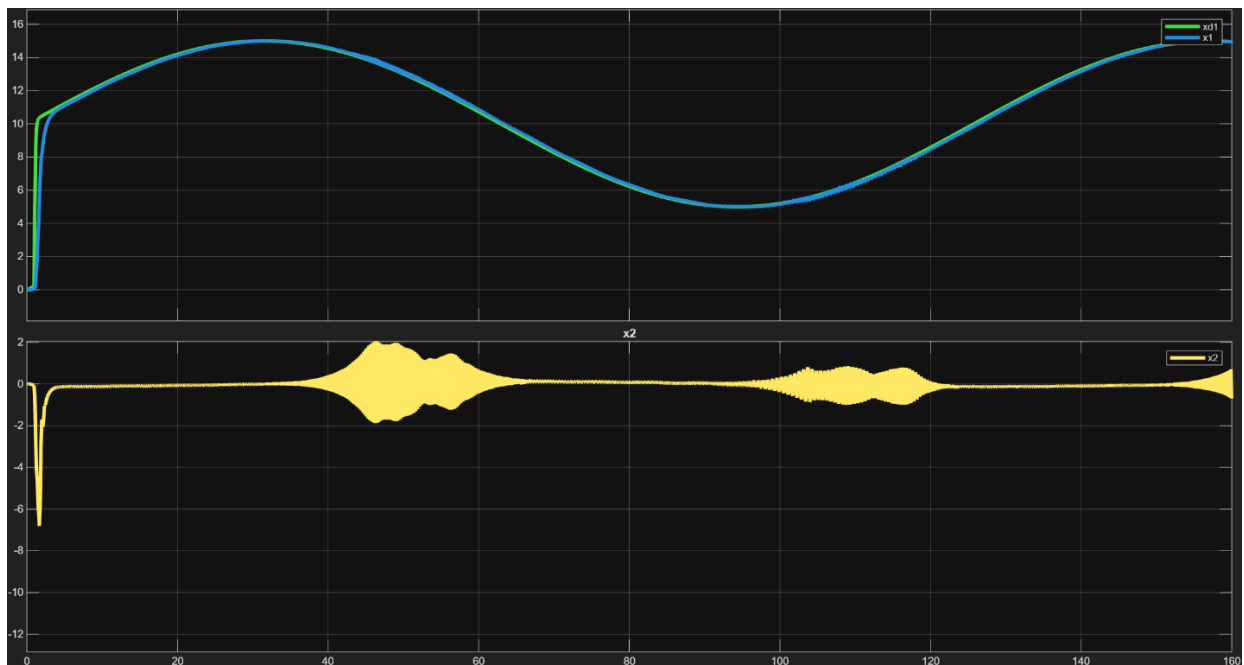


Figure 9.3 Time response of the adaptive backstepping regulator with an added adaptation of k_1 . Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

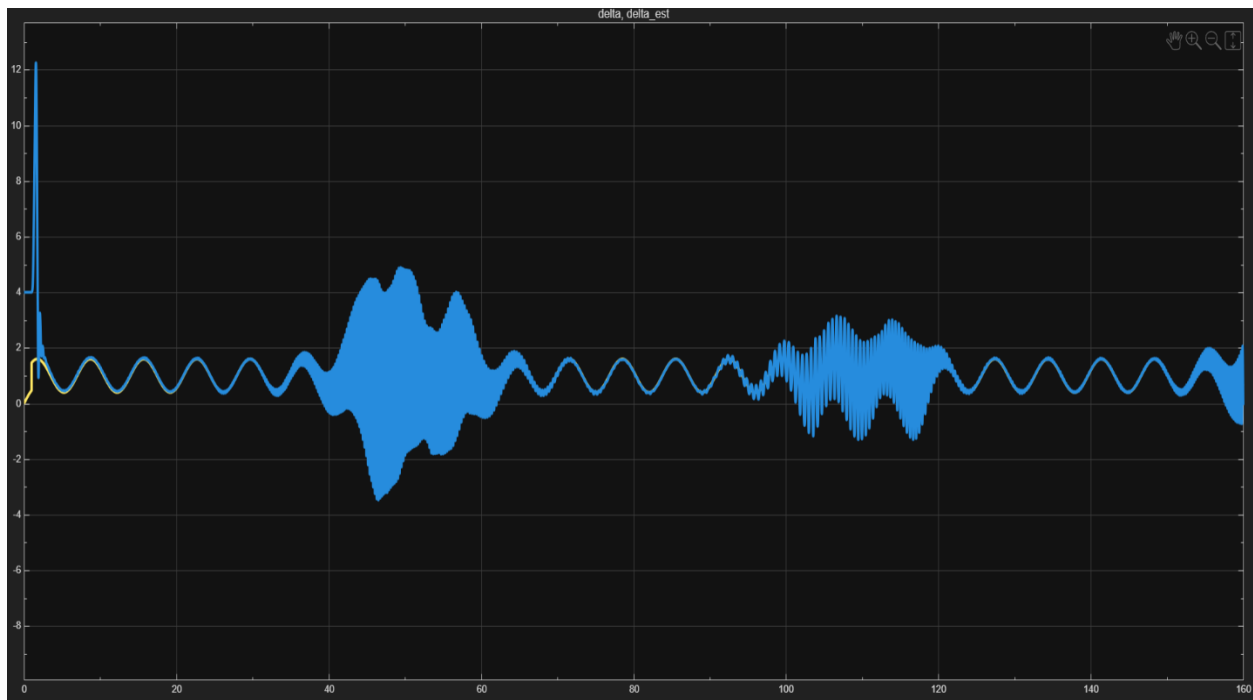


Figure 9.4 Evolution of the unknown parameter and its estimate under adaptive backstepping control with an added adaptation of k_1 parameter

Adaptive sliding regulator with adaptation of the δ parameter

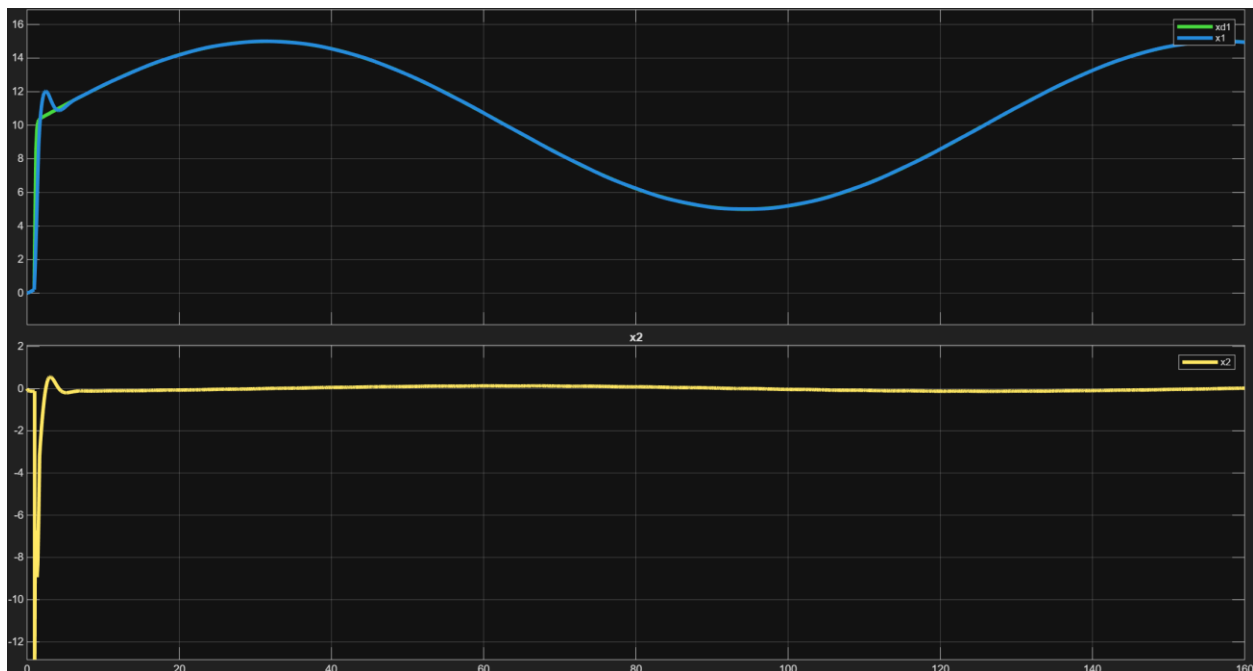


Figure 9.5 Time response of the adaptive SMC regulator with an added adaptation of k_1 . Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

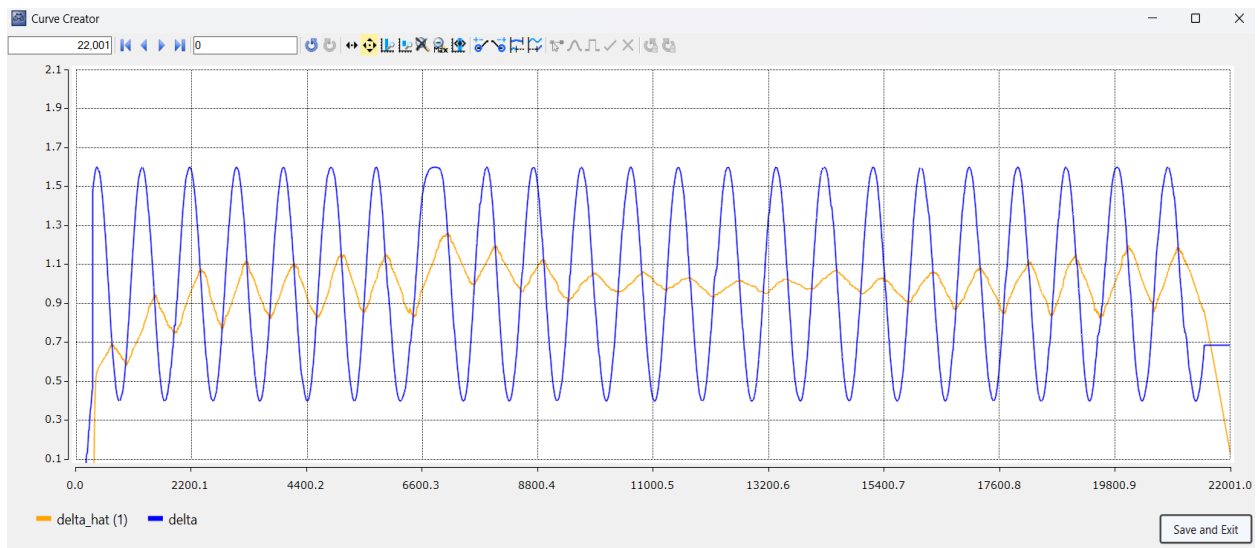


Figure 9.6 Evolution of the unknown parameter and its estimate under adaptive SMC control

Hybrid backstepping and SMC regulator with adaptation of the δ parameter

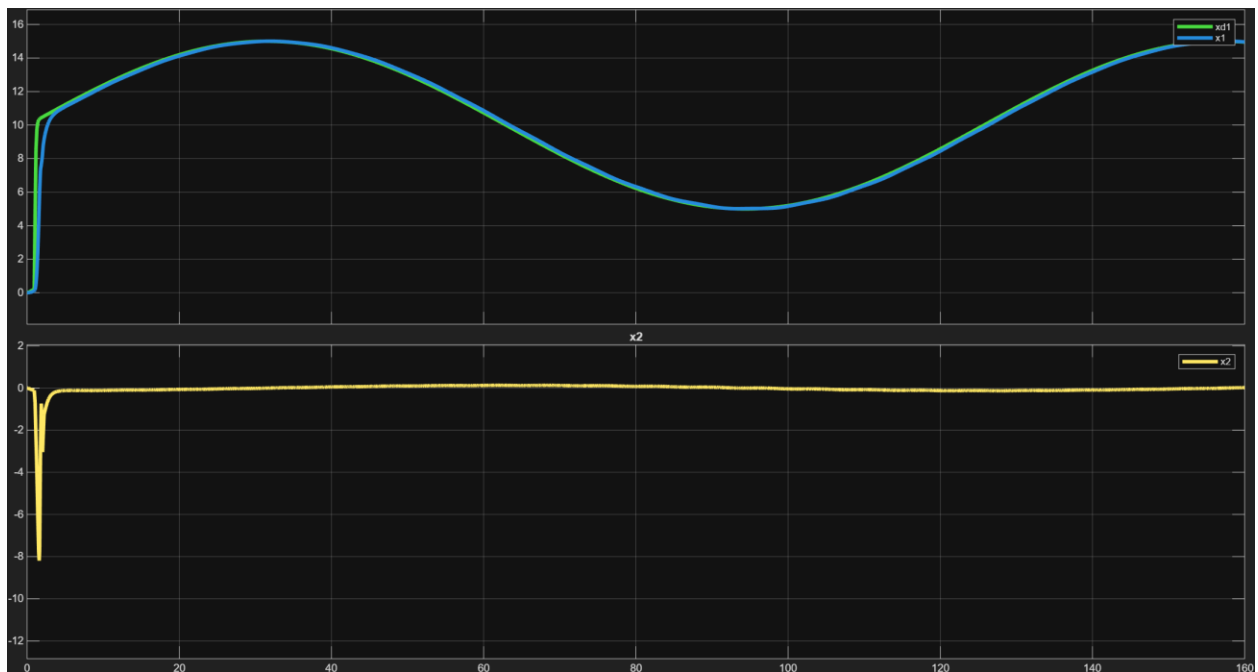


Figure 9.7 Time response of the adaptive backstepping regulator with an added sliding surface. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

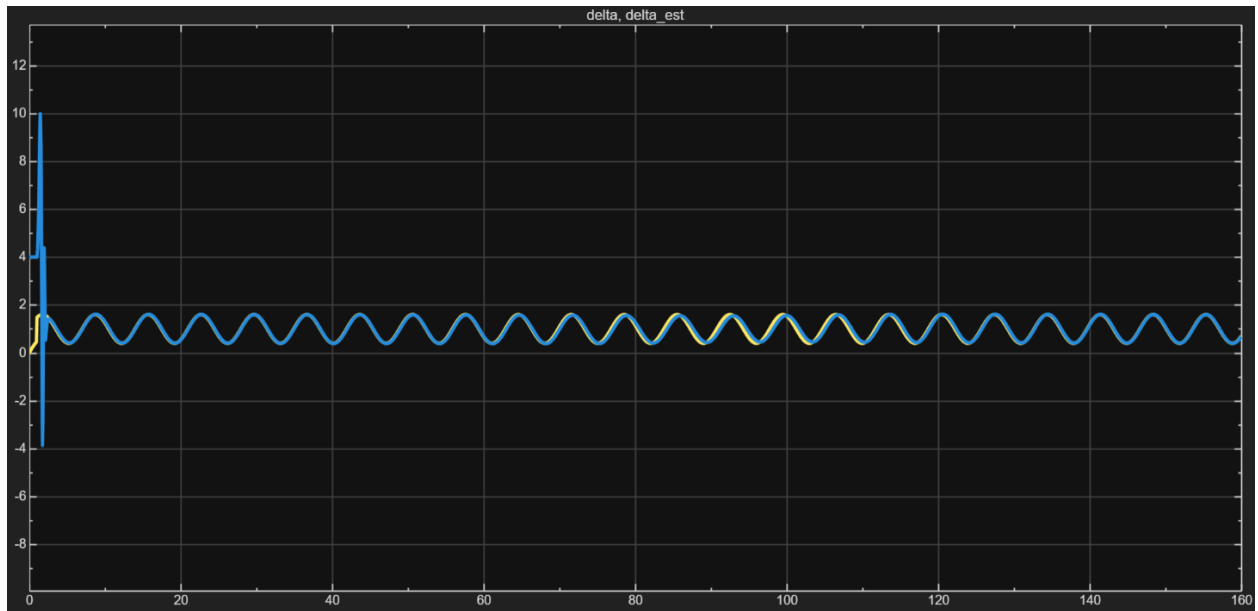


Figure 9.8 Evolution of the unknown parameter and its estimate under adaptive backstepping control with a sliding surface

Nonlinear compensation with PD control

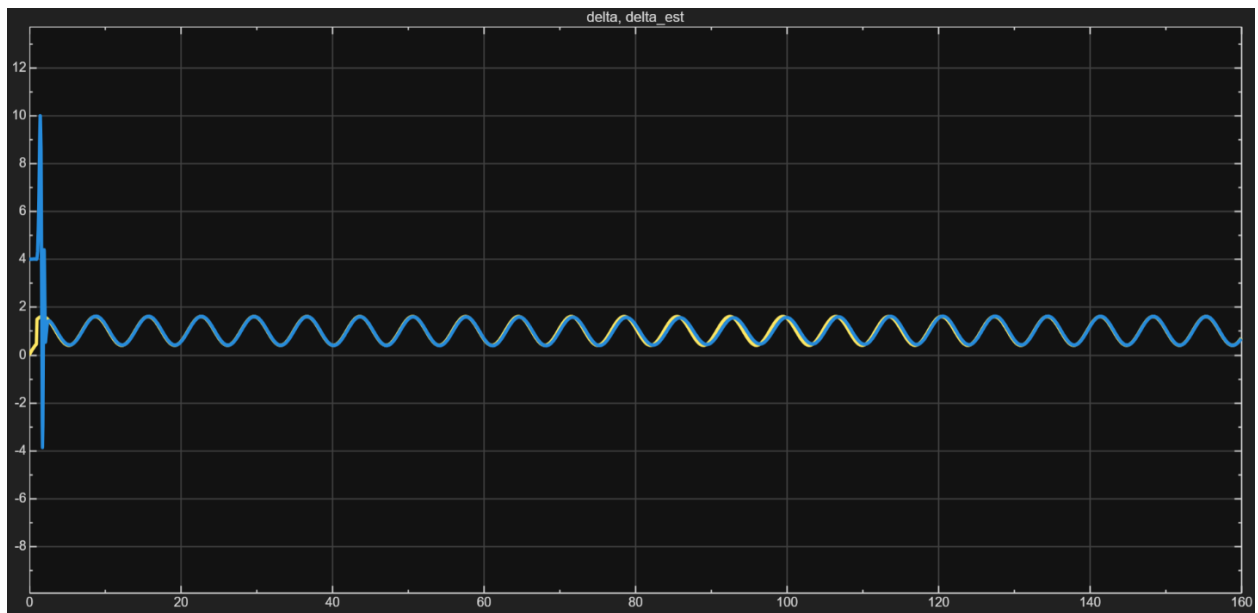


Figure 9.9 Time response of nonlinear compensation with PD regulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

9.2 Low update-rate simulation results

The second category of results was done with a reduced update rate of the controller. For the simulation, in Simulink's model configuration, the fixed step was reduced to a size of 0.06 approaching real-time simulation. Compared to the high update rate experiments, this represents a significantly slower closed-loop update, meaning that the PLC controller receives new measurements and sends back new control command only every 60 ms. This makes the closed loop more sensitive to communication latency and timing misalignment between the ADS transactions and the PLC cycle.

Adaptive Backstepping Regulator with adaptation of the δ parameter

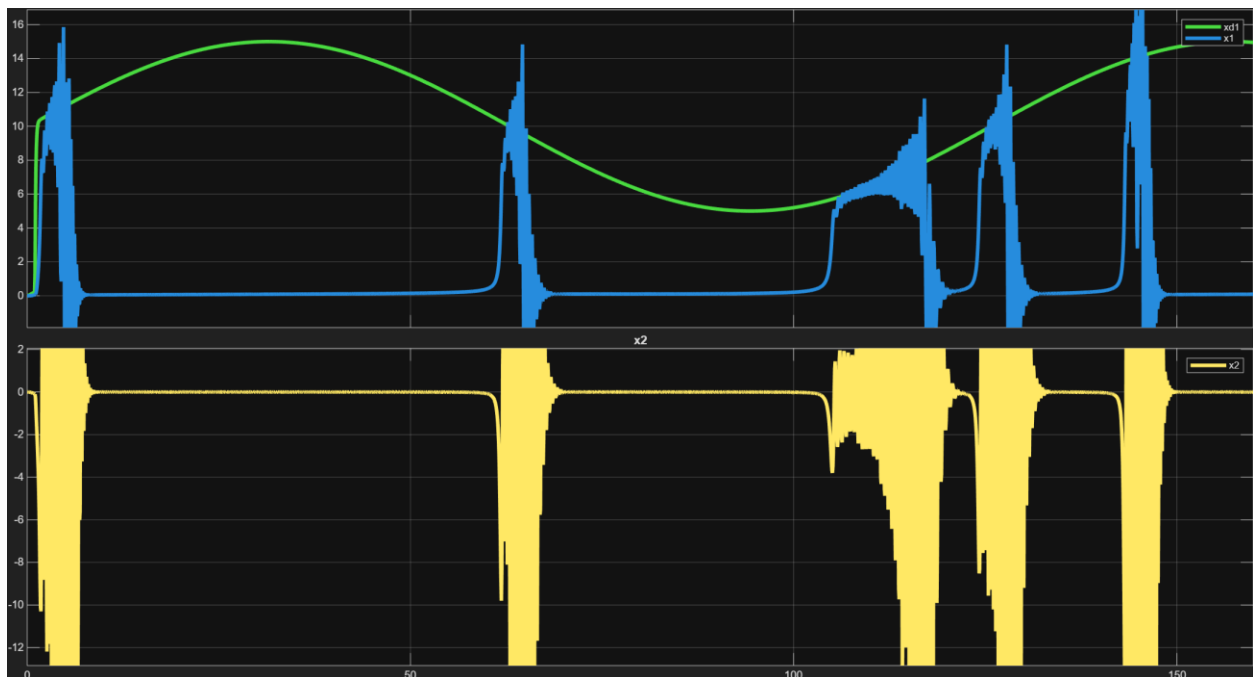


Figure 9.10 Time response of the adaptive backstepping regulator during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

Adaptive Backstepping Regulator with adaptation of the δ and k_1 parameters

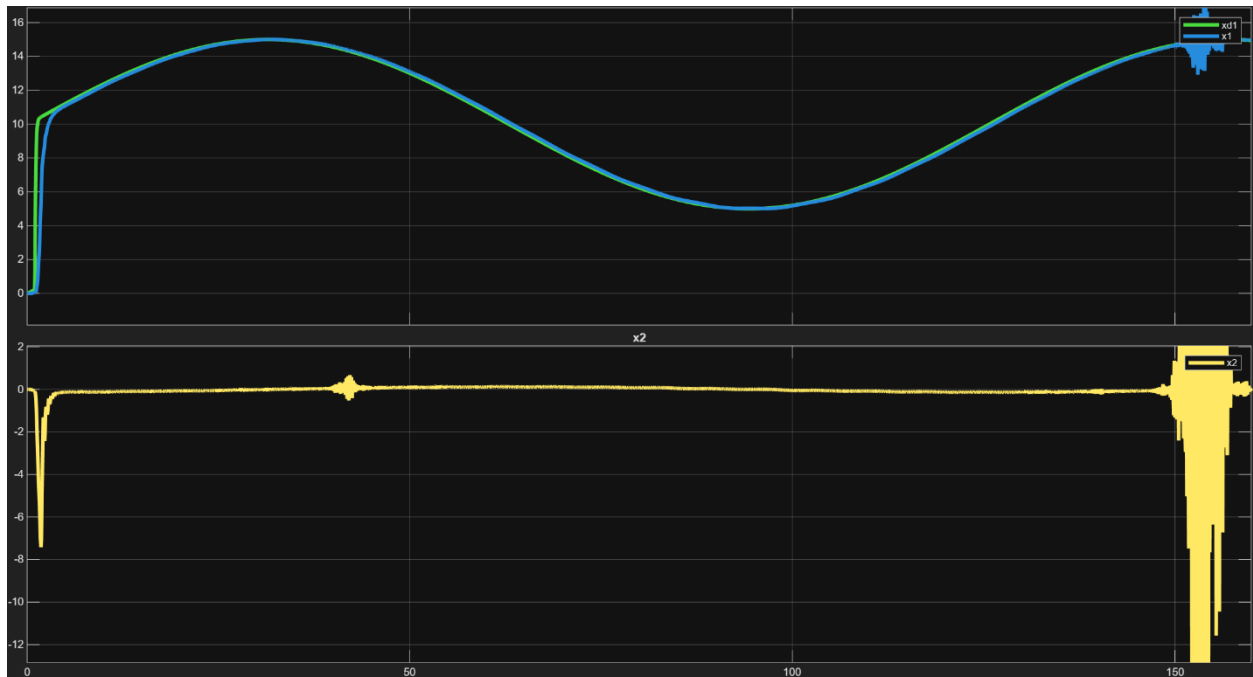


Figure 9.9 Time response of the adaptive backstepping regulator with an added adaptation of k_1 during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

Adaptive sliding regulator with adaptation of the δ parameter

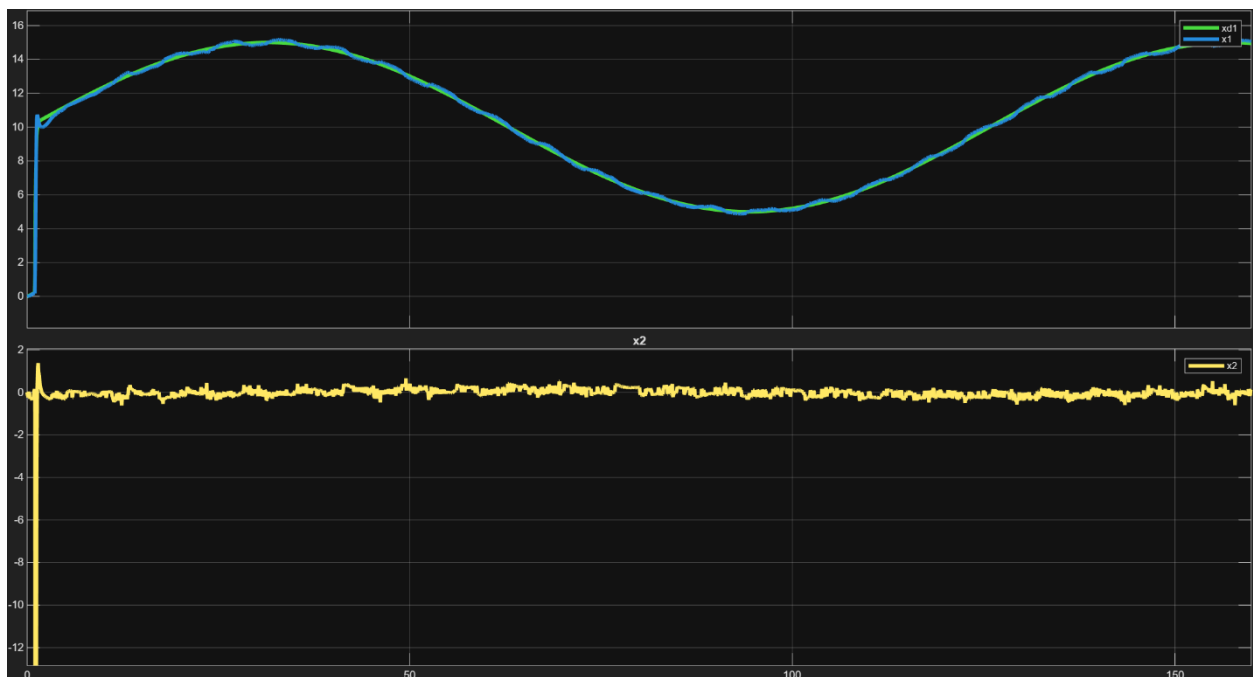


Figure 9.10 Time response of the adaptive SMC regulator during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

Hybrid backstepping and SMC Regulator with adaptation of the parameter

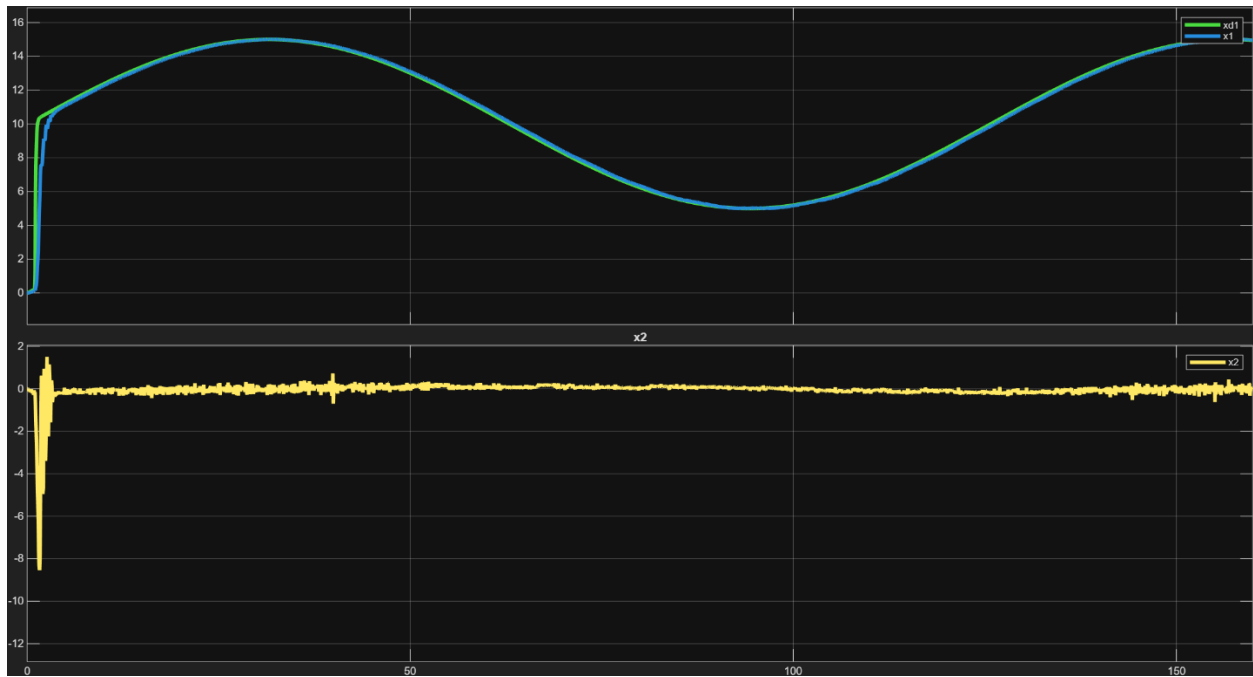


Figure 9.11 Time response of the adaptive backstepping regulator with an added sliding surface during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

Nonlinear compensation with PD control

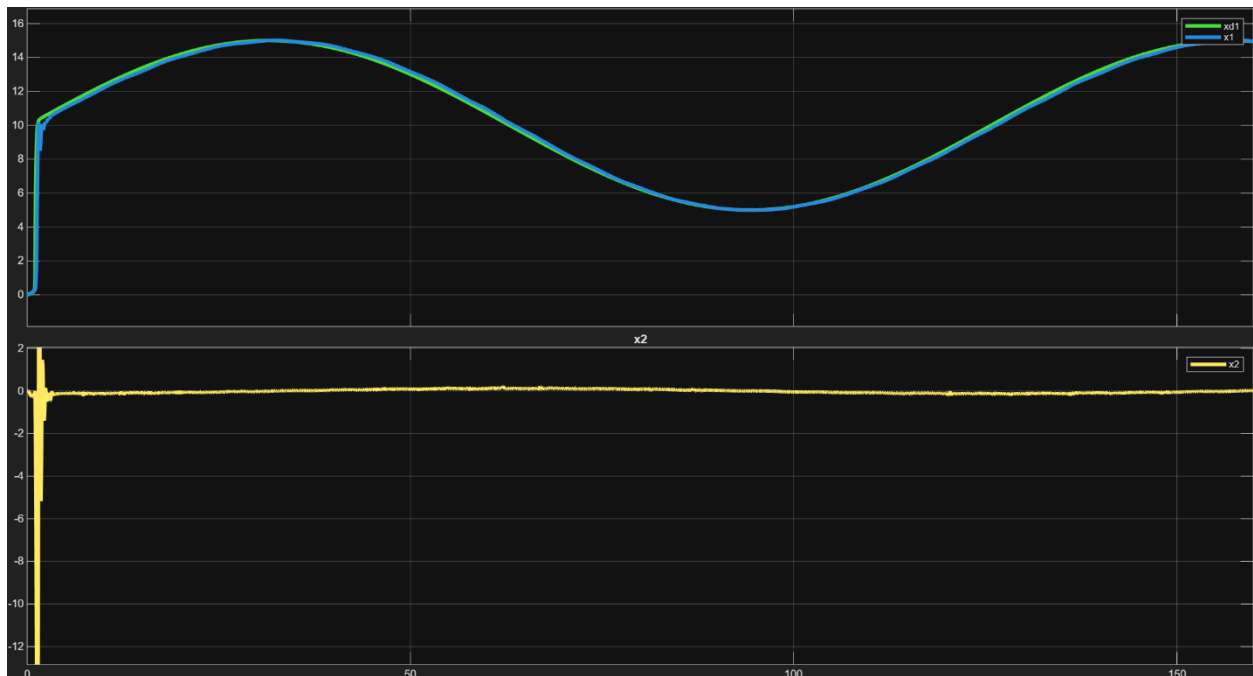


Figure 9.12 Time response of nonlinear compensation with PD control during low update rate simulation. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

9.3 Low update rate simulation with an added disturbance

The third category of results presented in this master's thesis were done with the aim of testing the regulator's behavior and capability of restoring accurate tracking after introducing a disturbance in state x_1 . In real-world applications, these types of disturbances may represent sensor noises, abrupt load changes or parameter inaccuracies.

In Simulink, the disturbance was implemented by using a Step function and adding the signal from the Step block to the output of the integrator. The step function introduced disturbance at 40 ms as specified in the block's settings.

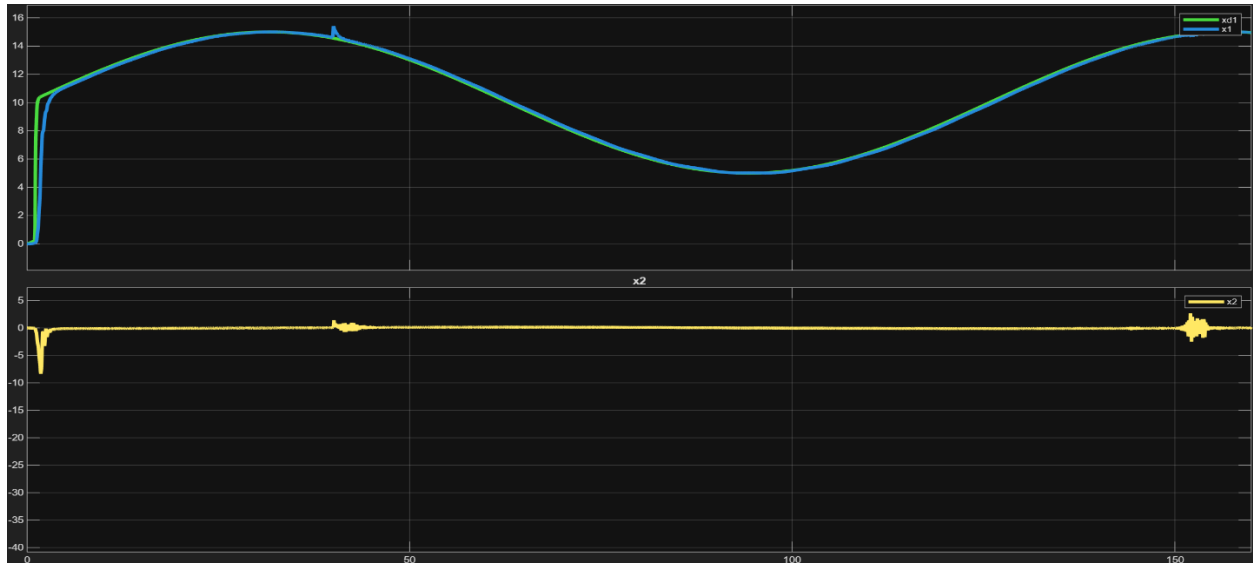


Figure 9.13 Time response of the adaptive backstepping regulator (k_1 added) during low update rate simulation with added disturbance. Top: reference x_{d1} and state x_1 . Bottom: velocity state x_2

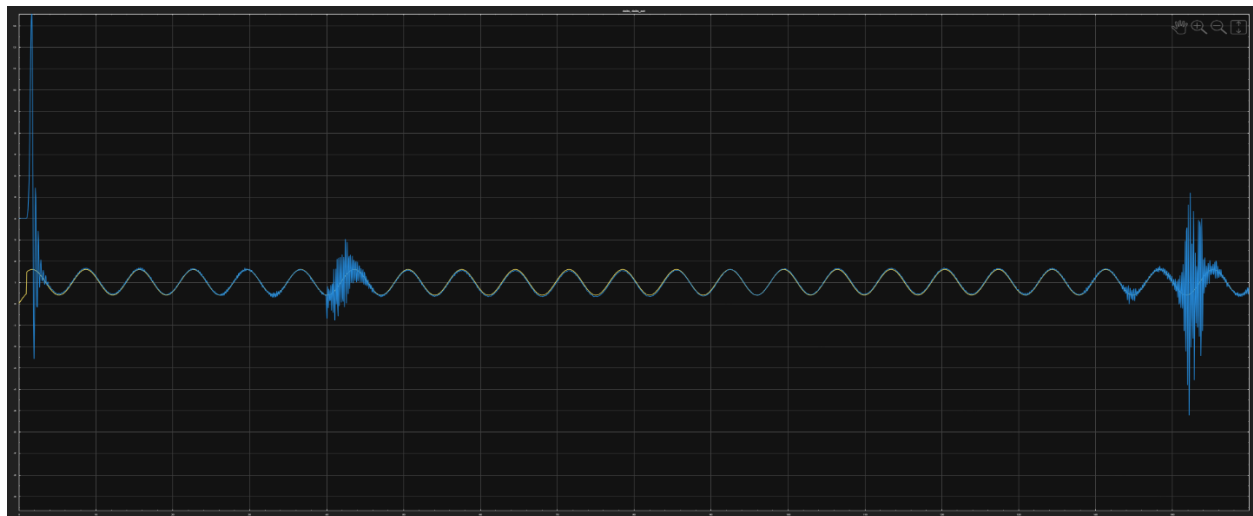


Figure 9.14 Evolution of the unknown parameter and its estimate under adaptive backstepping control with additional adaptation of k_1 with added disturbance

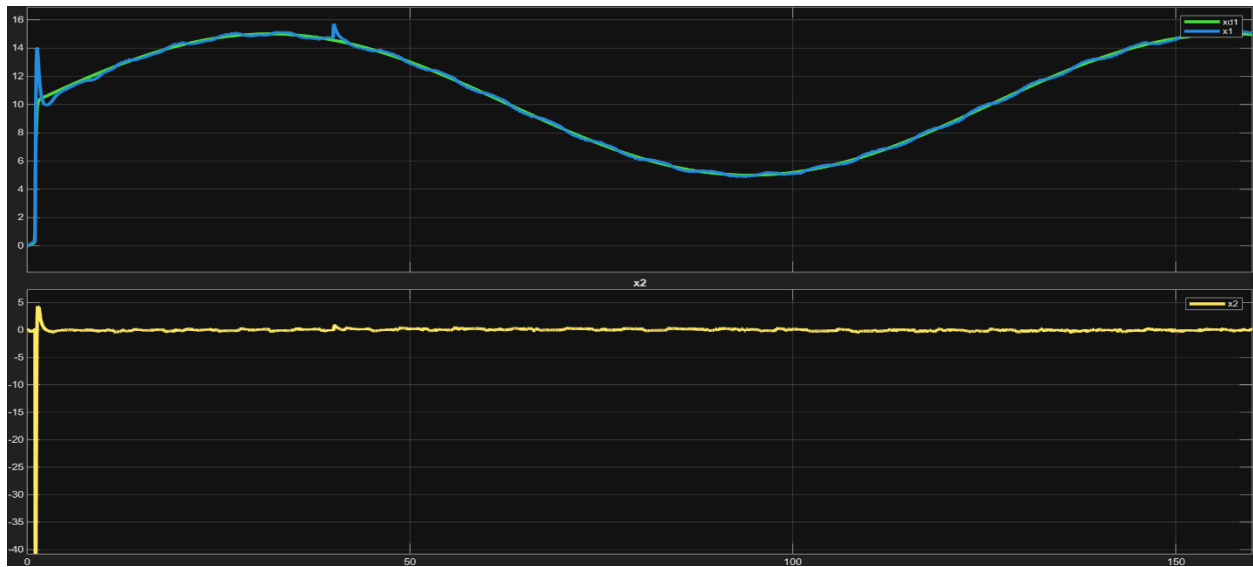


Figure 9.15 Time response of the adaptive SMC regulator during low update rate simulation with added disturbance. Top: reference $x1$ and state $x1$. Bottom: velocity state $x2$

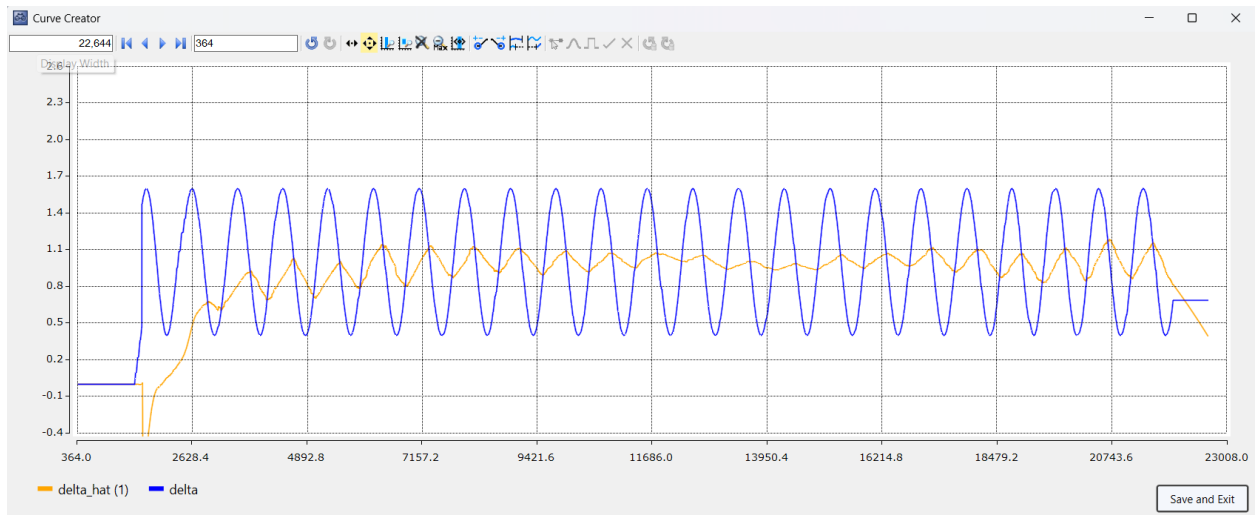


Figure 9.16 Evolution of the unknown parameter and its estimate under adaptive SMC control with added disturbance

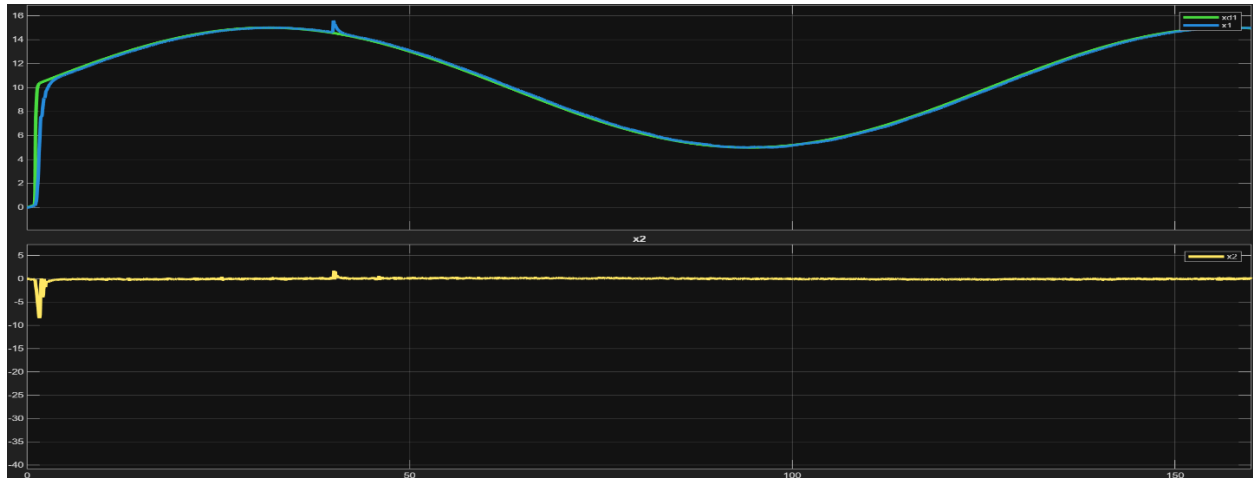


Figure 9.17 Time response of the adaptive hybrid backstepping regulator with a sliding surface during low update rate simulation with added disturbance. Top: reference $x1$ and state $x1$. Bottom: velocity state $x2$

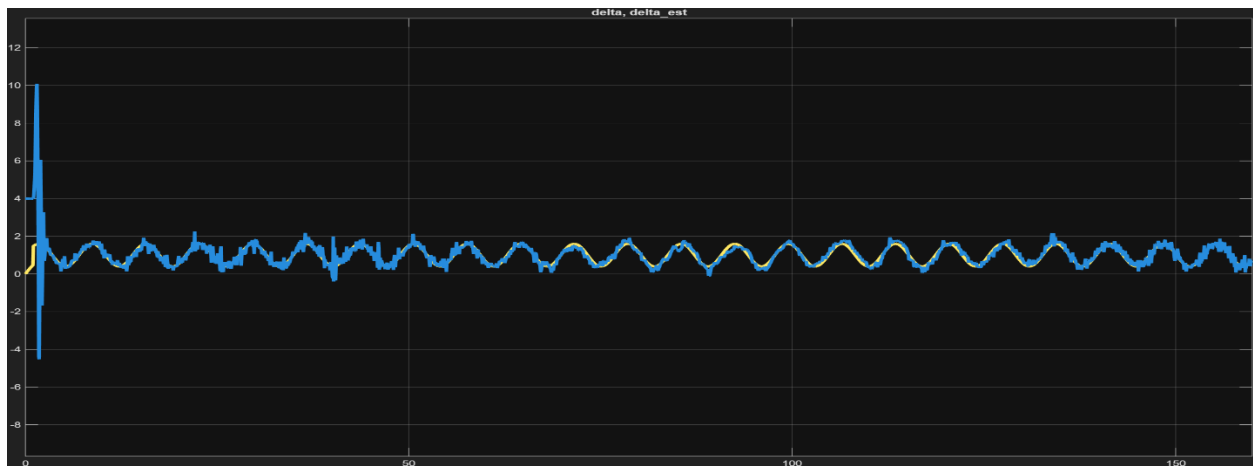


Figure 9.18 Evolution of the unknown parameter and its estimate under adaptive hybrid backstepping regulator with a sliding surface with added disturbance

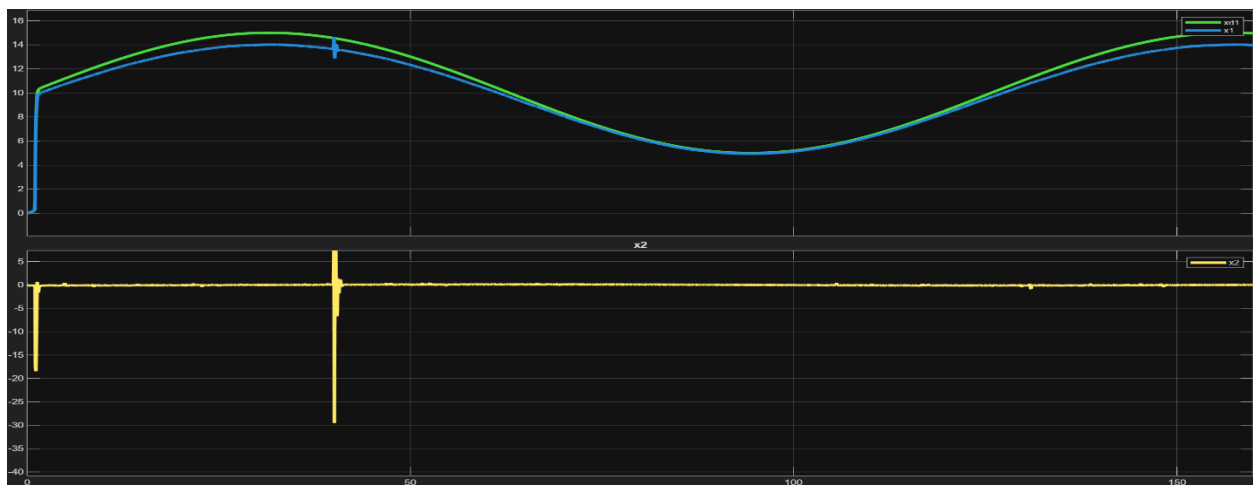


Figure 9.19 Time response of the nonlinear compensation with PD regulation during low update rate simulation with added disturbance. Top: reference $x1$ and state $x1$. Bottom: velocity state $x2$

9.4 Analysis

Chapter 9.4 focuses on the analysis of the tracking ability and performance of the regulators. What can be noted in Scope's graphs is that all the different control laws that were defined can follow the dynamic reference trajectory, albeit in some cases oscillations and small offsets can be seen.

Under ideal high-update rate conditions, all nonlinear controllers performed well. However, under reduced update-rate conditions, clear differences emerged highlighting the limitations of some of the implemented regulators. Mainly, the regulators relying on exact nonlinear cancellation were more sensitive to sampling delays, while the more robust ones maintained stable performance.

The results show that the adaptive backstepping regulator achieved excellent tracking of the dynamic reference at high update rates, but severely degraded when the update rate was low. This stems from the fact that the backstepping design relies on nonlinear cancellation using current state values. With slow updates, the controller acted with outdated information, and the correction was too late and too aggressive.

In contrast, the sliding regulator maintained stable tracking during both tests, which was fairly expected. The SMC design relies on enforcing convergence towards an ideal surface and does not depend as much on the modeling of the system nor the cancellation of the nonlinearities, making it much more robust than the backstepping regulator.

The hybrid backstepping-sliding approach shows the best performance overall in terms of accurate position tracking, stable control output while remaining robust to the real-time implementation effects. The hybrid approach, while mainly based on the backstepping methodology, also introduces a sliding term from the SMC methodology, and this is the combination that outperforms the other regulators. The backstepping component already cancels the nonlinearities of the model, in high update rates. However, under low update rates and communications delay, the model-based cancellation is affected by the outdated measurements. The added sliding mode term is what drives the trajectory, even if stirred in the wrong direction, towards the defined surface. As a result, the hybrid control acts as a backstepping regulator, when the model prediction is accurate, but more as a sliding regulator when the timing and mismatch effects affect the controller's design.

Finally, the PD controller with nonlinear compensation demonstrates that when timing is correctly configured, it can achieve semi-accurate tracking compared to sliding based designs. However, for the nonlinear compensation part, an accurate model of the controlled system is an important factor that decides how well the regulator performs.

To determine which of the regulators performed the best and obtain a fair comparison of all, a quantitative approach was used. The analysis was conducted on the low update rate of the controller with and without disturbance. The idea was that each regulator has a numerical representation which can then be compared and would be able to give insight into performance abilities, not always visible on the graphs. The reason for the choice of the low controller update rate simulations was that in these simulations sampling and delay effects were present, representative of real-time or networked systems.

For the nominal performance evaluation or the simulations where no disturbance was present, the performance metrics were computed over time interval [5s,160s], purposefully excluding startup transients.

The following metrics were computed in MATLAB script:

Root mean square error (RMSE) for quantifying tracking accuracy, integral of absolute error (IAE) for quantifying tracking deviation, 95th percentile of $|e|$ for providing a measure of worst-case error, as well as peak and RMS of x_2 to indicate smoothness and dynamics.

Table 9.1 Nominal Metrics

Regulator	RMSE	IAE(e)	P95 $ e $	Peak (x_2)	RMS (x_2)
Adaptive backstepping with adaptation of k_1	0.235694	15.931164	0.136475	57.742500	2.952877
Adaptive SMC	0.113097	15.828944	0.177228	0.645556	0.174112
Adaptive hybrid	0.085754	11.832246	0.129122	0.593322	0.147844
Nonlinear compensation with PD regulation	0.624023	79.210818	1.012983	4.372520	0.192041

Based on the results, the following can be concluded:

Overall, the adaptive hybrid regulator has the best results, the best average tracking accuracy i.e. lowest RMSE, IAE and P95 $|e|$. This means that the hybrid approach is not only the best at tracking the dynamic reference of all the regulators but also does so consistently throughout.

The adaptive sliding regulator has a very good tracking accuracy but P95 $|e|$ shows oscillations which can also be seen on the graph and are a drawback at in performance.

The adaptive backstepping regulator with an additional k_1 adaptation has a higher RMSE than both the Hybrid and the SMC, while the IAE is similar to the adaptive SMC. This indicates that the

additional gain adaptation is much more aggressive and requires a larger control output compared to the other regulators.

Adaptive PD has larger values for the RMSE and the IAE meaning that the x_1 deviates from the reference trajectory and does not track well. Additionally, it shows the limitation that the nonlinear compensation does not compensate sufficiently and therefore produces tracking error.

The $\text{peak}(x_2)$ is a measure of the magnitude of control effort that is being exerted. While the hybrid and sliding regulator show smooth tracking with minimal actuator stress and less aggressive control effort, the backstepping regulator as well as the PD regulation show the opposite. With this, a trade off is presented where tracking performance is evaluated not just by tracking error, but also with smooth or aggressive control effort.

The following analysis was done on the simulations where disturbance was introduced and was done in 2 integrals, before and after the disturbance. The following measurements were done: RMSE, IAE, and the 95th percentile of the tracking error to quantify average accuracy, deviation and worst-case behavior, together with RMS and the peak values of x_2 , same as before in the time interval [5s-39s]. After the disturbance, peak error, disturbance IAE and recovery time were measured to give indications of robustness and disturbance rejection performance.

Table 9.2 Nominal Metrix pre-disturbance [5s-39s]

Regulator	RMSE	IAE(e)	P95 e	Peak (x_2)	RMS (x_2)
Adaptive backstepping with adaptation of k1	0.073551	2.170429	0.125563	0.172507	0.076460
Adaptive SMC	0.116729	3.576589	0.190453	0.449924	0.161973
Adaptive hybrid	0.074026	2.186226	0.127641	0.596858	0.146563
Nonlinear compensation with PD regulation	0.872891	29.908977	1.024117	0.891155	0.097660

Table 9.2 shows the nominal performance of the regulators before the disturbance was introduced in the simulation. If compared to Table 9.1, differences across some measurements can be noted. The reason behind this is that the nominal performance was measured in a much smaller time interval. For example, what is also evident from Table 9.2 is that the adaptive backstepping regulator shows better performance in the first quarter of the simulation and has better tracking performance which then slowly degrades towards the end as a result of the aggressive tuning of the k1 parameter.

Table 9.3 Post disturbance metrics

Regulator	RMSE	IAE(e)	Recovery time
Adaptive backstepping with adaptation of k1	0.753445	1.326153	0.740 s
Adaptive SMC	0.902960	1.446323	0.920 s
Adaptive hybrid	0.749924	1.063789	0.560 s
Nonlinear compensation with PD regulation	1.888834	8.258916	24.860 s

Table 9.3 illustrates that; the adaptive hybrid regulator demonstrates overall best performance in handling unexpected disturbances. The most significant measurement is the recovery time, which is smaller than all the other regulators. However, compared to the nominal performance of the regulators, where sliding showed slightly better performance compared to the Backstepping regulator in tracking error, in post disturbance analysis, adaptive backstepping shows good rejection and faster handling of the disturbance compared to the SMC. The PD regulation exhibits significant performance degradation under low-update conditions and longer recovery time, highlighting the limitations of the PD regulation.

10. Conclusion

This master's thesis focused on implementing nonlinear control on brushless DC motor position tracking and evaluated the tracking performance in a controller HIL structure. Several control designs were tested and compared under three implementation conditions, a high-update rate setting, and a reduced update-rate real-time setting that introduced sampled data and communication-delay effects.

From an industrial perspective, these results reflect common control practice for BLDC drives. In real systems, fast current and speed loops are operating at millisecond rates, while outer loops, such as the position control, operate at slower rates and over communication networks. The measured CHIL communication latency and reduced update rate experiments represent a conservative stress test for outer-loop control.

A key aspect of this master thesis was also the inclusion of an unknown system parameter, which was included to represent modeling uncertainty. The idea is that the designed controller could still apply accurate control even with a parameter that varied during runtime. The use of CHIL structure was essential for this experiment, as it was used to test each regulator under realistic execution effects such as communication delays and discrete update rates. This structure provides practical validation because it allows the control algorithms to be verified under realistic conditions before deployment on physical hardware. With industry trends moving towards network-connected motion control systems, controllers are increasingly exposed to sampling and communication delays, as well as model uncertainty. The findings in this thesis show the bridging of control theory and practical implementation in a Controller-in-the-Loop framework. It shows that this framework can effectively lead to conclusive results of controller operations which are later implemented in industrial deployment. More specifically, the findings of this thesis pinpoint towards the adaptive hybrid regulator based on backstepping methodology coupled with a sliding term for robustness, showing the best performance overall. A further step and natural continuation of this master's thesis would be further research on development and optimization of the hybrid structure.

References:

- [1] Automation GmbH, B. *Manual TwinCAT 3 / ADS Basics*. Verl: Beckhoff Automation GmbH & Co. KG, 2016.
- [2] Bacic, M. On hardware-in-the-loop simulation. V: *Proceedings of the 44th IEEE Conference on Decision and Control*. 44th IEEE Conference on Decision and Control, Seville, 12.–15. december 2005. New York: IEEE, 2005, str. 3194–3198.
- [3] Bandyopadhyay, B., Janardhanan, S., Spurgeon, S. K. (ur.). *Advances in Sliding Mode Control*, vol. 440. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013.
- [4] Brayanov, N., Stoynova, A. Review of hardware-in-the-loop-a hundred years progress in the pseudo-real testing. V: *Proceedings of the X Anniversary International Scientific Conference*. Sofia: Technical University of Sofia, 2014.
- [5] Chapman, S. J. *Electric Machinery Fundamentals*, 4. edition. New York: McGraw-Hill, 2005.
- [6] Chen, K., Astolfi, A. Adaptive control for nonlinear systems with time-varying parameters and control coefficient. V: *IFAC-PapersOnLine*. Elsevier B.V., 2020, pg. 3829–3834..
- [7] Edwards, C., Spurgeon, S. *Sliding Mode Control*. London: CRC Press, 1998.
- [8] Evans, M. B., Schilling, L. J. *The Role of Simulation in the Development and Flight Test of the HiMAT Vehicle*. NASA Technical Memorandum 84912. Edwards, California: NASA Ames Research Center, 1984.
- [9] Galvan-Colmenares, S., Moreno-Armendáriz, M. A., Rubio, J. D. J., et al. Dual PD control regulation with nonlinear compensation for a ball and plate system. *Mathematical Problems in Engineering*, vol. 2014, (2014).
- [10] Hägglund, T., Astrom, K. J. *Advanced PID Control*. Research Triangle Park, NC: ISA - The Instrumentation, Systems, and Automation Society, 2006.
- [11] Kokotovic, P. V. The joy of feedback: nonlinear and adaptive. *IEEE Control Systems Magazine*, vol. 12, (1992), 3, pg. 7–17.
- [12] Koshkouei, A. J., Zinober, A. S. I. Adaptive Output Tracking Backstepping Sliding Mode Control of Nonlinear Systems. *IFAC Proceedings Volumes*, vol. 33, (2000), 14, pg. 167–172.
- [13] Krishnan, R. *Permanent Magnet Synchronous and Brushless DC Motor Drives*. Boca Raton: CRC Press, 2010.

- [14] Leb, S. Stator and Armature of the Electric Motor and Magnetic Field on the Rotor. *Journal of Electrical & Electronic Systems*, vol. 5, (2016), 3.
- [15] Legua, M. P., Morales, I., Sánchez Ruiz, L. M. The Heaviside Step Function and MATLAB. V: Computational Science and Its Applications – ICCSA 2008. Berlin: Springer, 2008, str. 1212–1221.
- [16] Mihalič, F., Truntič, M., Hren, A. Hardware-in-the-Loop Simulations: A Historical Overview of Engineering Challenges. *Electronics*, vol. 11, (2022), 15.
- [17] Nisar, K. S., Farman, M., Abdel-Aty, M., Cao, J. A review on epidemic models in sight of fractional calculus. *Alexandria Engineering Journal*, vol. 66, (2023), pg. 421–434.
- [18] Overall, S. Brushless DC Motors Market 2024–2028. London: Technavio, 2024.
- [19] Rios-Bolivar, M. Adaptive Backstepping and Sliding Mode Control of Uncertain Nonlinear Systems. Sheffield: University of Sheffield, 1997
- [20] Sayed-Ahmed, M. T., Sallam, A. A., El-Mogy, A. M., Abdel-Hamid, A. M. An optimal PID-computed-torque controller for three link robot using nonlinear least squares implementation. *International Journal of Neural Networks and Advanced Applications*, vol. 3, (2016), pg. 52–59
- [21] Scarratt, J. C., Zinober, A., Mills, R. E., et al. Dynamical Adaptive First and Second-Order Sliding Backstepping Control of Nonlinear Nontriangular Uncertain Systems. *Journal of Dynamic Systems, Measurement, and Control*, vol. 122, (2000), 4, pg. 746–752.
- [22] Shtessel, Y., Edwards, C., Fridman, L., Levant, A. Sliding Mode Control and Observation. New York: Springer, 2014.
- [23] Thi, N., Dung, N. An Overview on Electric Motors: Classification, Control and Applications. *International Journal of Research and Scientific Innovation*, vol. 10, (2023), 11.
- [24] Utkin, V. I. Sliding Modes in Control and Optimization. Berlin: Springer-Verlag, 1992.
- [25] Vo-Duy, T., Ta, M. C. A signal hardware-in-the-loop model for electric vehicles. *ROBOMECH Journal*, vol. 3, (2016), 1.
- [26] Zhao, J., Yu, Y. Brushless DC Motor Fundamentals Application Note. 2011.